Unmasking TRaccoon: A Lattice-Based Threshold Signature with An Efficient Identifiable Abort Protocol

Rafael del Pino[1], Shuichi Katsumata[1,2], Guilhem Niot[1,3], Michael Reichle[4], Kaoru Takemure[1,2]

[1]PQShield
[2]AIST
[3]Univ Rennes, CNRS, IRISA
[4]ETH Zurich

Abstract

TRaccoon is an efficient 3-round lattice-based T-out-of-N threshold signature, recently introduced by del Pino et al. (Eurocrypt 2024). While the design resembles the classical threshold Schnorr signature, Sparkle (Crites et al., Crypto 2023), one shortfall is that it has no means to identify malicious behavior, a property highly desired in practice. This is because to resist lattice-specific attacks, TRaccoon relies on a technique called *masking*, informally blinding each partial signature with a one-time additive mask. del Pino et al. left it as an open problem to add a mechanism to identify malicious behaviors to TRaccoon.

In this work, we propose TRaccoon-IA, a TRaccoon with an efficient *identifiable abort* protocol, allowing to identify malicious signers when the signing protocol fails. The identifiable abort protocol is a simple add-on to TRaccoon, keeping the original design intact, and comes with an added communication cost of $(60 + 6.4 \cdot |T|)$ KB *only when signing fails*. Along the way, we provide the first formal security analysis of a variant of LaBRADOR (Beullens et al., Crypto 2023) with zero-knowledge, encountering several hurdles when formalizing it in detail. Moreover, we give a new game-based definition for *interactive* identifiable abort protocols, extending the popular game-based definition used to prove unforgeability of recent threshold signatures.

This is the full version of a paper to appear in the proceedings of the 45th Annual International Cryptology Conference (CRYPTO 2025). This version includes additional security proofs and a full analysis of the succinct proof system LaBRADOR with Zero-Knowledge that are omitted in the proceedings version.

Contents

1 Introduction

A threshold signature is a multi-party protocol aiming to issue a signature in a distributed manner. Specifically, a T-out-of-N threshold signature enables an arbitrary subset of signers of size T to run an interactive protocol. The protocol messages can then be aggregated into a valid signature for some verification key vk. A secure threshold signature scheme ensures that it is not possible to forge a signature on a fresh message, even if honest protocol executions are observed in the presence of up to $T-1$ malicious signers. Due to the ability to distribute trust, threshold signatures have garnered attention from the cryptographic community, further supported by recent NIST calls [PB23, BD22] for threshold systems.

Post-quantum security ensures the security of a cryptographic primitive against powerful quantum adversaries. Compared to other post-quantum building blocks, lattice-based schemes have attracted considerable attention due to their versatility and efficiency. While many classical primitives have been successfully ported to lattice-based constructions, efficient threshold signatures remained elusive for years. Indeed, until recently, candidates either relied on heavy cryptographic tools such as threshold (fully) homomorphic encryption ((F)HE) or trapdoor homomorphic commitment (HTDC) [BGG+18, DOTT21, DOTT22, ASY22, GKS24].

del Pino et al. [DPKM+24] recently made significant progress, providing an efficient 3-round lattice-based threshold signature, TRaccoon. TRaccoon is a thresholdized construction of the signature scheme Raccoon [dPEK+23, dKPR24], which was submitted to the additional NIST call for signature proposals [NIS22]. The design principle of TRaccoon is similar to the classical 3-round threshold signature Sparkle by Crites et al. [CKM23], based on the Schnorr signature. However, to get around lattice-specific attacks (cf. [DPKM+24, Sec. 2.2]), they introduce a simple yet powerful technique called *masking*. Informally, the signers use a one-time additive mask in a way that allows individual signers to hide their responses while preserving correctness. This technique has already been used in [EKT24, KRT24, BKL+25] to construct variants of TRaccoon with different tradeoffs in round complexity, communication costs, and security.

1.1 Dealing with Malicious Signers

Beyond efficiency, an important property of a threshold signature is *availability*, as highlighted by the recent NIST calls [PB23, BD22]. Availability guarantees the presence of a mechanism to prevent misbehaving signers from arbitrarily causing the signing protocol to fail. Without it, a malicious signer can covertly mount a DoS attack on the set of signers so that no signatures are ever generated.

Robustness. One way to achieve availability is to craft a signing protocol that *always* outputs a valid signature, even in the presence of misbehaving signers. Constructing protocols with this strong notion, known as *robustness* [Har94, GJKR96a, GJKR96b], has been successful in the classical setting, e.g., Schnorr-based [SS01, GJKR07, RRJ+22, BHK+24, Sho23, GS23] and BLS-based [Bol03, LJY14]. In contrast, in the lattice setting, robustness has been an elusive property. Bendlin et al. [BKP13] proposed a robust threshold Gaussian sampling protocol over lattices, leading to the first robust threshold GPV signature [GPV08]. However, this remained purely theoretical due to the heavy reliance on general MPC techniques. To the best of our knowledge, the only practical scheme is Pelican [ENP24]; a robust threshold lattice-based hash-and-sign signature based on Plover [EEN+24] and akin to GPV signatures. While Pelican produces compact signatures, the concrete overhead is significant. Because it requires detecting malicious behavior during the signing protocol, Pelican requires at least $3T$ active signers per signing session, resulting in higher communication overhead compared to TRaccoon, which requires only T active signers (cf. Table 1). If all T signers behave honestly, this overhead incurs a high cost without any benefit.

Identifiable Abort. A meaningful relaxation of robustness is to identify dishonest signers *only when* the signing protocol fails. Constructing protocols with this relaxed notion, known as *identifiable abort* (IA) [GKSŚ20, CGG+20, GG20], has recently become a popular choice for efficient classical threshold signatures, e.g., [KG20, CCL+23, Lin22, CKM23]. The mere existence of such a protocol improves availability, as disruptive signers can be identified after a signing failure and subsequently excluded from future signing protocols. In the post-quantum setting, to the best of our knowledge, the only threshold signature scheme with identifiable abort protocols are by [BGG+18, ASY22, GKS24][1]. The schemes [BGG+18, ASY22] how-

[1] While [BGG+18, ASY22] call their protocol robust, it deviates from the classical definition requiring a valid signature to

Table 1: Comparison with other lattice-based threshold signatures.

Scheme	Q	Size	Signing		Mechanism to identify
			Comm.	|SS|	malicious behavior
Pelican [ENP24] for $T \leqslant 16$	2^{36}	12.3	$26.8 + 56T$	$3T$	Robust
Pelican [ENP24] for $T \leqslant 1024$	2^{48}	26.4	$53.8 + 116T$	$3T$	Robust
TRaccoon	2^{60}	12.7	28.2	T	None
TRaccoon-IA	2^{50}	12.7	28.2	T	IA with $60 + 6.4T$ communication per party

We do not include schemes relying on (F)HE and HS. Q denotes an upper bound on the number of signing sessions, and |SS| denote the number of active signers during signing. "Size" denotes the signature size and "Comm." denotes the communication per party for signing, all given in kilobytes (kB). We assume an arbitrary $T \leqslant 1024$ for TRaccoon-IA as the constant terms remain unchanged.

ever remain impractical due to their reliance on heavy cryptographic tools such as FHE and homomorphic signatures (HS). The scheme [GKS24] appears more efficient as it relies on threshold *linearly* homomorphic encryption (as opposed to *fully* homomorphic). This scheme provides the identification of malicious behavior relying on relatively simple zero-knowledge proofs during the signing protocol. However, it is not still sufficiently efficient in practice compared to TRaccoon because of the heavy use of the homomorphic trapdoor commitment.

1.2 Goal of Our Work

Coming back to TRaccoon, del Pino et al. [DPKM+24] left it as an important future work to add availability to TRaccoon, and so has their successors [EKT24, KRT24, BKL+25]. Ironically, the masking technique, essential for mitigating the aforementioned lattice-specific attacks, presents a key obstacle to identifying malicious signers. Indeed, for classical schemes where no masking is needed, Sparkle [CKM23], a counter part of TRaccoon, and Frost [KG20], a counterpart of the 2-round TRaccoon [EKT24] and Ringtail [BKL+25], support a trivial (non-interactive) identifiable abort protocol. A detailed explanation will be given in Section 1.5.

Given that TRaccoon is currently the most efficient lattice-based threshold signature, and its successors [EKT24, KRT24, BKL+25] rely on the same masking technique, developing a mechanism to identify malicious signers is highly desirable.

1.3 Our Contributions

In this paper, we present TRaccoon-IA, a lattice-based threshold signature scheme with an efficient identifiable abort (IA) protocol. The proposed IA protocol serves as a simple add-on to TRaccoon, preserving its original design and incurring an additional communication cost of $(60 + 6.4 \cdot |T|)$ KB per user *only when signing fails*. Importantly, TRaccoon-IA retains the same efficient signing protocol as TRaccoon. Compared to Pelican [ENP24], which achieves a stronger robustness property, our protocol significantly reduces communication overhead, as shown in Table 1. Even for small thresholds, such as $T = 10$, TRaccoon-IA achieves an order-of-magnitude reduction in communication cost.

Theoretically, adding an IA protocol to any threshold signature is straightforward. Each signer simply publishes a commitment to their partial signing key during key generation. To identify a malicious signer, the signers can prove that they honestly executed the signing protocol with respect to the committed signing key, typically using non-interactive zero-knowledge proofs (NIZKs), as in [GKS24]. Any signer failing to provide a valid proof is identified as misbehaving.

This approach would normally yield an efficient IA protocol if the statement being proved is simple. However, this is not the case for TRaccoon due to its masking technique, which involves a combination of algebraic (*i.e.*, lattice-related) and non-algebraic (*i.e.*, hash-related) operations. Proving such "mixed"

be output even when malicious signers are present during the signing protocol. Their definition is indeed closer to identifiable abort.

relations is highly impractical. We overcome this limitation with a novel approach: instead of naively proving both types of relations simultaneously, we craft a very simple IA protocol proving the non-algebraic relation without relying on NIZKs, allowing us to exclusively rely on efficient NIZKs for algebraic relations. We believe our technique can be applied to other threshold signatures [EKT24, KRT24, BKL+25] relying on the same masking technique. However, as our main focus is on TRaccoon, we leave further investigation to future work.

Along the way, we make two notable side contributions, which may be of independent interests. First, we provide the first formal security analysis of a variant of LaBRADOR [BS23], a lattice-based succinct argument of knowledge (SNARK), with zero-knowledge. Technically, we use the efficient lattice-based (non-succinct) NIZK exact proof LNP [LNP22] to endow LaBRADOR with zero-knowledge by compressing the last LNP prover message via LaBRADOR. While this approach was sketched in prior works [BS23, ADDG24], several hurdles appear when formalizing it in detail. Notably, in our application, the intermediate prover messages of LNP are large, and therefore must also be compressed. Furthermore, analyzing security of the composed protocol requires revisiting the security of LNP in the *predicate special soundness* framework of [AAB+24]. We further prove that we can extract a witness from $n = \mathsf{poly}(\lambda)$ ZK-SNARK proofs simultaneously without incurring an exponential loss. Such *multi-proof* knowledge soundness is key to prove identifiable abort of TRaccoon-IA.

Second, we provide the first game-based definition of an *interactive* IA protocol for threshold signatures, inspired by the one defined in the UC model [CGG+20]. While a definition in the UC model provides stronger guarantees such as composability, a game-based definition is arguably much simpler to understand and handle, allowing to distill unforgeability and identifiable abort as two distinct security goals. Moreover, this aligns with the recent popular choice of game-based unforgeability definitions, e.g., [KG20, BCK+22, CKM23, DPKM+24, BLT+24, EKT24, BKL+25]. Similarly to [CGG+20], our IA definition assume a synchronous authenticated broadcast channel.

It is worth highlighting there exist several game-based IA definitions [BGG+18, ASY22, RRJ+22, BDLR25]. However, these only support a restricted class of threshold signatures admitting a *non-interactive* IA protocol. Moreover, these security games are simplified by implicitly assuming a trusted third-party, say $\mathcal{I}$, to identify the malicious signers. In particular, how $\mathcal{I}$ obtains the inputs required to identify the malicious signers, and whether the view of this input is consistent across the signers, are outside the scope of the model — these practical considerations are inherently captured in the UC model. We note that this is a relatively minor issue for non-interactive IA protocols since the signers can agree on the input to $\mathcal{I}$ relying on a synchronous authenticated broadcast channel, and then locally running a copy of $\mathcal{I}$. However, for interactive IA protocols, such simplification is not possible. See Sections 1.6 and 3 for more details and related works.

1.4 Independent and Concurrent Work

We are aware of a concurrent work [dPENP25] that studied identifiable aborts for TRaccoon simultaneously. Their techniques to achieve identifiable abort are very different from ours and rely on modifying the original protocol with new short secret sharing techniques. In particular, this allows them to avoid the need of a zero-share during signing, and enables partial signature verification and *non-interactive*[2] identifiable aborts. They propose two instantiations for these sharings based on replicated secret sharing or ramp secret sharing. However, the former only works for small T and N (e.g., $N \leqslant 16$) as the size of private keys increases exponentially, while the latter requires the correctness and security threshold to differ, deviating from the standard guarantee of threshold signatures.

1.5 Technical Overview

We explain the high-level overview of TRaccoon-IA. Throughout this section, we omit standard optimization techniques like bit-dropping and focus on the big picture. Below, we first recall TRaccoon by Katsumata, Reichle, and Takemure [KRT24], an optimized variant of TRaccoon by del Pino et al. [DPKM+24].

[2] *Non-interactive* IA protocols refer to schemes where aborts can be identified only from the signing protocol transcript, however still implicitly assuming additional properties from the communication channel as in the *interactive* case. For more detail, see the paragraph before Section 1.4.

An Overview of TRaccoon. TRaccoon is a thresholdized Raccoon [dPEK+23, dKPR24], a variant of Lyubashevsky's lattice-based signature scheme [Lyu09, Lyu12] removing rejection sampling by exploiting the noise flooding technique [GKPV10, ASY22]. The verification key vk consists of a matrix $\mathbf{A} \in \mathcal{R}_q^{k \times \ell}$ and a vector $\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e}$ for short vectors $(\mathbf{s}, \mathbf{e}) \in \mathcal{R}_q^{\ell} \times \mathcal{R}_q^{k}$. The signing key $\mathbf{s}$ is distributed into N shares $[\![\mathbf{s}]\!] = ([\![\mathbf{s}]\!]_i)_{i \in [N]}$ using Shamir's secret sharing so that $\mathbf{s} = \sum_{i \in \mathsf{SS}} L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i$ for any signer set $\mathsf{SS} \in [N]$ of size T where $L_{\mathsf{SS},i}$ is the corresponding Lagrange coefficient. Moreover, each pair of signers i and j share random seeds $(\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})$. The partial secret key of signer i is then $\mathsf{sk}_i = ([\![\mathbf{s}]\!]_i, (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]})$.

The signing protocol proceeds as follows:

Round 1: Each signer $i \in \mathsf{SS}$ generates a commitment $\mathbf{w}_i = \mathbf{A} \cdot \mathbf{r}_i + \mathbf{e}'_i$ by sampling short vectors $\mathbf{r}_i$ and $\mathbf{e}'_i$. It then generates a hash commitment $\mathsf{cmt}_i = \mathsf{H}_{\mathsf{com}}(i, \mathbf{w}_i)$ and outputs it.

Round 2: After receiving cmt_j from all $j \in \mathsf{SS}$, each signer opens the hash commitment by outputting $\mathbf{w}_i$. This deals with rushing adversaries that may otherwise bias the sum $\mathbf{w} = \sum_{j \in \mathsf{SS}} \mathbf{w}_j$ used in **Round 3**.

Round 3: Each signer checks $\mathsf{cmt}_j = \mathsf{H}_{\mathsf{com}}(j, \mathbf{w}_j)$ holds for all signers $j \in \mathsf{SS}$. If not, it aborts the signing protocol. Otherwise, it aggregates $\mathbf{w} = \sum_{j \in \mathsf{SS}} \mathbf{w}_j$, computes the challenge $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, \mathbf{w})$, and a *mask* $\Delta_i = \sum_{j \in \mathsf{SS}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i}$, where $\mathbf{m}_{i,j} = \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$ and $\mathsf{ctnt}_{\mathbf{z}} = \mathsf{SS} \| \mathsf{M} \| (\mathsf{cmt}_j, \mathbf{w}_j)_{j \in \mathsf{SS}}$. Lastly, it outputs a *masked* response $\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \Delta_i$.

The final signatures is $(c, \mathbf{z}, \mathbf{h})$, where $\mathbf{z} = \sum_{j \in \mathsf{SS}} \mathbf{z}_j$ and $\mathbf{h} = \mathbf{w} - \mathbf{A} \cdot \mathbf{z} + c \cdot \mathbf{t}$. We note that if we ignore the mask, the signing protocol is analogous to how Sparkle [CKM23] thresholdizes the Schnorr signature.

The verification algorithm is identical to the non-thresholdized Raccoon and consists of checking $c = \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, \mathbf{A} \cdot \mathbf{z} - c \cdot \mathbf{t} + \mathbf{h})$ and that the norms of $\mathbf{z}$ and $\mathbf{h}$ are short. Correctness immediately follows from Raccoon due to $\sum_{j \in \mathsf{SS}} \Delta_j = \mathbf{0}$. In particular, the seeds $(\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]}$ allow to non-interactively generate an additive secret sharing Δ_i of the value $\mathbf{0} \in \mathcal{R}_q^{\ell}$. As mentioned before, this masking term Δ_i is a crucial component to prevent lattice-specific attacks, abscent in classical schemes (see [DPKM+24, Sec. 2.2] for more discussion).

No Efficient Identifiable Abort. Notice that for a malformed response $\mathbf{z}_i \neq c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \Delta_i$, the aggregated signature no longer verifies.[3] The problem with this is that there is no apparent way to efficiently detect *who* the misbehaving signer was since the mask Δ_i hides all information. The classical analog of TRaccoon (*i.e.*, Sparkle [CKM23]) that does not require masks, sidesteps this issue as it natively supports a trivial IA protocol. Take as example (an insecure version of) an unmasked TRaccoon and assume the response was merely an unmasked $\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i$ and the signer's partial verification key $\mathbf{t}_i = \mathbf{A} \cdot [\![\mathbf{s}]\!]_i + \mathbf{e}_i$ was public. Then, by viewing $\mathbf{z}_i$ as a *partial* signature, we can check its validity by running the verification algorithm with respect to $\mathbf{t}_i$. Unfortunately, the masks Δ_i (and the large Lagrange coefficients $L_{\mathsf{SS},i}$), hinder such a simple IA protocol.

Another natural approach is to use NIZKs. When an aggregated signature does not verify, the signers can prove using an NIZK that they executed the signing protocol honestly. Any signer unable to produce a proof is deemed misbehaving. While simple, this approach is also made difficult due to the way TRaccoon uses masks in the response. Specifically, we need to prove the relations (i) $\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \Delta_i$, (ii) $\Delta_i = \sum_{j \in \mathsf{SS}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i}$, (iii) $\mathbf{m}_{i,j} = \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$, and that (iv) $\mathbf{r}_i$ is short, where everything except for the challenge c, the coefficient $L_{\mathsf{SS},i}$, and the view $\mathsf{ctnt}_{\mathbf{z}}$ are private. This is a heavy mixture of algebraic (i.e., lattice-related) and non-algebraic (i.e., hash-related) relations, for which, to our knowledge, no efficient NIZKs currently exist.

Bypassing the Non-Algebraic Relation. Notice above that if relation (iii) holds, then the relations (i), (ii) and (iv) are only lattice-related relations. Thus, our intermediate goal will be to get around relation (iii) without relying on NIZKs.

First, recall that relation (iii) enforces each pair of signers i and j to compute the same $\mathbf{m}_{i,j}$ and $\mathbf{m}_{j,i}$, which is then used to establish that the masks cancel out as $\sum_{j \in \mathsf{SS}} \Delta_j = \mathbf{0}$. Our main observation is that we never care *how* the vectors $\mathbf{m}_{i,j}$ are generated; we only care that each signer uses the same vector $\mathbf{m}_{i,j}$ to compute their masks Δ_i. As long as the masks cancel out, the other relations guarantee that the

[3] While there are other ways to trigger a failed signing session (e.g., output a wrong opening for the hash commitment cmt), these are common to the classical setting. We thus only focus on the lattice-specific issues in the technical overview.

aggregated signature verifies. Put differently, even if two signers i and j maliciously crafted $\mathbf{m}_{i,j}$ and $\mathbf{m}_{j,i}$ (*i.e.*, relation (iii) does not hold), this won't be an issue so long as they compute Δ_i and Δ_j honestly as those $\mathbf{m}$'s cancel out.

We exploit this fact and consider the following simple idea: in the IA protocol, each signer i computes *lattice-based* commitments $D_{i,j} := \mathsf{Com}(\mathbf{m}_{i,j}; \delta_{i,j})$ and $D_{j,i} := \mathsf{Com}(\mathbf{m}_{j,i}; \delta_{j,i})$ for all $j \in \mathsf{SS} \backslash \{i\}$, where $\delta_{i,j} := \mathsf{H}_{\mathsf{rnd}}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$ is a commitment randomness derived from the seed and view $\mathsf{ctnt}_{\mathbf{z}}$. Since the randomness is derived from $\mathsf{seed}_{i,j}$, honest signers i and j will compute the same commitments $D_{i,j}$ and $D_{j,i}$. Let us consider the case where all the commitments output by the signers are consistent with each other. In this case, we can first rely on the binding property of the commitment to be sure that even misbehaving signers commit to the same $\mathbf{m}$'s.[4] Then, the remaining relations that need to be proven are all lattice-based as desired. Namely, each signer i proves that the committed values in the lattice-based commitments $(D_{i,j}, D_{j,i})_{j \in \mathsf{SS}}$ satisfy relations (i), (ii), and (iv). We later explain how to efficiently construct a lattice-based NIZK for such a relation.

Now, let us consider the other case when there exists $D_{i,j}$ and $D'_{i,j}$ output by signers i and j, respectively, such that $D_{i,j} \neq D'_{i,j}$. It is clear that at least one of the signers is misbehaving since these commitments are deterministic once $\mathsf{seed}_{i,j}$ is fixed. Our idea to detect which of these two are misbehaving is to let the two signers disclose $\mathsf{seed}_{i,j}$ and let the other $T-2$ signers simply recompute the commitment *and* mask $\mathbf{m}_{i,j}$. Namely, we *publicly* check the validity of the relation (iii) $\mathbf{m}_{i,j} = \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$. Clearly, the recomputed commitment can only be equal to one of the $D_{i,j}$ or $D'_{i,j}$, and the signer that outputs the one not matching will be identified as the misbehaving signer.

While the protocol is intuitive, we need to be careful that revealing $\mathsf{seed}_{i,j}$ does not harm the unforgeability of the honest signer, since in general, revealing terms included in the partial signing key may harm security as they are reused in other signing sessions. Fortunately, for our above idea, it will be safe to output $\mathsf{seed}_{i,j}$. The main observation is that $\mathsf{seed}_{i,j}$ is shared by *both* signers i and j. Assuming signer j was the malicious signer, an adversary gains no knowledge when an honest signer i reveals $\mathsf{seed}_{i,j}$ as it already possessed $\mathsf{seed}_{i,j}$.

Lattice-based NIZK. It remains to show that we can efficiently construct an NIZK to the fact that the committed values $\mathbf{m}$'s in the lattice-based commitments $(D_{i,j}, D_{j,i})_{j \in \mathsf{SS}}$ satisfy relations (i) $\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \Delta_i$, (ii) $\Delta_i = \sum_{j \in \mathsf{SS}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i}$, and (iv) $\mathbf{r}_i$ is short. While there has been much progress in the area of lattice-based NIZKs over the last decade, there is no good fit for our specific problem. Ideally, the NIZK is (a) compatible with *compact* commitments D, as there are $|\mathsf{SS}| = 2 \cdot T$ commitments per user, (b) is *succinct* as the witness grows linear in T, (c) is multi-proof extractable[5], and (d) it allows to prove that $\mathbf{r}_i$ is short, ideally with *no slack* to not impact parameters of underlying threshold signature $\mathsf{TRaccoon}$.

Let us give some insight into our choices to achieve (a) to (d) simultaneously. We stress that a careful choice of primitives is vital, else the commitment or proof size quickly becomes impractical. For point (a), we choose Ajtai commitments [Ajt96] to commit to a binary-decomposed $\mathbf{m}$, allowing for a commitment size that is independent of the size of $\mathbf{m}$. This further increases the number of witnesses, stressing the need for a succinct proof system. For this, LaBRADOR [BS23] is a good choice as it is succinct for point (b), compatible with Ajtai commitments, and allows to prove shortness for point (d). However, unfortunately, LaBRADOR is neither zero-knowledge nor known to be multi-proof extractable. On the positive side, recent work [AAB+24] analyzes *single-proof* extraction for LaBRADOR in their *predicate special soundness* (PSS) framework. We extend their analysis and show that any PSS interactive proof is *multi-proof* extractable (when compiled with Fiat-Shamir). This covers point (c), as LaBRADOR is PSS [AAB+24].

Finally, we endow LaBRADOR with zero-knowledge via the exact NIZK LNP [LNP22]. That is, we prove the desired statements (i), (ii), and (iv) in LNP, and then compress the last prover message in LNP via LaBRADOR. This natural approach is already sketched in prior work [BS23, ADDG24], but no formal security proof is given. We therefore analyze security of this approach formally in a modular manner.

[4] The formal proof is more subtle due to our commitments being only *computationally* binding and we refer the readers to Footnote 5 and Section 5.4 for more details.

[5] Multi-proof extraction allows to extract from many proof-statement pairs at once. The reason our proof requires this property is rather technical. Roughly, the IA protocol relies on the fact that given some D, the user is fixed to some mask $\mathbf{m}$. For computationally binding commitments, the commitment itself does not actually fix this value $\mathbf{m}$. For the proof to go through, we need to extract all $\mathbf{m}$'s from T proofs simultaneously. See Section 5.4 for more details.

While zero-knowledge follows immediately, proving soundness is non-trivial. For the latter, we revisit the security of LNP in the PSS framework of [AAB+24]. This requires a delicate and technical analysis under which conditions a witness can be extracted from several proof transcripts, akin to the technical analysis of LaBRADOR in [AAB+24]. Then, PSS security of the composed protocol follows immediately, as PSS is a "composition-friendly" notion. A remaining issue is that the above approach only compresses the last LNP prover message via LaBRADOR. In our application, this is not sufficient to obtain a practical proof size as the intermediate LNP prover messages scale with the number of statements that are proven. Therefore, we further compress the intermediate prover messages via Ajtai commitments to their binary decomposition, similar to the compression mechanism in LaBRADOR. The commitments are then opened in the last prover message and the openings are checked by the verifier. Now, the LNP transcript is independent of the number of statements, except for the last prover's message. As this message is compressed via LaBRADOR, we obtain an efficient lattice-based ZK-SNARK.

1.6 Related Works

Lattice-based Threshold Signatures. Bendlin et al. [BKP13] constructed a threshold signature scheme based on the GPV signature [GPV08] by exploiting generic multi-party computation (MPC). Tang et al. also proposed a threshold signature scheme with key refreshment mechanism by relying on MPC [TPCZ23]. Boneh et al. [BGG+18] proposed the first one-round lattice-based threshold signature scheme by relying on FHE. Subsequently, it was improved by Agrawal et al. [ASY22]. Gur et al. [GKS24] constructed a two-round scheme based on threshold linear homomorphic encryption and homomorphic trapdoor commitment (HTDC) [GVW15, DOTT21]. They also proposed a lattice-based distributed key generation (DKG) protocol for LWE-type keys. del Pino et al. [DPKM+24] proposed a practical three-round scheme TRaccoon without using any heavy tools like FHE and HTDC. Later, Katsumata et al. [KRT24] provided a formal security proof of an optimized variant of this three-round scheme, removing signatures to authenticate the view of the signing protocol, along with a variant achieving adaptive security. Espitau et al. [EKT24] proposed a two-round scheme based on techniques used in [DPKM+24] by introducing a new lattice-based assumption the called algebraic one-more MLWE. Concurrently, Chairattana-Apirom et al. [CATZ24] proposed a two-round scheme based on standard assumptions at the cost of a large signature size. Both schemes achieve two-round protocol with offline-online efficiency in which the first round can be executed before deciding a message and a set of signers. Recently, Boschini et al. [BKL+25] constructed a two-round scheme under standard assumptions. While this scheme is as efficient as [DPKM+24, KRT24], it requires to fix the set of signers in the first round. Espitau et al. [ENP24] proposed a robust DKG and a threshold signature scheme by constructing a verifiable short secret sharing scheme.

Identifiable Abort. Identifiable abort (IA) ensures that malicious signers can be identified when the signing protocol terminates without producing a valid signature. There are numerous classical threshold signatures that support IA such as Schnorr-based threshold signatures [KG20, BCK+22, TZ23, CKM23, RRJ+22, Lin22, BLT+24, BDLR25, Mak22], ECDSA-based threshold signatures [CGG+20, FMM+24, GG20, CCL+23, WMYC23], and BLS-based threshold signature [DR24, BTZ22, LJY14, BL22]. Many of the Schnorr-based [KG20, BCK+22, TZ23, CKM23, RRJ+22, Lin22, BLT+24, BDLR25] and BLS-based [DR24, BTZ22, LJY14, BL22] scheme support partial verification, allowing any party to check a partial signature using a corresponding partial verification key and his own view of the signing protocol. These protocols typically do not explicitly define nor prove to have identifiable abort as partial verification implies an intuitive non-interactive IA protocol. There are also lattice-based threshold signatures that achieve non-interactive IA by relying on FHE and FHS, or HTDC and NIZK [BGG+18, ASY22, GKS24].

There are several works that substantiate their IA claims with a formal security proof. There are game-based identifiable abort (IA) definitions [BGG+18, RRJ+22, DR24, ASY22, BDLR25] and a UC-based IA definition [CGG+20]. Castagnos et al. [CCL+23] and Wong et al. [WMYC23] use game-based definitions to show unforgeability but analyze the security of IA through exhaustive analysis of all possible abort scenarios. All game-based definitions only handle non-interactive IA protocols. We refer to Section 3 for a comparison between our game-based definition handling general (interactive) IA protocols.

Robustness. Robustness ensures that all honest users eventually obtain a valid signature, regardless of some malicious signers taking part in the signing protocol. There are numerous works providing robust

threshold signatures under various assumptions, e.g., DL-based [Har94, GJKR96b, SS01, AF04, GJKR07, RRJ+22, BHK+24, Sho23, GS23, BLSW25], RSA-based [GJKR96a, FGY96, FGMY97, Rab98, Sho00, DD05], ECDSA-based [GKSŚ20], and pairing-based [Bol03, LJY14]. The lattice-based scheme [BKP13] achieve robustness by relying on generic MPC. Recently, Espitau et al. [ENP24] proposed a formal game-based definition of robustness and a practical lattice-based robust threshold signature Pelican without relying on FHE or FHS.

2 Preliminary

2.1 Notations

In this paper, we mostly follow the conventions of [EKT24, KRT24]. We use lower (resp. upper) case bold fonts $\mathbf{v}$ (resp. $\mathbf{M}$) for vectors (resp. matrices). The vectors are in the column form and we use v_i (resp. $\mathbf{m}_i$) to indicate the i-th entry (resp. column) of $\mathbf{v}$ (resp. $\mathbf{M}$). In protocol descriptions, whenever a **req** statement fails, the protocol outputs $\perp$; otherwise, it continues. With an overload in notations, in security game descriptions, **req** means restricting the class of valid adversaries to those not violating the condition.

2.2 Linear Shamir Secret Sharing

We use the *linear Shamir secret sharing* scheme [Sha79] as linear random secret sharing. Let $N < q$ be an integer such that $(i - j)$ is invertible over $\mathbb{Z}_q$ for distinct $i, j \in [N]$ and $S \subseteq [N]$ be a set of cardinality at least T. For a set S and $i \in S$, the Lagrange coefficient $L_{S,i}$ for $i \in S$ is defied by

$$L_{S,i} := \prod_{j \in S \setminus \{i\}} \frac{-j}{i-j}.$$

Let $s \in \mathbb{Z}_q$ be a secret to be shared, and let $P \in \mathbb{Z}_q[X]$ be a degree $T-1$ polynomial such that $P(0) = s$. Given any set of evaluation points $E = \{(i, [\![s]\!]_i)\}_{i \in S}$ such that $[\![s]\!]_i = P(i)$ for all $i \in S$, the secret can be reconstructed as

$$s = \sum_{i \in S} L_{S,i} \cdot [\![s]\!]_i.$$

We extend the notations to secrets in vector form. With a slight abuse of notation, we say $\vec{P} \in \mathbb{Z}_q^{\ell}[X]$ is of degree $T-1$ if each entry of $\vec{P}$ is a degree $T-1$ polynomial. Moreover, $\vec{P}(x)$ denotes the evaluation of each entry of $\vec{P}$ on the point x. When we consider a secret $\mathbf{s} \in \mathcal{R}^{\ell}$, we generate shares $[\![\mathbf{s}]\!] \in \mathbb{Z}_q^{n\ell \times N}$ of $\mathbf{s}$ by interpreting $\mathbf{s}$ as a coefficient vector $\mathbf{s} \in \mathbb{Z}_q^{n\ell}$ and generating secret shares for each entry in parallel. We denote $\mathbb{Z}_q^{n\ell}[X]$ by $\mathcal{R}_q^{\ell}[X]$. Without loss of generality, for $a, s \in \mathcal{R}_q$, $a \cdot [\![s]\!]_i$ means the multiplication over $\mathcal{R}_q$ by interpreting $[\![s]\!]_i$ as a polynomial.

2.3 Lattices, Gaussians, and Rounding

For integers $n, q \in \mathbb{N}$ we define the ring $\mathcal{R}$ as $\mathbb{Z}[X]/(X^n + 1)$ and $\mathcal{R}_q$ as $\mathcal{R}/q\mathcal{R}$. For a positive real σ, let $\rho_\sigma(\mathbf{z}) = \exp\left(-\frac{\|\mathbf{z}\|_2}{2\sigma^2}\right)$. The discrete Gaussian distribution over $S \subseteq \mathcal{R}^k$ and standard deviation σ is defined by its probability distribution function: $\mathcal{D}_{S,\sigma}(\mathbf{z}) = \frac{\rho_\sigma(\mathbf{z})}{\sum_{\mathbf{z}' \in S} \rho_\sigma(\mathbf{z}')}$. We may simply note $\mathcal{D}_\sigma$. If the support S is clear from context, we may simply note $\mathcal{D}_\sigma$ by omitting S. We denote $\mathcal{C} \subset \mathcal{R}_q$ as the set of polynomials with $\{-1, 0, 1\}$-coefficient and fixed hamming weight W, i.e., $\{c \in \mathcal{R}_q \mid \|c\|_\infty = 1 \wedge \|c\|_1 = W\}$.

We recall a useful bound on the norm of discrete Gaussians due to [DPKM+24, Lemma 3.4] which itself is based on [MR04, Lyu12].

Lemma 2.1. *For $\mathbf{s} \xleftarrow{\$} \mathcal{D}_\sigma^k$ and $v \in \mathcal{R}$, we have*

$$\Pr\left[\|v \cdot \mathbf{s}\|_2 \geqslant e^{1/4}\|v\|_1 \sigma \cdot \sqrt{nk}\right] \leqslant 2^{-\frac{nk}{10}}.$$

We recall a convolution result for discrete Gaussians from [MR04, GMPW20].

Lemma 2.2. *Let T be a positive integer lower than n and $\sigma > \sqrt{\frac{\log(2n)+\lambda}{\pi}}$. Then, the distribution of $x := \sum_{i\in T} x_i$ for $x \xleftarrow{\$} \mathcal{D}_\sigma$ is within statistical distance $2^{-\lambda}$ of the distribution $x \xleftarrow{\$} \mathcal{D}_{\sqrt{T}\cdot\sigma}$.*

We also rely on *rounding* for efficiency purpose as in [dPEK+23, DPKM+24]. We provide a brief overview and refer to [DPKM+24] for more details. For a positive integer q and ν such that $q > 2^\nu$, let $q_\nu = \lfloor q/2^\nu \rceil$. We then define the rounding function as follows:

$$\lfloor\cdot\rceil_\nu : \mathbb{Z}_q \mapsto \mathbb{Z}_{q_\nu} \quad \text{s.t.} \quad \lfloor x\rceil_\nu = \lfloor \bar{x}/2^\nu \rceil + q_\nu\mathbb{Z},$$

where $\bar{x} \in [0, 1, \cdots, q-1]$ denotes the canonical unsigned representation of $x \in \mathbb{Z}_q$. The function $\lfloor\cdot\rceil_\nu$ naturally extends to vectors coefficient-wise.

2.4 Hardness Assumptions

We recall some standard lattice-based hardness assumptions and their relations.

Definition 2.1 (MLWE). *Let ℓ, k, q be integers and $\mathcal{D}$ be a probability distribution over $\mathcal{R}_q$. The advantage of an adversary $\mathcal{A}$ against the* Module Learning with Errors $\mathsf{MLWE}_{q,\ell,k,\mathcal{D}}$ *problem is defined as:*

$$\mathsf{Adv}^{\mathsf{MLWE}}_{\mathcal{A}}(\lambda) = \left|\Pr\left[1 \xleftarrow{\$} \mathcal{A}(\mathbf{A}, \mathbf{As}+\mathbf{e})\right] - \Pr\left[1 \xleftarrow{\$} \mathcal{A}(\mathbf{A}, \mathbf{b})\right]\right|$$

where $(\mathbf{A}, \mathbf{b}, \mathbf{s}, \mathbf{e}) \xleftarrow{\$} \mathcal{R}_q^{k\times\ell} \times \mathcal{R}_q^k \times \mathcal{D}^\ell \times \mathcal{D}^k$. The $\mathsf{MLWE}_{q,\ell,k,\mathcal{D}}$ assumption states that any efficient adversary $\mathcal{A}$ has negligible advantage. We may write $\mathsf{MLWE}_{q,\ell,k,\sigma}$ as a shorthand for $\mathsf{MLWE}_{q,\ell,k,\mathcal{D}}$ when $\mathcal{D}$ is the Gaussian distribution of standard deviation σ. Lastly, we also define a variant called uniform MLWE (UMLWE) *where the secret key is sampled from the uniform distribution $\mathcal{R}_q^\ell$.*

Definition 2.2 (MSIS). *Let ℓ, k, q be integers and $\beta > 0$ a real number. The advantage of an adversary $\mathcal{A}$ against the* Module Short Integer Solution $\mathsf{MSIS}_{q,\ell,k,\beta}$ *problem, is defined as:*

$$\mathsf{Adv}^{\mathsf{MSIS}}_{\mathcal{A}}(\lambda) = \Pr\left[\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell}, \mathbf{s} \xleftarrow{\$} \mathcal{A}(\mathbf{A}) : (0 < \|\mathbf{s}\|_2 \leqslant \beta) \ \wedge \ \left[\mathbf{A} \mid \mathbf{I}\right]\mathbf{s} = \mathbf{0} \mod q\right].$$

The $\mathsf{MSIS}_{q,\ell,k,\beta}$ assumption states that any efficient adversary $\mathcal{A}$ has negligible advantage.

Lemma 2.3 (Hardness of MLWE ([LS15])). *Let $k(\lambda)$, $\ell(\lambda)$, $q(\lambda)$, $n(\lambda)$, $\sigma(\lambda)$ such that $q \leqslant \mathsf{poly}(n\ell)$, $k \leqslant \mathsf{poly}(\ell)$, and $\sigma \geqslant \sqrt{\ell}\cdot\omega(\sqrt{\log n})$. If $\mathcal{D}$ is a discrete Gaussian distribution with standard deviation σ, then the $\mathsf{MLWE}_{q,\ell,k,\mathcal{D}}$ problem is as hard as the worst-case lattice Generalized-Independent-Vector-Problem (GIVP) in dimension $N = n\ell$ with approximation factor $\sqrt{8\cdot N\ell}\cdot\omega(\sqrt{\log n})\cdot q/\sigma$.*

Lemma 2.4 (Hardness of MSIS ([LS15])). *For any $k(\lambda), \ell(\lambda), q(\lambda), n(\lambda), \beta(\lambda)$ such that $q > \beta\sqrt{nk}\cdot\omega(\log(nk))$, and $\ell, \log q \leqslant \mathsf{poly}(nk)$. The $\mathsf{MSIS}_{q,\ell,k,\beta}$ problem is as hard as the worst-case lattice Generalized-Independent-Vector-Problem (GIVP) in dimension $N = nk$ with approximation factor $\beta\sqrt{N}\cdot\omega(\sqrt{\log N})$.*

We define a rounded variant of the MLWE problem which will be convenient in our security proofs.

Definition 2.3. *For any $\mathbf{A} \in \mathcal{R}_q^{k\times\ell}$, let $\mathcal{D}^{\mathsf{bd\text{-}MLWE}}_{q,\ell,k,\sigma,\nu}(\mathbf{A})$ be the distribution defined as $\left\{\lfloor\mathbf{A}\cdot\mathbf{s}+\mathbf{e}\rceil_\nu \mid (\mathbf{s},\mathbf{e}) \xleftarrow{\$} \mathcal{D}_\sigma^\ell \times \mathcal{D}_\sigma^k\right\}$. That is, it samples an $\mathsf{MLWE}_{q,\ell,k,\sigma}$ instance with ν bits dropped.*

The following is an immediate application of the regularity lemma [LPR13].

Lemma 2.5. *For any $\sigma > \sqrt{\frac{\log(2n\cdot\max\{\ell,k\})+\lambda}{\pi}}$ and $\sigma > 2n\cdot q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$ and $\nu < \log(q)-2$, the following holds with all but probability $2^{-\lambda}$:*

$$\Pr_{\mathbf{A}\xleftarrow{\$}\mathcal{R}_q^{k\times\ell}}\left[H_\infty\left(\mathcal{D}^{\mathsf{bd\text{-}MLWE}}_{q,\ell,k,\sigma,\nu}(\mathbf{A})\right) \geqslant n-1\right] \geqslant 1-2^{-n+1}.$$

We recall the *hint* MLWE (Hint-MLWE) [KLSS23] and *self-target* MSIS (SelfTargetMSIS) [DPKM+24] assumptions.

Definition 2.4 (Hint-MLWE). *Let ℓ, k, q, Q be integers, $\mathcal{D}, \mathcal{G}$ be probability distributions over $\mathcal{R}_q$, and $\mathcal{C}$ be a set over $\mathcal{R}_q$. The advantage of an adversary $\mathcal{A}$ against the* Hint Module Learning with Errors $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,Q,\mathcal{D},\mathcal{G},\mathcal{C}}$ *problem is defined as:*

$$\mathsf{Adv}^{\mathsf{Hint\text{-}MLWE}}_{\mathcal{A}}(\lambda) = \left|\Pr\left[1 \leftarrow \mathcal{A}\Big(\mathbf{A}, \mathbf{A}\cdot\mathbf{s}+\mathbf{e}, (c_i, \mathbf{z}_i, \mathbf{z}'_i)_{i\in[Q]}\Big)\right] - \Pr\left[1 \leftarrow \mathcal{A}\Big(\mathbf{A}, \mathbf{b}, (c_i, \mathbf{z}_i, \mathbf{z}'_i)_{i\in[Q]}\Big)\right]\right|$$

where $(\mathbf{A}, \mathbf{b}, \mathbf{s}, \mathbf{e}) \xleftarrow{\$} \mathcal{R}_q^{k\times\ell} \times \mathcal{R}_q^k \times \mathcal{D}^\ell \times \mathcal{D}^k$, $c_i \xleftarrow{\$} \mathcal{C}$ for $i \in [Q]$. Moreover, $(\mathbf{z}_i, \mathbf{z}'_i) = (c_i\cdot\mathbf{s}+\mathbf{r}_i, c_i\cdot\mathbf{e}+\mathbf{e}'_i)$ where $(\mathbf{r}_i, \mathbf{e}'_i) \xleftarrow{\$} \mathcal{G}^\ell \times \mathcal{G}^k$ for $i \in [Q]$. The $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,Q,\mathcal{D},\mathcal{G},\mathcal{C}}$ *assumption states that any efficient adversary $\mathcal{A}$ has negligible advantage. We may write* $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,Q,\sigma_\mathcal{D},\sigma_\mathcal{G},\mathcal{C}}$ *as a shorthand when $\mathcal{D}$ and $\mathcal{G}$ are the discrete Gaussian distributions of standard deviation $\sigma_\mathcal{D}$ and $\sigma_\mathcal{G}$, respectively.*

Definition 2.5 (SelfTargetMSIS). *Let ℓ, k, q be integers and $B_{\mathsf{stmsis}} > 0$ be a real number. Let $\mathcal{C}$ be a subset of $\mathcal{R}_q$ and let $\mathsf{H} : \mathcal{R}_q^k \times \{0,1\}^{2\lambda} \to \mathcal{C}$ be a cryptographic hash function modeled as a random oracle. The advantage of an adversary $\mathcal{A}$ against the* Self Target MSIS *problem, noted* $\mathsf{SelfTargetMSIS}_{q,\ell,k,\mathcal{C},B_{\mathsf{stmsis}}}$*, is defined as:*

$$\mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{A}}(\lambda) = \Pr\Big[\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell}, (\mathsf{M}, \mathbf{z}) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(\mathbf{A}) : (\mathsf{M}, \mathbf{z}) \in \{0,1\}^{2\lambda} \times \mathcal{R}_q^{\ell+k} \\ \wedge \left(\mathbf{z} = \begin{bmatrix} c \\ \mathbf{z}' \end{bmatrix}\right) \wedge (\|\mathbf{z}\|_2 \leqslant B_{\mathsf{stmsis}}) \wedge \mathsf{H}\left(\left[\mathbf{A} \mid \mathbf{I}\right]\cdot\mathbf{z},\ \mathsf{M}\right) = c\Big].$$

The $\mathsf{SelfTargetMSIS}_{q,\ell,k,C,B_{\mathsf{stmsis}}}$ *assumption states that any efficient adversary $\mathcal{A}$ has no more than negligible advantage.*

We recall the following MLWE to Hint-MLWE reduction. Below, $s_1(c)$ denotes the spectral norm of $c \in \mathcal{R}_q$ and c^* denotes the Hermitian adjoint of c. Kim et al. [KLSS23] showed that the reduction is tight.

Lemma 2.6 (Hardness of Hint-MLWE [KLSS23]). *For any integers ℓ, k, q, n, Q, set $\mathcal{C} \subset \mathcal{R}_q$, and positive reals $B_{\mathsf{hint}}, \sigma, \sigma_\mathcal{D}, \sigma_\mathcal{G}$ such that $\Pr[s_1(\sum_{i\in[Q]} c_i\cdot(c_i)^*) < B_{\mathsf{hint}} : c_i \leftarrow \mathcal{C}] \geqslant 1 - \mathsf{negl}(\lambda)$, $\sigma = \omega(\sqrt{\log n})$, and $\frac{1}{\sigma^2} = 2\cdot\left(\frac{1}{\sigma_\mathcal{D}^2} + \frac{B_{\mathsf{hint}}}{\sigma_\mathcal{G}^2}\right)$, the* $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,Q,\sigma_\mathcal{D},\sigma_\mathcal{G},\mathcal{C}}$ *problem is as hard as the* $\mathsf{MLWE}_{q,\ell,k,\sigma}$ *problem.*

Lastly, we recall the following MSIS to SelfTargetMSIS reduction. The reduction is a simple invocation of the forking lemma [PS00, BN06], running the adversary against the SelfTargetMSIS problem twice via rewinding. del Pino et al. [DPKM+24] provides a concrete bound on the reduction.

Lemma 2.7 (Hardness of SelfTargetMSIS [DPKM+24]). *Let ℓ, k, q be integers and $B_{\mathsf{stmsis}} > 0$ be a real number. Let $\mathcal{C}$ be a subset of $\mathcal{R}_q$ and let $\mathsf{H} : \mathcal{R}_q^k \times \{0,1\}^{2\lambda} \to \mathcal{C}$ be a cryptographic hash function modeled as a random oracle. For any adversary $\mathcal{A}$ against the* $\mathsf{SelfTargetMSIS}_{q,\ell,k,\mathsf{H},\mathcal{C},B_{\mathsf{stmsis}}}$ *problem making at most Q_H queries to H, there exists an adversary $\mathcal{B}$ against the* $\mathsf{MSIS}_{q,\ell,k,B_{\mathsf{msis}}}$ *problem with $B_{\mathsf{msis}} = 2B_{\mathsf{stmsis}}$ such that*

$$\mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{A}}(\lambda) \leqslant \sqrt{Q_\mathsf{H}\cdot\mathsf{Adv}^{\mathsf{MSIS}}_{\mathcal{B}}(\lambda)} + \frac{Q_\mathsf{H}}{|\mathcal{C}|},$$

where $\mathsf{Time}(\mathcal{B}) \approx 2\cdot\mathsf{Time}(\mathcal{A})$.

2.5 Non-interactive Zero-knowledge Proofs

We consider non-interactive zero-knowledge proofs (NIZK) in the random oracle model (ROM) with common random string $\mathsf{crs} \in \{0,1\}^\ell$. Throughout, for some NP-relation $\mathcal{R}$, we denote by $\mathcal{L}_\mathcal{R} = \{\mathbb{x} \in \{0,1\}^* \mid \exists w \text{ s.t. } (\mathbb{x}, \mathbb{w}) \in \mathcal{R}\}$ the language induced by $\mathcal{R}$. Sometimes, we write $\mathbb{w}_i \in \mathcal{R}(\mathbb{x})$ for $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$.

Definition 2.6. *An NIZK Π for the NP-relation $\mathcal{R}$ consists of PPT algorithms $(\Pi.\mathsf{Prove}, \Pi.\mathsf{Verify})$ with access to random oracle H defined as follows:*

$\Pi.\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, \mathbb{x}, \mathbb{w}) \to \pi$**:** *Given* $\mathsf{crs} \in \{0,1\}^\ell$*, statement* $\mathbb{x}$ *and witness* $\mathbb{w}$ *with* $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$*, it outputs a proof* π*.*

$\Pi.\mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, \mathbb{x}, \pi) \to 0/1$**:** *Given* $\mathsf{crs} \in \{0,1\}^\ell$*, statement* $\mathbb{x}$ *and proof* π*, it outputs a bit.*

We sometimes omit crs as parameter if it is clear by context. Below, we recall the correctness, zero-knowledge and knowledge soundness properties of NIZKs.

Definition 2.7 (Correctness). *An NIZK* Π *is correct, if for any* $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$*, we have*

$$\Pr[\pi \xleftarrow{\$} \Pi.\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, \mathbb{x}, \mathbb{w}) : \mathsf{Verify}^{\mathsf{H}}(\mathsf{crs}, \mathbb{x}, \pi) = 1] = 1 - \mathsf{negl}(\lambda).$$

Definition 2.8 (Zero-knowledge). *An NIZK* Π *is zero-knowledge if there exists a PPT simulator* $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1)$*, where* Sim_0 *and* Sim_1 *have a shared state, such that for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{zk}}_{\mathcal{A},\Pi}(\lambda) := \left|\Pr\left[\mathcal{A}^{\mathsf{H},\mathsf{Prove}}(\mathsf{crs}) = 1\right] - \Pr\left[\mathcal{A}^{\mathsf{Sim}_0,\mathcal{S}}(\mathsf{crs}) = 1\right]\right|,$$

where $\mathsf{crs} \xleftarrow{\$} \{0,1\}^\ell$*, and* Prove *and* $\mathcal{S}$ *are oracles that on input* $(\mathbb{x}, \mathbb{w})$ *return* $\perp$ *if* $(\mathbb{x}, \mathbb{w}) \notin \mathcal{R}$ *and otherwise return* $\mathsf{Prove}^{\mathsf{H}}(\mathsf{crs}, \mathbb{x}, \mathbb{w})$ *or* $\mathsf{Sim}_1(\mathbb{x})$*, respectively.*

Definition 2.9 (Adaptive Multi-Proof Knowledge Soundness). *An NIZK is* N-adaptively knowledge sound *with knowledge error* κ *for relation* $\mathsf{R}_{\mathsf{snd}}$ *if there exists a PPT algorithm* Ext *such that given oracle access to any PPT adversary* $\mathcal{A}$ *(with explicit random tape* ρ*) that makes* $Q_H = \mathsf{poly}(\lambda)$ *random oracle queries with*

$$\Pr[((\mathbb{x}_i, \pi_i)_{i\in[N]}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}(\mathsf{crs}; \rho) : \forall i \in [N], \mathsf{Verify}^{\mathsf{H}}(\mathbb{x}_i, \pi_i) = 1] \geqslant \mu(\lambda),$$

we have

$$\Pr\left[\begin{array}{l} ((\mathbb{x}_i, \pi_i)_{i\in[N]}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}(\mathsf{crs}; \rho), \\ (\mathbb{w}_1, \ldots, \mathbb{w}_N) \leftarrow \mathsf{Ext}^{\mathcal{A}}(\mathsf{crs}, \rho, \mathcal{H}) \end{array} : \forall i \in [N], (\mathbb{x}_i, \mathbb{w}_i) \in \mathsf{R}_{\mathsf{snd}}\right] \geqslant \mu(\lambda) - \kappa(\lambda, Q_H).$$

where $\mathcal{H}$ *are the outputs of* H*, and the probability is over the random tape* ρ *and the random choices of* H *and* Ext*. Also, we require that the runtime of* Ext *is bounded by* $\mathsf{poly}(\lambda, Q_H) \cdot T_{\mathcal{A}}$*, where* $T_{\mathcal{A}}$ *denotes the runtime of* $\mathcal{A}$*.*

2.6 Commitments

A commitment scheme C with message space $\mathcal{M}$, randomness $\mathcal{R}$, and uniform public parameters $\mathsf{pp} \in \{0,1\}^\ell$ of length $\ell(\lambda)$ are PPT algorithms $(\mathsf{C.Com}, \mathsf{C.Verify})$ such that

$\mathsf{C.Com}(\mathsf{pp}, m; r) \to (c, d)$**:** Given the public parameters $\mathsf{pp} \in \{0,1\}^\ell$, message $m \in \mathcal{M}$ and randomness $r \in \mathcal{R}$, it outputs a commitment c to m and opening information d.

$\mathsf{C.Verify}(\mathsf{pp}, c, m, d) \to 0/1$**:** Given the public parameters $\mathsf{pp} \in \{0,1\}^\ell$, commitment c, message $m \in \mathcal{M}$, and opening information d, it outputs a bit b.

We sometimes omit pp as parameter if it is clear by context. Below, we recall the correctness, hiding and binding notions of commitment schemes.

Definition 2.10 (Correctness). *A commitment scheme* C *is correct, if for all* $\mathsf{pp} \in \{0,1\}^\ell$*, messages* $m \in \mathcal{M}$ *and* $r \in \mathcal{R}$*, it holds that* $\mathsf{C.Verify}(\mathsf{pp}, c, m, d) = 1$ *for* $(c, d) = \mathsf{C.Com}(\mathsf{pp}, m; r)$

We define a variant of hiding, where the adversary is given access to a left-or-right commitment oracle. This variant is implied by the variant of hiding, where the adversary only gets a single access to the oracle, which follows via a straightforward hybrid argument.

Definition 2.11 (Hiding). *A commitment scheme* C *is hiding if for any PPT adversaries* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{hide}}_{\mathcal{A},\mathsf{C}}(\lambda) := \left|\Pr\left[\mathcal{A}^{\mathcal{C}_0}(\mathsf{pp}) = 1\right] - \Pr\left[\mathcal{A}^{\mathcal{C}_1}(\mathsf{pp}) = 1\right]\right| = \mathsf{negl}(\lambda),$$

where $\mathsf{pp} \leftarrow \{0,1\}^\ell$ *and the oracle* $\mathsf{C}_b(m_0, m_1)$ *is defined as follows. On input* (m_0, m_1)*,* $\mathcal{C}$ *checks if both* $m_0, m_1 \in \mathcal{M}$*, and outputs* $\perp$ *if not. Else, samples* $r \xleftarrow{\$} \mathcal{R}$*, and outputs* $c \leftarrow \mathsf{Com}(\mathsf{pp}, m_b; r)$*.*

Definition 2.12 (Binding). *A commitment scheme* C *is* binding *if for any PPT adversary* $\mathcal{A}$*, we have*

$$\mathsf{Adv}^{\mathsf{bind}}_{\mathcal{A},\mathsf{C}}(\lambda) := \Pr\left[\begin{array}{ll} \mathsf{pp} \xleftarrow{\$} \{0,1\}^\ell, & \\ (c, m_0, m_1, d_0, d_1) \leftarrow \mathcal{A}(\mathsf{pp}) & : \begin{array}{l}\forall b \in \{0,1\}, \mathsf{C.Verify}(\mathsf{pp}, C, m_b, d_b) = 1 \\ \wedge m_0 \neq m_1\end{array}\end{array}\right] = \mathsf{negl}(\lambda).$$

3 Threshold Signatures with Identifiable Abort

In this section, we give a new game-based definition of threshold signatures with a general identifiable abort (IA) protocol.

3.1 Syntax

We fist provide our syntax of a R-round threshold signature with an IA protocol. Below, N, T, and T_{ia} denote the number of total signers, the number of signers required to produce a signature, and the number of parties required to run the IA protocol, respectively. We assume $T \leqslant T_{\mathsf{ia}} \leqslant N$. Moreover, SS (resp., IAS) denotes the set of signers in $[N]$ of size T (resp. T_{ia}). Lastly, each signing session is assigned a unique session identifier sid, used by both the signing and IA protocols. Each signer is assumed to maintain a state st_i where $\mathsf{st}_i[\mathsf{sid}]$ is the state corresponding to session sid.

$\mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}}) \to \mathsf{tspar}$**:** The setup algorithm takes as input a security parameter 1^λ, N, T, and T_{ia} and outputs public parameters tspar. We assume that tspar includes N, T, and T_{ia}.

$\mathsf{KeyGen}(\mathsf{tspar}) \to (\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf})$**:** The key generation algorithm takes tspar as input and outputs a verification key vk, secret key shares $(\mathsf{sk}_i)_{i\in[N]}$, and the public information for IA protocol inf [6]. It implicitly sets up an empty state $\mathsf{st}_i := \varnothing$ for all N signers. We assume vk includes tspar.

$\mathsf{Sign} = (\mathsf{Sign}_1, \cdots, \mathsf{Sign}_R, \mathsf{Agg})$**:** The signing protocol of a R-round threshold signature consists of the following $(R+1)$ algorithms.

- $\mathsf{Sign}_r(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(r-1)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i) \to (\mathsf{pm}_i^{(r)}, \mathsf{st}_i)$**:** The signing algorithm for the r-th round for $r \in [R]$ takes as input vk, SS, sid, a message M, an index i of a signer, a tuple of protocol messages of the $(r-1)$-th round $(\mathsf{pm}_j^{(r-1)})_{j\in\mathsf{SS}}$, sk_i, and st_i and outputs a protocol message $\mathsf{pm}_i^{(r)}$ and an updated state st_i. We assume that $(\mathsf{pm}_j^{(0)})_{j\in\mathsf{SS}}$ is $\perp$. If round r can be executed prior to choosing SS and/or M, Sign_r does not take them as input. When Sign_r aborts, it outputs $(\mathsf{pm}_i^{(r)} = \perp, \mathsf{st}_i)$.
- $\mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, (\mathsf{pm}_i^{(r)})_{r\in[R], i\in\mathsf{SS}}) \to \mathsf{sig}$**:** The aggregation algorithm takes as input vk, SS, sid, M, and a tuple of protocol messages $(\mathsf{pm}_i^{(r)})_{r\in[R], i\in\mathsf{SS}}$ and outputs a signature sig.

$\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig}) \to 0/1$**:** The verification algorithm takes as input vk, M, and sig and outputs 1 if sig is valid and 0 otherwise.

$\mathsf{IA} = (\mathsf{IA}_{\mathsf{rnd}_\perp,1}, \cdots, \mathsf{IA}_{\mathsf{rnd}_\perp, R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}})_{\mathsf{rnd}_\perp\in[R+1]}$**:** We define separate IA protocols, one per signing round and one for aggregation. If the signing protocol aborts in round $\mathsf{rnd}_\perp$, then the $\mathsf{rnd}_\perp$-th protocol allows to detect the malicious signer(s) that caused the abort. If $\mathsf{rnd}_\perp = R+1$, then an abort occurred in aggregation (*i.e.*, verification fails). The IA protocol for round $\mathsf{rnd}_\perp$ consists of the following $R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}$ algorithms.

[6] Public information inf contains additional information that enables parties to prove honest behavior, for instance, partial verification keys or commitments to sk_i.

$\mathsf{IA}_{\mathsf{rnd}_\perp,1}(\mathsf{vk},\mathsf{inf},\mathsf{SS},\mathsf{IAS},\mathsf{sid},\mathsf{M},i,(\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[\mathsf{rnd}_\perp]},\mathsf{sk}_i,\mathsf{st}_i) \to (\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i},\mathsf{st}_i)$**:** The algorithm for the first round of the IA protocol for signing round $\mathsf{rnd}_\perp$ takes as input vk, inf, SS, IAS, sid, M, an index i of a signer, a tuple of protocol messages $(\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[\mathsf{rnd}_\perp]}$ for the signing protocol, sk_i, and st_i and outputs a protocol message $\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i}$ and an updated state st_i. For $R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}=1$, it outputs $\mathsf{CS}_i^{\mathsf{IA}}$ as $\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i}$ where $\mathsf{CS}_i^{\mathsf{IA}}$ is a set of detected malicious signers. If $\mathsf{rnd}_\perp = R+1$, we suppose that $\mathsf{pm}_j^{(\mathsf{rnd}_\perp)}$ is set to $\perp$.

$\mathsf{IA}_{\mathsf{rnd}_\perp,r}(\mathsf{vk},\mathsf{inf},\mathsf{SS},\mathsf{IAS},\mathsf{sid},\mathsf{M},i,(\mathsf{pmia}^{(r-1)}_{\mathsf{rnd}_\perp,j})_{j\in\mathsf{IAS}},\mathsf{sk}_i,\mathsf{st}_i) \to (\mathsf{pmia}^{(r)}_{\mathsf{rnd}_\perp,i},\mathsf{st}_i)$**:** For $r\in[2,R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}]$, the r-th round of the IA protocol for signing round $\mathsf{rnd}_\perp$ takes as input vk, inf, SS, IAS, sid, M, an index i of a signer, a tuple of protocol messages of the $(r-1)$th round $(\mathsf{pmia}^{(r-1)}_{\mathsf{rnd}_\perp,j})_{j\in\mathsf{IAS}}$, sk_i, and st_i and outputs a protocol message $\mathsf{pmia}^{(r)}_{\mathsf{rnd}_\perp,i}$ and an updated state st_i. For $r=R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}$, it outputs $\mathsf{CS}_i^{\mathsf{IA}}$ as $\mathsf{pmia}^{(R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)})}_{\mathsf{rnd}_\perp,i}$.

3.2 Correctness

Below, we define the correctness of a R-round threshold signature scheme.

Definition 3.1 (Correctness). *We say that a R-round threshold signature scheme with identifiable abort TS satisfies correctness if, for all $\lambda\in\mathbb{N}$, $N,T,T_{\mathsf{ia}}\in\mathsf{poly}(\lambda)$ s.t. $T\leqslant T_{\mathsf{ia}}\leqslant N$, message M, session identifier sid, and $\mathsf{SS}\subseteq[N]$ s.t. $|\mathsf{SS}|=T$, the following respectively hold:*

$$\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda,N,T,T_{\mathsf{ia}},\mathsf{M},\mathsf{sid},\mathsf{SS})=1\right]\geqslant 1-\mathsf{negl}(\lambda),$$

where $\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}$ are shown in Fig. 1.

$\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda,N,T,T_{\mathsf{ia}},\mathsf{M},\mathsf{sid},\mathsf{SS})$

1 : **for** $i\in\mathsf{SS}$ **do** $\mathsf{st}_i[\mathsf{sid}]:=\varnothing$
2 : $\mathsf{tspar}\xleftarrow{\$}\mathsf{Setup}(1^\lambda,N,T,T_{\mathsf{ia}})$
3 : $(\mathsf{vk},(\mathsf{sk}_i)_{i\in[N]},\mathsf{inf})\xleftarrow{\$}\mathsf{KeyGen}(\mathsf{tspar})$
4 : **for** $i\in\mathsf{SS}$ **do** $\mathsf{pm}_i^{(0)}:=\perp$
5 : **for** $r\in[R]$ **do**
6 : **for** $i\in\mathsf{SS}$ **do**
7 : $(\mathsf{pm}_i^{(r)},\mathsf{st}_i)\xleftarrow{\$}\mathsf{Sign}_r(\mathsf{vk},\mathsf{SS},\mathsf{sid},\mathsf{M},i,(\mathsf{pm}_j^{(r-1)})_{j\in\mathsf{SS}},\mathsf{sk}_i,\mathsf{st}_i)$
8 : **if** $\mathsf{pm}_i^{(r)}=\perp$ **then**
9 : **return** 0
10 : $\mathsf{sig}\xleftarrow{\$}\mathsf{Agg}(\mathsf{vk},\mathsf{SS},\mathsf{sid},\mathsf{M},(\mathsf{pm}_i^{(r)})_{r\in[R],i\in\mathsf{SS}})$
11 : **return** $\mathsf{Verify}(\mathsf{vk},\mathsf{M},\mathsf{sig})$

Figure 1: Correctness game for a R-round threshold signature scheme.

3.3 Unforgeability

Unforgeability guarantees that the threshold signature remains secure even against $T-1$ corrupted signers. The definition is similar to prior definitions, e.g., [DPKM+24], with the key difference being that the adversary can run the IA protocol in the security game. In particular, the scheme must remain secure even if the adversary learns information through the IA protocol.

The formal definition is defined via a game in Fig. 2. The adversary is allowed to query the signing oracles $(\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}$, the aggregation oracle $\mathcal{O}_{\mathsf{Agg}}$, and IA oracles $(\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\perp,r}})_{\mathsf{rnd}_\perp\in[R+1],r\in[R_{\mathsf{ia}}^{(\mathsf{rnd}_\perp)}]}$, where oracle $\mathcal{O}_{\mathsf{Agg}}$ is required to deal with invalid (aggregated) signatures. While the definition may seem daunting on first glance, the complexity only comes from the extra book keeping required by the IA protocol. For reference, we provide a brief description of each oracles below, where we ignore $\mathcal{O}_{\mathsf{Sign}_1}$ due to its simplicity.

Signing Oracles for $r > 1$: It checks whether the signing protocol aborts before invoking the signing algorithms of honest signers. If so, it stores the index of the round and view into the tables $\mathsf{Round}_\perp[\mathsf{sid}]$ and $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}]$, respectively, and terminates the signing protocol by returning $\perp$. These tables are used inside the IA oracles to check if the the signing sessions correspond to aborted IA queries, and to run the IA algorithms. If not, it continues the signing protocol similarly to existing definitions.

Aggregation Oracle: It first checks if the final round aborts. If so, it stores the index R (corresponding to the final round) and the view into the tables $\mathsf{Round}_\perp[\mathsf{sid}]$ and $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}]$, respectively. If not, it generates an aggregated signature and verifies it. If it is invalid, it stores the index $R+1$ and the view. As this is only for book keeping of the game, it always returns $\perp$.

IA Oracles: The oracle associated to the first round checks whether the signing session corresponding to sid aborted and whether IAS satisfies some conditions. If so, it starts the IA protocol. For oracles after the first round, it executes each round of the IA protocol. The outputs of honest signers are stored in the table $\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}]$.

Formally, we define unforgeability as follows.

Definition 3.2 (Unforgeability). *Let* TS *be an* R*-round threshold signature scheme. The advantage of an adversary* $\mathcal{A}$ *against the unforgeability of* TS *is defined as*

$$\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) = \Pr[\mathsf{Game}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) = 1],$$

where $\mathsf{Game}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}$ *is described in Fig. 2. We say that* TS *is unforgeable in the random oracle model if, for all* $\lambda \in \mathbb{N}$, $N, T, T_{\mathsf{ia}} \in \mathsf{poly}(\lambda)$ *s.t.* $T \leqslant T_{\mathsf{ia}} \leqslant N$ *and PPT adversary* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) \leqslant \mathsf{negl}(\lambda)$ *holds.*

Communication Channel. We note that when an adversary queries the signing oracle $\mathcal{O}_{\mathsf{Sign}_r}$ for $r \in [2, R]$, the game executes the r-th round signing algorithm of *all* the honest signers. This is in contrast to prior game-based definitions, e.g., [KG20, BCK+22, CKM23, DPKM+24, BLT+24, EKT24, BKL+25], allowing the adversary to individually invoke each signer. This reflects the difference in the assumed communication channel: while we assume a *synchronous authenticated broadcast* channel, prior works assume an *asynchronous* channel. Informally, the former assumes a broadcast channel where the same message is guaranteed to arrive at all signers within a fixed time window. In contrast, the latter assumes the messages to only arrive *eventually*.[7] While the asynchronous setting models real-world channels more faithfully, identifiable abort turns out to be only possible in the synchronous setting [Sho23, BHK+24]. This is because in an asynchronous setting, we cannot tell the difference between a signer that is misbehaving by staying silent and a signer that is just slow or temporarily disconnected.[8] Indeed, this is the communication channel assumed in the prior UC model IA definition [CGG+20]. We emphasize that our threshold signature $\mathsf{TRaccoon\text{-}IA}$ remains secure even under the previous definition of unforgeability that assumes an asynchronous channel. This is because the signing protocol is identical to $\mathsf{TRaccoon}$ [DPKM+24].

Remark 3.1 (Unforgeability for Threshold Signature Scheme without Identifiable Abort). To show the security of our proposed scheme $\mathsf{TRaccoon\text{-}IA}$ based on $\mathsf{TRaccoon}$ [DPKM+24, KRT24], we will rely on the security of $\mathsf{TRaccoon}$. Although the syntax and security definitions in this paper are tailored to TS

[7] We note that whether the channel is authenticated or not is a minor issue as we can always authenticate it, relying on a standard signature scheme.

[8] There are some game-based definitions on non-interactive identifiable abort that assume an asynchronous channel. As explained in Section 1.3, these definitions circumvent the impossibility by implicitly assuming some trusted third-party to identify the malicious signer.

$\mathsf{Game}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}})$

$\mathsf{Q}_{\mathsf{M}} := \emptyset, \mathsf{Q}_{\mathsf{Sign}}[\cdot] := \emptyset, \mathsf{Q}_{\mathsf{ia}}[\cdot] := \emptyset, \mathsf{Round}_\perp[\cdot] := \emptyset$
$\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}})$
$(\mathsf{CS}, \mathsf{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(\mathsf{tspar})$
req $(\mathsf{CS} \subseteq [N]) \wedge (|\mathsf{CS}| \leqslant T-1)$
$\mathsf{HS} := [N] \backslash \mathsf{CS}$
for $i \in \mathsf{HS}$ **do** $\mathsf{st}_i := \emptyset$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$
$\mathsf{oracles} := \Big((\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}, \mathcal{O}_{\mathsf{Agg}}, (\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\perp,r}})_{\substack{\mathsf{rnd}_\perp\in[R+1],\\ r\in[R^{(\mathsf{rnd}_\perp)}_{\mathsf{ia}}]}}, \mathsf{H}\Big)$
$(\mathsf{sig}^*, \mathsf{M}^*) \xleftarrow{\$} \mathcal{A}^{\mathsf{oracles}}(\mathsf{vk}, \mathsf{inf}, (\mathsf{sk}_i)_{i\in\mathsf{CS}}, \mathsf{st}_{\mathcal{A}})$
req $(\mathsf{M}^* \notin \mathsf{Q}_{\mathsf{M}})$
return $\mathsf{Verify}(\mathsf{vk}, \mathsf{M}^*, \mathsf{sig}^*)$

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{SS}, \mathsf{M}, \mathsf{sid})$

req $(\mathsf{SS} \subseteq [N]) \wedge (|\mathsf{SS}| = T) \wedge (\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] = \emptyset)$
$\mathsf{Q}_{\mathsf{M}} \leftarrow \mathsf{Q}_{\mathsf{M}} \cup \{\mathsf{M}\}$
for $i \in \mathsf{HS} \cap \mathsf{SS}$ **do**
  $(\mathsf{pm}_i^{(1)}, \mathsf{st}_i) \xleftarrow{\$} \mathsf{Sign}_1(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, \perp, \mathsf{sk}_i, \mathsf{st}_i)$
$\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (1, \mathsf{SS}, \mathsf{M}, \perp, (\mathsf{pm}_i^{(1)})_{i\in\mathsf{HS}\cap\mathsf{SS}})$
return $(\mathsf{pm}_i^{(1)})_{i\in\mathsf{HS}\cap\mathsf{SS}}$

$\mathcal{O}_{\mathsf{Sign}_r}(\mathsf{sid}, (\mathsf{pm}_i^{(r-1)})_{i\in\mathsf{CS}\cap\mathsf{SS}})$ // $r \in [2, R]$.

req $(\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] = (r-1, \cdot, \cdot, \cdot, \cdot)) \wedge (\mathsf{Round}_\perp[\mathsf{sid}] = \perp)$
parse $(r-1, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[r-2]}, (\mathsf{pm}_i^{(r-1)})_{i\in\mathsf{HS}\cap\mathsf{SS}}) \leftarrow \mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}]$
  // Checking if abort occurs in a previous round.
if $\exists i \in \mathsf{SS}, \mathsf{pm}_i^{(r-1)} = \perp$ **then**
  $\mathsf{Round}_\perp[\mathsf{sid}] \leftarrow r-1$
  $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (r-1, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[r-1]}, \perp)$
  return $\perp$
for $i \in \mathsf{HS} \cap \mathsf{SS}$ **do**
  $(\mathsf{pm}_i^{(r)}, \mathsf{st}_i) \xleftarrow{\$} \mathsf{Sign}_r(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(r-1)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$
$\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (r, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[r-1]}, (\mathsf{pm}_i^{(r)})_{i\in\mathsf{HS}\cap\mathsf{SS}})$
return $(\mathsf{pm}_i^{(r)})_{i\in\mathsf{HS}\cap\mathsf{SS}}$

$\mathcal{O}_{\mathsf{Agg}}(\mathsf{sid}, (\mathsf{pm}_i^{(R)})_{i\in\mathsf{CS}\cap\mathsf{SS}})$ // Aborting in final round

req $(\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] = (R, \cdot, \cdot, \cdot, \cdot)) \wedge (\mathsf{Round}_\perp[\mathsf{sid}] = \perp)$
parse $(R, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[R-1]}, (\mathsf{pm}_i^{(R)})_{i\in\mathsf{HS}\cap\mathsf{SS}}) \leftarrow \mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}]$
if $\exists i \in \mathsf{SS}, \mathsf{pm}_i^{(R)} = \perp$ **then**
  $\mathsf{Round}_\perp[\mathsf{sid}] \leftarrow R$
  $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (R, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[R]}, \perp)$
  return $\perp$
$\mathsf{sig} \leftarrow \mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, (\mathsf{pm}_i^{(r)})_{r\in[R], i\in\mathsf{SS}})$
if $\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig}) = 0$ **then**
  for $i \in \mathsf{SS}$ **do** $\mathsf{pm}_i^{(R+1)} := \perp$
  $\mathsf{Round}_\perp[\mathsf{sid}] \leftarrow R+1$
  $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (R+1, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[R+1]}, \perp)$
return $\perp$

$\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\perp,1}}(\mathsf{sid}, \mathsf{IAS})$

req $(\mathsf{Round}_\perp[\mathsf{sid}] \neq \perp) \wedge (|\mathsf{IAS}| = T_{\mathsf{ia}})$
parse $(\mathsf{rnd}_\perp, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[\mathsf{rnd}_\perp]}, \perp) \leftarrow \mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}]$
  // $\mathsf{Round}_\perp[\mathsf{sid}] = \mathsf{rnd}_\perp$ holds.
req $(\mathsf{SS} \subseteq \mathsf{IAS} \subseteq [N])$
  // There is at least one honest user in IAS.
for $i \in \mathsf{HS} \cap \mathsf{IAS}$ **do**
  $(\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i}, \mathsf{st}_i)$
    $\xleftarrow{\$} \mathsf{IA}_{\mathsf{rnd}_\perp,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_i^{(j)})_{i\in\mathsf{SS}, j\in[\mathsf{rnd}_\perp]}, \mathsf{sk}_i, \mathsf{st}_i)$
$\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}] \leftarrow (1, \mathsf{rnd}_\perp, \mathsf{SS}, \mathsf{IAS}, \mathsf{M}, (\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i})_{i\in\mathsf{HS}\cap\mathsf{IAS}})$
return $(\mathsf{pmia}^{(1)}_{\mathsf{rnd}_\perp,i})_{i\in\mathsf{HS}\cap\mathsf{IAS}}$

$\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\perp,r}}(\mathsf{sid}, (\mathsf{pmia}^{(r-1)}_{\mathsf{rnd}_\perp,j})_{j\in\mathsf{CS}\cap\mathsf{IAS}})$ // $r \in [2, R^{(\mathsf{rnd}_\perp)}_{\mathsf{ia}}]$ for $R^{(\mathsf{rnd}_\perp)}_{\mathsf{ia}} > 1$.

req $(\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}] = (r-1, \cdot, \cdot, \cdot, \cdot, \cdot))$
parse $(r-1, \mathsf{rnd}_\perp, \mathsf{SS}, \mathsf{IAS}, \mathsf{M}, (\mathsf{pmia}^{(r-1)}_{\mathsf{rnd}_\perp,i})_{i\in\mathsf{HS}\cap\mathsf{IAS}}) \leftarrow \mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}]$
for $i \in \mathsf{HS} \cap \mathsf{IAS}$ **do**
  $(\mathsf{pmia}^{(r)}_{\mathsf{rnd}_\perp,i}, \mathsf{st}_i)$
    $\xleftarrow{\$} \mathsf{IA}_{\mathsf{rnd}_\perp,r}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pmia}^{(r-1)}_{\mathsf{rnd}_\perp,j})_{j\in\mathsf{IAS}}, \mathsf{sk}_i, \mathsf{st}_i)$
$\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}] \leftarrow (r, \mathsf{rnd}_\perp, \mathsf{IAS}, \mathsf{M}, (\mathsf{pmia}^{(r)}_i)_{i\in\mathsf{HS}\cap\mathsf{IAS}})$
return $(\mathsf{pmia}^{(r)}_i)_{i\in\mathsf{HS}\cap\mathsf{IAS}}$

Figure 2: The unforgeability game for an R-round threshold signature with identifiable abort. In the above, H denotes the random oracle.

scheme supporting IA protocol, when we consider the TS schemes without IA, we implicitly fix $T_{\mathsf{ia}} = \perp$ and consider the slightly modified unforgeability game in which the IA oracle always outputs $\perp$.

Note that signing oracle in our definition is slightly different from the one used in [DPKM+24, KRT24]. Specifically, the signing oracle in the above unforgeability game executes the signing algorithm for all honest users in SS. This is to ensure that $\mathcal{A}$ cannot control contributions for honest users and is sometimes used when considering robustness [ENP24]. In contrast, in the security game of [KRT24], the signing oracle is only for an honest user at a time. This type of definition is often used for TS schemes without robustness, e.g., [BCK+22, CKM23, DPKM+24, EKT24, KRT24]. Since the latter game puts less restrictions on the adversary's queries, security in this model implies security in our model.

Indeed, if there is an adversary $\mathcal{A}$ against our unforgeability game making at most Q_{S} signing queries, there is an adversary against the latter unforgeability game making at most $T \cdot Q_{\mathsf{S}}$ signing queries without an additional reduction loss. We will use this fact when we show the security of TRaccoon-IA.

3.4 Identifiable Abort

Identifiable abort (IA) guarantees that all honest signers can detect and agree on one or more misbehaving signers when the signing protocol aborts. Our game-based definition is inspired by the UC-based definition of threshold signatures with identifiable abort [CGG+20], and is defined via a game in Fig. 3.

The game proceeds similarly to unforgeability, with the only difference being the winning condition. Recall that an IA protocol needs to first ensure that all honest signers agree with the same subset of signers. Moreover, the subset should include at least one corrupted signer but not any honest signer. If any of these guarantees are not met, then an adversary $\mathcal{A}$ is said to win.

Formally, in our game, $\mathcal{A}$ runs identically as in the unforgeability game, but it outputs a session identifier sid^*, as opposed to a forgery at the end of the game. The challenger checks that the IA protocol associated with sid^* is completed up to the final round. $\mathcal{A}$ wins if any of the following three conditions are met, where HS (resp. CS) denotes the set of honest (resp. corrupt) signers.

Condition 1. There are distinct honest signers $i, j \in \mathsf{HS} \cap \mathsf{IAS}$ that disagree on the set of identified signers, *i.e.*, $\mathsf{CS}_i^{\mathsf{IA}} \neq \mathsf{CS}_j^{\mathsf{IA}}$.

Condition 2. Some signer $i \in \mathsf{HS} \cap \mathsf{IAS}$ fails to identify any malicious signer that caused the abort, *i.e.*, $\mathsf{CS}_i^{\mathsf{IA}} = \varnothing$.

Condition 3. An honest signer $i \in \mathsf{HS} \cap \mathsf{IAS}$ is wrongly identified as responsible for the abort, *i.e.*, $\mathsf{CS}_i^{\mathsf{IA}} \nsubseteq \mathsf{CS}$.

It is worth noting that prior game-based definitions for non-interactive IA protocols [BGG+18, ASY22, RRJ+22, BDLR25] are defined in such a way that Condition 1 always hold (cf. Footnote 8). This is the main reason why no synchronous broadcast channels are required. In contrast, as we consider non-interactive IA protocols, we cannot make this simplification. We will rely on a synchronous broadcast channel to prove Condition 1, enforcing the honest signers' views to be consistent.

Definition 3.3 (Identifiable Abort). *Let* TS *be an* R*-round threshold signature scheme with identifiable abort. The advantage of an adversary* $\mathcal{A}$ *against identifiable abort is defined as*

$$\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) = \Pr[\mathsf{Game}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) = 1],$$

where $\mathsf{Game}^{\mathsf{ts\text{-}ia}}$ *is described in Fig. 3. We say that* TS *is identifiable abort in the random oracle model if, for all* $\lambda \in \mathbb{N}$, $N, T \in \mathsf{poly}(\lambda)$ *with* $T \leqslant T_{\mathsf{ia}} \leqslant N$ *and PPT adversaries* $\mathcal{A}$, $\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) \leqslant \mathsf{negl}(\lambda)$ *holds.*

Remark 3.2 (Correctness from Identifiable Abort). If a threshold signature scheme is identifiable abort, it must be correct. Indeed, if the adversary $\mathcal{A}$ behaves honestly but verification of the aggregated signature fails, then an abort is caused despite all users behaving honestly. Thus, no misbehavior can be identified and $\mathcal{A}$ wins. The formal proof is provided in Lemma A.1.

4 TRaccoon-IA: TRaccoon with Identifiable Abort

We present TRaccoon-IA, a lattice-based threshold signature scheme with an efficient identifiable abort (IA) protocol. Below, we explain how the key generation, signing, and IA protocols work in detail, where the complete protocol is given in Figs. 4 and 5. We refer the readers to Section 1.5 for the overview.

4.1 The Key Generation and Signing Protocols

The signing protocol of TRaccoon-IA is identical to TRaccoon [KRT24, DPKM+24]. The only difference is that algorithm KeyGen outputs public information inf used by the IA protocol. This modification is necessary to bind the signers to a specific partial signing key. The key generation and signing protocols of TRaccoon-IA are given in Fig. 4, where the difference between TRaccoon is highlighted in blue. See Table 2 for an overview of the parameters we use.

$\mathsf{Game}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}})$

1 : $\mathsf{Q}_{\mathsf{Sign}}[\cdot] := \varnothing, \mathsf{Q}_{\mathsf{ia}}[\cdot] := \varnothing, \mathsf{Round}_\perp[\cdot] := \varnothing$ // Q_{M} is unnecessary

2 : $\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}})$

3 : $(\mathsf{CS}, \mathsf{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(\mathsf{tspar})$

4 : **req** $(\mathsf{CS} \subseteq [N]) \wedge (|\mathsf{CS}| \leqslant T-1)$

5 : $\mathsf{HS} := [N] \backslash \mathsf{CS}$

6 : **for** $i \in \mathsf{HS}$ **do** $\mathsf{st}_i := \varnothing$

7 : $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$

// All Oracles except $\mathcal{O}_{\mathsf{Sign}_1}$ are the same as in $\mathsf{Game}^{\mathsf{ts\text{-}uf}}$

8 : $\mathsf{oracles} := \left((\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}, \mathcal{O}_{\mathsf{Agg}}, (\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\perp,r}})_{\substack{\mathsf{rnd}_\perp\in[R+1],\\ r\in[R^{(\mathsf{rnd}_\perp)}_{\mathsf{ia}}]}}, \mathsf{H}\right)$

9 : $\mathsf{sid}^* \xleftarrow{\$} \mathcal{A}^{\mathsf{oracles}}(\mathsf{vk}, \mathsf{inf}, (\mathsf{sk}_i)_{i\in\mathsf{CS}}, \mathsf{st}_{\mathcal{A}})$

10 : **req** $\left(\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}^*] = (R^{(\mathsf{rnd}^*_\perp)}_{\mathsf{ia}}, \mathsf{rnd}^*_\perp, \cdot, \cdot, \cdot)\right)$

11 : **parse** $(R^{(\mathsf{rnd}^*_\perp)}_{\mathsf{ia}}, \mathsf{rnd}^*_\perp, \mathsf{SS}^*, \mathsf{IAS}^*, \mathsf{M}^*, (\mathsf{CS}^*_i)_{i\in\mathsf{HS}\cap\mathsf{IAS}}) \leftarrow \mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}^*]$

12 : $\mathsf{sHS}^* \leftarrow \mathsf{HS} \cap \mathsf{IAS}^*$

13 : **return** $\left(\exists i, j \in \mathsf{sHS}^*, \mathsf{CS}^*_i \neq \mathsf{CS}^*_j\right) \vee \left(\exists i \in \mathsf{sHS}^*, \mathsf{CS}^*_i = \varnothing\right) \vee \left(\exists i \in \mathsf{sHS}^*, \mathsf{CS}^*_i \nsubseteq \mathsf{CS}\right)$

$\mathcal{O}_{\mathsf{Sign}_1}(\mathsf{SS}, \mathsf{M}, \mathsf{sid})$

1 : **req** $(\mathsf{SS} \subseteq [N]) \wedge (|\mathsf{SS}| = T) \wedge \left(\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] = \varnothing\right)$

2 : // $\mathsf{Q}_{\mathsf{M}} \leftarrow \mathsf{Q}_{\mathsf{M}} \cup \{\mathsf{M}\}$ is unnecessary

3 : **for** $i \in \mathsf{HS} \cap \mathsf{SS}$ **do**

4 : $(\mathsf{pm}^{(1)}_i, \mathsf{st}_i) \xleftarrow{\$} \mathsf{Sign}_1(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, \perp, \mathsf{sk}_i, \mathsf{st}_i)$

5 : $\mathsf{Q}_{\mathsf{Sign}}[\mathsf{sid}] \leftarrow (1, \mathsf{SS}, \mathsf{M}, \perp, (\mathsf{pm}^{(1)}_i)_{i\in\mathsf{HS}\cap\mathsf{SS}})$

6 : **return** $(\mathsf{pm}^{(1)}_i)_{i\in\mathsf{HS}\cap\mathsf{SS}}$

Figure 3: Security game of identifiable abort for a R-round threshold signature scheme with identifiable abort, where H denotes the random oracle. The differences from $\mathsf{Game}^{\mathsf{ts\text{-}uf}}$ (Fig. 2) are highlighted in blue.

Parameter	Explanation
$\mathcal{R}_q$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}[X]/(q, X^n+1)$
(k, ℓ)	Dimension of public matrix $\mathbf{A} \in \mathcal{R}_q^{k\times\ell}$
$(\mathcal{D}_{\mathbf{t}}, \sigma_{\mathbf{t}})$	Gaussian distribution with width $\sigma_{\mathbf{t}}$ used for the verification key $\mathbf{t}$
$(\mathcal{D}_{\mathbf{w}}, \sigma_{\mathbf{w}})$	Gaussian distribution with width $\sigma_{\mathbf{w}}$ used for the commitment $\mathbf{w}$
$\nu_{\mathbf{t}}$	Amount of bit dropping performed on verification key
$\nu_{\mathbf{w}}$	Amount of bit dropping performed on (aggregated) commitment
$(q_{\nu_{\mathbf{t}}}, q_{\nu_{\mathbf{w}}})$	Rounded moduli satisfying $(q_{\nu_{\mathbf{t}}}, q_{\nu_{\mathbf{w}}}) := (\lfloor q/2^{\nu_{\mathbf{t}}} \rfloor, \lfloor q/2^{\nu_{\mathbf{w}}} \rfloor) = (\lfloor q/2^{\nu_{\mathbf{t}}} \rceil, \lfloor q/2^{\nu_{\mathbf{w}}} \rceil)$
$(\mathcal{C} \subset \mathcal{R}_q, W)$	Challenge set $\{c \in \mathcal{R}_q \mid \|c\|_\infty = 1 \wedge \|c\|_1 = W\}$ s.t. $\lvert\mathcal{C}\rvert \geqslant 2^\lambda$
B	Two-norm bound on the signature
$B_{\mathbf{r}}, B_{\mathbf{e}}$	Two-norm bound on randomness of commitment $\mathbf{w}$

Table 2: Overview of parameters used in TRaccoon-IA. The parameters are identical to TRaccoon, except for $B_{\mathbf{r}}, B_{\mathbf{e}}$ which are only used in the IA protocol.

In more detail, the algorithm KeyGen outputs a partial signing key $\mathsf{sk}_i := \left([\![\mathbf{s}]\!]_i, \vec{\mathsf{seed}}_i, \gamma_i\right)$ for the i-th signer, where the only difference with the original $\mathsf{TRaccoon}$ is that it includes a commitment randomness γ_i. This is used to commit to the partial secret shares $[\![\mathbf{s}]\!]_i$ based on the lattice-based BDLOP commitment scheme $\mathsf{C}_{\mathsf{share}}$ [BDL+18] (see Section B.5.2 for the details). The commitment C_i is then included in the public information inf, later used by the IA protocol. In addition, we commit to the seeds $\vec{\mathsf{seed}}_i$ using a hash function $\mathsf{H}_{\mathsf{seed}} : \{0,1\}^* \to \{0,1\}^{2\lambda}$, and include it in inf. It can be checked that the partial signing keys of the original $\mathsf{TRaccoon}$ is now publicly committed in inf.

The signing protocol, identical to $\mathsf{TRaccoon}$, relies on a helper algorithm $\mathsf{ZeroShare}$ [KRT24], used to generate the masks. It takes as input a tuple of seeds $\vec{\mathsf{seed}}_i[\mathsf{SS}] := (\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j\in\mathsf{SS}}$ for any signing set $\mathsf{SS} \subseteq [N]$ and a string $x \in \{0,1\}^*$, and outputs a zero share (a.k.a. mask) $\boldsymbol{\Delta}_i := \sum_{j\in\mathsf{SS}\setminus\{i\}}(\mathbf{m}_{i,j} - \mathbf{m}_{j,i})$ where $\mathbf{m}_{i,j} = \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j}, x)$ and $\mathsf{H}_{\mathsf{msk}}$ is a hash function. It can be easily checked that $\sum_{i\in\mathsf{SS}} \boldsymbol{\Delta}_i = \mathbf{0}$.

4.2 The Identifiable Abort Protocol

The IA protocol is given in Fig. 5. As $\mathsf{TRaccoon}$ is a 3-round protocol, there are four associated IA protocols. The first three are used when the signing protocol ends prematurely; these cases are straightforward. The last one, denoted as $(\mathsf{IA}_{4,r})_{r\in[3]}$, is the critical one where every signer outputs a partial signature but the aggregated signature fails to verify.

As explained in Section 1.5, each pair of signers $(i,j) \in \mathsf{SS} \times \mathsf{SS}$ prove, using a simple commit-and-compare protocol, that they generated the zero shares with the same partial masks $(\mathbf{m}_{i,j}, \mathbf{m}_{j,i})$. For this, we use the lattice-based Ajtai commitment scheme $\mathsf{C}_{\mathsf{msk}}$ [Ajt96] to commit to partial masks $(\mathbf{m}_{i,j}, \mathbf{m}_{j,i})_{j\in\mathsf{SS}\setminus\{i\}}$. It is worth mentioning that we use Ajtai, as opposed to the BDLOP commitment used during algorithm KeyGen, as Ajtai commitments do not scale in the size of the message — notice we commit to many more messages during the IA protocol. See Section B.5.2 for more details on the Ajtai commitment. Finally, recall from Section 1.5 that we need to generate the Ajtai commitments deterministically. For this, we use a hash function $\mathsf{H}_{\mathsf{rnd}} : \{0,1\} \to \mathcal{R}_{\mathsf{msk}}$ that maps to the randomness space of $\mathsf{C}_{\mathsf{msk}}$. This deterministic version is coined as $\mathsf{DetMaskCom}$ in Fig. 5.

Let us explain the IA protocol $(\mathsf{IA}_{4,r})_{r\in[3]}$ in more detail. In the first round, all signers create an Ajtai commitment $D_{i,j}$ to the partial masks $\mathbf{m}_{i,j}$ they used in the signing protocol. We postpone the explanation of the NIZK part (cf. Fig. 5, line 10) for now. In the second round, if there is a dispute in the output commitments (cf. Fig. 5, line 8), then at least one of the signers must be dishonest. We collect all such disputes and use the third round to identify which of the two signers was dishonest by asking them to reveal the seed.

The tricky case is when there are no disputes in the second round. This can happen when there are more than two dishonest signers i and j that output the same commitment $D_{i,j}$ but for a message that is not the proper partial mask. To enforce that this cannot happen, the signers generate in the first round a NIZK proof that the zero shares Δ_i they use is consistent with the committed values in $(D_{i,j}, D_{j,i})_{j\in\mathsf{SS}\setminus\{i\}}$ (cf. Fig. 5, line 10). Formally, we rely on an NIZK for the following relation:

$$\mathsf{R}_{\mathsf{z}} := \left\{ (\mathbb{x}, \mathbb{w}) \;\middle|\; \begin{array}{l} \mathbf{z}_i := c \cdot L_{\mathsf{SS},i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \boldsymbol{\Delta}_i \;\wedge\; \mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i \;\wedge \\ \forall j \in \mathsf{SS}\setminus\{i\}, (D_{i,j} = \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m}_{i,j}, \delta_{i,j}) \;\wedge\; D_{j,i} = \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m}_{j,i}, \delta_{j,i})) \;\wedge \\ \boldsymbol{\Delta}_i = \sum_{j\in\mathsf{SS}\setminus\{i\}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i} \;\wedge\; C_i = \mathsf{C}_{\mathsf{share}}.\mathsf{Com}([\![\mathbf{s}]\!]_i, \gamma_i) \;\wedge \\ \|\mathbf{r}_i\|_2 \leqslant B_{\mathbf{r}} \;\wedge\; \|\mathbf{e}'_i\|_2 \leqslant B_{\mathbf{e}} \end{array} \right\},$$

$$\mathbb{x} = (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D_{i,j}, D_{j,i})_{j\in\mathsf{SS}\setminus\{i\}}, C_i),\; \mathbb{w} = ([\![\mathbf{s}]\!]_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j\in\mathsf{SS}\setminus\{i\}}).$$

Thanks to the way we set up the commitments, this is a purely lattice-based relation. Thus, it remains to construct an efficient lattice-based NIZK for this relation. As explained in Section 1.5, we must make non-trivial (and tedious) arguments, proving that we can combine the lattice-based SNARK (without zero-knowledge) LaBRADOR [BS23] with the lattice-based (non-succinct) zero-knowledge proof by [LNP22]. For more details, see Sections B and 6.

5 Security of TRaccoon-IA

Here we show unforgeability and identifiable abort of $\mathsf{TRaccoon\text{-}IA}$. We do not explicitly show correctness as it is implied by identifiable abort as described in Remark 3.2 (see Lemma A.1 for the formal proof).

$\mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}})$

$\mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{k\times\ell}$
$\mathsf{tspar} := (\mathbf{A}, N, T, T_{\mathsf{ia}})$
return tspar

$\mathsf{KeyGen}(\mathsf{tspar})$

$(\mathbf{s}, \mathbf{e}) \xleftarrow{\$} \mathcal{D}_{\mathbf{t}}^\ell \times \mathcal{D}_{\mathbf{t}}^k$
$\mathbf{t} := \lfloor \mathbf{As} + \mathbf{e} \rceil_{\nu_{\mathbf{t}}} \in \mathcal{R}_{q_{\nu_{\mathbf{t}}}}^k$
for $(i, j) \in [N]^2$ **do**
  $\mathsf{rand}_{i,j} \xleftarrow{\$} \{0,1\}^\lambda$
  $\mathsf{seed}_{i,j} := i \| j \| \mathsf{rand}_{i,j}$
  $C_{i,j}^{\mathsf{seed}} := \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{i,j})$
$(\vec{\mathsf{seed}}_i)_{i\in[N]} := \left((\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j\in[N]}\right)_{i\in[N]}$
$\vec{P} \xleftarrow{\$} \mathcal{R}_q^\ell[X]$ with $\deg(\vec{P}) = T-1, \vec{P}(0) = \mathbf{s}$
$(\llbracket \mathbf{s} \rrbracket_i)_{i\in[N]} := (\vec{P}(i))_{i\in[N]}$
$\mathsf{pp}_{\mathsf{msk}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{msk}}}, \mathsf{pp}_{\mathsf{share}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{share}}}$
for $i \in [N]$ **do**
  $\gamma_i \xleftarrow{\$} \mathcal{R}_{\mathsf{share}}$
  $C_i \leftarrow \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\llbracket \mathbf{s} \rrbracket_i; \gamma_i)$
$\mathsf{crs}_{\mathsf{z}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{z}}}$
$\mathsf{vk} := (\mathsf{tspar}, \mathbf{t})$
$\mathsf{inf} := (\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}, (C_{i,j}^{\mathsf{seed}})_{i,j\in[N]}, (C_i)_{i\in[N]})$
$(\mathsf{sk}_i)_{i\in[N]} := (\llbracket \mathbf{s} \rrbracket_i, \vec{\mathsf{seed}}_i, \gamma_i)_{i\in[N]}$
return $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf})$

$\mathsf{Agg}(\mathsf{vk}, \mathsf{SS}, \mathsf{M}, (\mathsf{pm}_j^{(b)})_{(b,j)\in[3]\times\mathsf{SS}})$

parse $(\mathbf{w}_j, \mathbf{z}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pm}_j^{(2)}, \mathsf{pm}_j^{(3)})_{j\in\mathsf{SS}}$
$\mathbf{w} := \left\lfloor \sum_{j\in\mathsf{SS}} \mathbf{w}_j \right\rceil_{\nu_{\mathbf{w}}}$
$\mathbf{z} := \sum_{j\in\mathsf{SS}} \mathbf{z}_j \in \mathcal{R}_q^\ell$
$c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, \mathbf{w})$
$\mathbf{y} := \lfloor \mathbf{Az} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \mathbf{t} \rceil_{\nu_{\mathbf{w}}} \in \mathcal{R}_{q_{\nu_{\mathbf{w}}}}^k$
$\mathbf{h} := \mathbf{w} - \mathbf{y} \in \mathcal{R}_{q_{\nu_{\mathbf{w}}}}^k$
return $\mathsf{sig} := (c, \mathbf{z}, \mathbf{h})$

$\mathsf{Sign}_1(\mathsf{vk}, \mathsf{sid}, i, \mathsf{sk}_i, \mathsf{st}_i)$

$(\mathbf{r}_i, \mathbf{e}_i') \xleftarrow{\$} \mathcal{D}_{\mathbf{w}}^\ell \times \mathcal{D}_{\mathbf{w}}^k$
$\mathbf{w}_i := \mathbf{Ar}_i + \mathbf{e}_i' \in \mathcal{R}_q^k$
$\mathsf{cmt}_i := \mathsf{H}_{\mathsf{com}}(i, \mathbf{w}_i)$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i)$
return $(\mathsf{pm}_i^{(1)} := \mathsf{cmt}_i, \mathsf{st}_i)$

$\mathsf{Sign}_2(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(1)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

req $\left(\mathsf{st}_i[\mathsf{sid}] = (\mathsf{pm}_i^{(1)}, \cdot, \cdot)\right)$
parse $(\mathsf{cmt}_i, \mathbf{w}_i, \mathbf{r}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
parse $(\mathsf{cmt}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow \mathsf{pm}_j^{(1)}$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j)_{j\in\mathsf{SS}}, \mathbf{w}_i, \mathbf{r}_i)$
return $(\mathsf{pm}_i^{(2)} := \mathbf{w}_i, \mathsf{st}_i)$

$\mathsf{Sign}_3(\mathsf{vk}, \mathsf{SS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(2)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

req $\left(\mathsf{st}_i[\mathsf{sid}] = (\mathsf{SS}, \mathsf{M}, \cdot, \mathsf{pm}_i^{(2)}, \cdot)\right)$
parse $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j)_{j\in\mathsf{SS}}, \mathbf{w}_i, \mathbf{r}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
parse $(\mathbf{w}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pm}_j^{(2)})_{j\in\mathsf{SS}\setminus\{i\}}$
// Update state for IA protocol
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i)$
req $(\forall j \in \mathsf{SS},\ \mathsf{cmt}_j = \mathsf{H}_{\mathsf{com}}(j, \mathbf{w}_j))$
$\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS} || \mathsf{M} || (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}$
$\mathbf{w} := \left\lfloor \sum_{j\in\mathsf{SS}} \mathbf{w}_j \right\rceil_{\nu_{\mathbf{w}}} \in \mathcal{R}_{q_{\nu_{\mathbf{w}}}}^k$
$c := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, \mathbf{w})$ // $c \in \mathcal{C}$
$\boldsymbol{\Delta}_i := \mathsf{ZeroShare}(\vec{\mathsf{seed}}_i[\mathsf{SS}], \mathsf{ctnt}_{\mathbf{z}}) \in \mathcal{R}_q^\ell$
$\mathbf{z}_i := c \cdot L_{\mathsf{SS},i} \cdot \llbracket \mathbf{s} \rrbracket_i + \mathbf{r}_i + \boldsymbol{\Delta}_i \in \mathcal{R}_q^\ell$
return $(\mathsf{pm}_i^{(3)} := \mathbf{z}_i, \mathsf{st}_i)$

$\mathsf{Verify}(\mathsf{vk}, \mathsf{M}, \mathsf{sig})$

parse $(c, \mathbf{z}, \mathbf{h}) \leftarrow \mathsf{sig}$
$c' := \mathsf{H}_c(\mathsf{vk}, \mathsf{M}, \lfloor \mathbf{Az} - 2^{\nu_{\mathbf{t}}} \cdot c \cdot \mathbf{t} \rceil_{\nu_{\mathbf{w}}} + \mathbf{h})$
if $\left(c = c'\right) \wedge \left(\|(\mathbf{z}, 2^{\nu_{\mathbf{w}}} \cdot \mathbf{h})\|_2 \leqslant B\right)$ **then**
  return 1
return 0

Figure 4: The algorithms Setup, KeyGen, and Verify and the signing protocol of TRaccoon-IA. The differences between [DPKM+24] are highlighted in blue. Note that Sign_2 implicitly checks $i \in \mathsf{SS} \subseteq [N]$ and $|\mathsf{SS}| = T$.

$\mathsf{IA}_{1,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(1)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

// Abort occurs in Sign_1

$\mathsf{st}_i[\mathsf{sid}] \leftarrow \bot$

$\mathsf{CS}_i^{\mathsf{IA}} := \{j \mid j \in \mathsf{SS}, \mathsf{pm}_j^{(1)} = \bot\}$

return $(\mathsf{pmia}_{1,i}^{(1)} := \mathsf{CS}_i^{\mathsf{IA}}, \mathsf{st}_i)$

$\mathsf{IA}_{2,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(1)}, \mathsf{pm}_j^{(2)})_{j\in\mathsf{SS}}, \mathsf{sk}_i, \mathsf{st}_i)$

// Abort occurs in Sign_2

$\mathsf{st}_i[\mathsf{sid}] \leftarrow \bot$

$\mathsf{CS}_i^{\mathsf{IA}} := \{j \mid j \in \mathsf{SS}, \mathsf{pm}_j^{(2)} = \bot\}$

return $(\mathsf{pmia}_{2,i}^{(1)} := \mathsf{CS}_i^{\mathsf{IA}}, \mathsf{st}_i)$

$\mathsf{IA}_{3,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[3]}, \mathsf{sk}_i, \mathsf{st}_i)$

// Abort occurred in Sign_3

req $(\mathsf{IAS} = \mathsf{SS})$

parse $(\mathbf{w}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pm}_j^{(2)})_{j\in\mathsf{SS}\setminus\{i\}}$

parse $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$

$\mathsf{CS}_i^{\mathsf{IA}} := \{j \mid j \in \mathsf{SS}, \mathsf{cmt}_j \neq \mathsf{H}_{\mathsf{com}}(j, \mathbf{w}_j)\}$

if $\mathsf{CS}_i^{\mathsf{IA}} = \varnothing$ **then**

  $\mathsf{CS}_i^{\mathsf{IA}} \leftarrow \{j \mid j \in \mathsf{SS}, \mathsf{pm}_j^{(3)} = \bot\}$

return $(\mathsf{pmia}_{3,i}^{(1)} := \mathsf{CS}_i^{\mathsf{IA}}, \mathsf{st}_i)$

$\mathsf{IA}_{4,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[4]}, \mathsf{sk}_i, \mathsf{st}_i)$

// Aggregated signature does not verify

req $(\mathsf{IAS} = \mathsf{SS})$

parse $(\mathbf{z}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pm}_j^{(3)})_{j\in\mathsf{SS}\setminus\{i\}}$

parse $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$

$\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS}||\mathsf{M}||(\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}$

for $j \in \mathsf{SS}\setminus\{i\}$ **do**

  $(D_{i,j}^{(i)}, \delta_{i,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$

  $(D_{j,i}^{(i)}, \delta_{j,i}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,i}, \mathsf{ctnt}_{\mathbf{z}})$

$\mathbb{x}_i := (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}\setminus\{i\}}, C_i)$

$\mathbb{w}_i := (\llbracket \mathbf{s} \rrbracket_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j\in\mathsf{SS}\setminus\{i\}})$

$\pi_i \leftarrow \Pi_{\mathsf{z}}.\mathsf{Prove}_{\mathsf{z}}^{\mathsf{H}}(\mathbb{x}_i, \mathbb{w}_i)$

$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}\setminus\{i\}})$

$\mathsf{pmia}_{4,i}^{(1)} := ((D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}\setminus\{i\}}, \pi_i)$

return $(\mathsf{pmia}_{4,i}^{(1)}, \mathsf{st}_i)$

$\mathsf{DetMaskCom}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}})$

// Helper algorithm

$\mathbf{m} := \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}}); \delta := \mathsf{H}_{\mathsf{rnd}}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}})$

$D := \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m}, \delta)$

return (D, δ)

$\mathsf{IA}_{4,2}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pmia}_{4,j}^{(1)})_{j\in\mathsf{IAS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $((D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}\setminus\{j\}}, \pi_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pmia}_{4,j}^{(1)})_{j\in\mathsf{SS}\setminus\{i\}}$

parse $(\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}\setminus\{i\}}) \leftarrow \mathsf{st}_i[\mathsf{sid}]$

$\mathsf{CS}_i^{\mathsf{IA}} := \varnothing, \mathsf{proofs}_i := \varnothing$

for $j \in \mathsf{SS}\setminus\{i\}$ **do**

  $\mathbb{x}_j := (L_{\mathsf{SS},j}, \mathbf{z}_j, \mathbf{w}_j, c, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}\setminus\{j\}}, C_j)$

  if $\Pi_{\mathsf{z}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{z}}}(\mathbb{x}_j, \pi_j) = 0$ **then**

    $\mathsf{CS}_i^{\mathsf{IA}} \leftarrow \mathsf{CS}_i^{\mathsf{IA}} \cup \{j\}$

  if $\left(D_{i,j}^{(i)} \neq D_{i,j}^{(j)}\right) \vee \left(D_{j,i}^{(i)} \neq D_{j,i}^{(j)}\right)$ **then**

    $\mathsf{CS}_i^{\mathsf{IA}} \leftarrow \mathsf{CS}_i^{\mathsf{IA}} \cup \{j\}$

    // Proof honest commitment behaviour

    $\mathsf{proof}_j \leftarrow (i, j, \mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})$

    $\mathsf{proofs} \leftarrow \mathsf{proofs} \cup \{\mathsf{proof}_j\}$

$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{(j,k)\in\mathsf{SS}^2, j\neq k}, \mathsf{CS}_i^{\mathsf{IA}})$

return $(\mathsf{pmia}_{4,i}^{(2)} := \mathsf{proofs}, \mathsf{st}_i)$

$\mathsf{IA}_{4,3}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pmia}_{4,j}^{(2)})_{j\in\mathsf{IAS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $(\mathsf{proofs}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pmia}_{4,j}^{(2)})_{j\in\mathsf{SS}}$

parse $(\mathsf{SS}, \mathsf{M}, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{(j,k)\in\mathsf{SS}^2, j\neq k}, \mathsf{CS}_i^{\mathsf{IA}}) \leftarrow \mathsf{st}_i[\mathsf{sid}]$

for $(j, k) \in (\mathsf{SS}\setminus\{i\})^2, j \neq k$ **do**

  if $\left(D_{j,k}^{(j)} \neq D_{j,k}^{(k)}\right) \vee \left(D_{k,j}^{(j)} \neq D_{k,j}^{(k)}\right)$ **then**

    // Check behaviour of signer j during IA

    $\mathsf{bad\text{-}proof} := (j, k, \mathsf{seed}_{j,k}, \mathsf{seed}_{k,j}) \notin \mathsf{proofs}_j$

    $\mathsf{bad\text{-}seed} := C_{j,k}^{\mathsf{seed}} \neq \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{j,k})$

    $\mathsf{bad\text{-}seed} \leftarrow \mathsf{bad\text{-}seed} \vee (C_{k,j}^{\mathsf{seed}} \neq \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{k,j}))$

    $(\overline{D}_{j,k}^{(j)}, \overline{\delta}_{j,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,k}, \mathsf{ctnt}_{\mathbf{z}})$

    $(\overline{D}_{k,j}^{(j)}, \overline{\delta}_{k,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{k,j}, \mathsf{ctnt}_{\mathbf{z}})$

    $\mathsf{bad\text{-}vm} := (\overline{D}_{j,k}^{(j)}, \overline{D}_{k,j}^{(j)}) \neq (D_{j,k}^{(j)}, D_{k,j}^{(j)})$

    if $\mathsf{bad\text{-}proof} \vee \mathsf{bad\text{-}seed} \vee \mathsf{bad\text{-}vm}$ **then**

      $\mathsf{CS}_i^{\mathsf{IA}} \leftarrow \mathsf{CS}_i^{\mathsf{IA}} \cup \{j\}$

return $\mathsf{pmia}_{4,i}^{(3)} := \mathsf{CS}_i^{\mathsf{IA}}, \mathsf{st}_i$

Figure 5: The identifiable abort protocol of TRaccoon-IA. DetMaskCom is a helper algorithm, deterministically generating an Ajtai commitment $\mathsf{C}_{\mathsf{msk}}$ from the seed.

5.1 Asymptotic Parameters

We first provide a candidate asymptotic parameter set $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ for TRaccoon-IA. This parameter set is almost identical to the parameter set $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon}}$ used in TRaccoon [KRT24]. The differences are as follows:

- $(B_{\mathbf{r}}, B_{\mathbf{e}})$ is newly added, only to be used by the NIZK in our IA protocol.
- Q_{S} is replaced with $T \cdot Q_{\mathsf{S}}$.[9]
- The bound B in the verification algorithm, dictating the shortness of the signature, is slightly larger. Namely, the second term in B is changed from $e^{1/4} \cdot \sqrt{T} \cdot \sigma_{\mathbf{w}} \cdot \sqrt{n(k+\ell)}$ to $e^{1/4} \cdot T \cdot \sigma_{\mathbf{w}} \cdot (\sqrt{nk} + \sqrt{n\ell})$.

To understand the last modification, recall in TRaccoon [KRT24], B is only used to ensure correctness of honestly generated *average-case* signatures. Since a valid signature contains the sum of the randomness $\sum_{i \in \mathsf{SS}} \mathbf{r}_i$ and $\sum_{i \in \mathsf{SS}} \mathbf{e}'_i$, where $(\mathbf{r}_i, \mathbf{e}'_i)$ are discrete Gaussians with standard deviation $\sigma_{\mathbf{w}}$, B depends only on $\sqrt{T}\sigma_{\mathbf{w}}$ based on the convolution of Gaussians. In contract, in TRaccoon-IA, we must also consider maliciously generated *worst-case* signatures. Notice that the NIZK relation $\mathsf{R}_{\mathbf{z}}$ can only check the validity of the individual randomness $\mathbf{r}_i$ and $\mathbf{e}'_i$. In particular, nothing prevents a set of malicious signers to pick their randomness in such a way that their sum becomes larger than the sum of randomly chosen $\mathbf{r}_i$ and $\mathbf{e}'_i$. Put differently, we are not able to rely on the convolution of Gaussians, and thus, $\mathcal{B}$ now depends on $T \cdot \sigma_{\mathbf{w}}$.

Concretely, we set $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ as follows:

- $n, \ell, k = \mathsf{poly}(\lambda)$ such that $n \geqslant \lambda$ and $T \cdot Q_{\mathsf{S}} > n^2$.
- $W = \omega(1)$ for $|\mathcal{C}| \geqslant 2^{\lambda}$ for Lemma 2.7 (hardness of MSIS) .
- $B_{\mathsf{hint}} = T \cdot Q_{\mathsf{S}} \cdot W \cdot \left(1 + n\frac{1}{\sqrt{T \cdot Q_{\mathsf{S}}}}(\lambda + 1 + 2\log(n))\right)$, $\frac{1}{\sigma^2} = 2 \cdot \left(\frac{1}{\sigma_{\mathbf{t}}^2} + \frac{B_{\mathsf{hint}}}{\sigma_{\mathbf{w}}^2}\right)$, and $\sigma \geqslant \sqrt{\ell} \cdot \omega(\sqrt{\log n})$ for Lemmata 2.3 and 2.6 and [DPKM+24, Lemma B.2] (reduction from Hint-MLWE to MLWE and hardness of MLWE).
- $(\sigma_{\mathbf{t}}, \sigma_{\mathbf{w}}) = \left(2\sqrt{\ell} \cdot \log n, 2\sqrt{B_{\mathsf{hint}} \cdot \ell} \cdot \log n\right)$ and $\sigma_{\mathbf{w}} > 2n \cdot q^{\frac{1}{k+\ell} + \frac{2}{n\ell}}$ for Lemmata 2.2 and 2.5.
- $\nu_{\mathbf{t}}, \nu_{\mathbf{w}} = O(\log \lambda)$ such that $\nu_{\mathbf{w}} \geqslant 4$ and $\nu_{\mathbf{w}} < \log(q) - 2$ for correctness and Lemma 2.5.
- $B_{\mathbf{r}} = e^{1/4} \cdot \sigma_{\mathbf{w}} \cdot \sqrt{n\ell}$ and $B_{\mathbf{e}} = e^{1/4} \cdot \sigma_{\mathbf{w}} \cdot \sqrt{nk}$.
- $B = e^{1/4} \cdot W \cdot \sigma_{\mathbf{t}} \cdot \sqrt{n(k+\ell)} + T(B_{\mathbf{r}} + B_{\mathbf{e}}) + (W \cdot 2^{\nu_{\mathbf{t}}} + 2^{\nu_{\mathbf{w}}+1}) \cdot \sqrt{nk}$ for correctness and identifiable abort (see Lemma 5.5).
- $B_{\mathsf{stmsis}} = B + \sqrt{W} + (W \cdot 2^{\nu_{\mathbf{t}}} + 2^{\nu_{\mathbf{w}}+1}) \cdot \sqrt{nk}$ for Lemmata 5.1 and 5.3.
- q is an integer than $2B_{\mathsf{stmsis}} \cdot \sqrt{nk} \cdot \log(nk)^2$ such that $(q, \nu_{\mathbf{t}}, \nu_{\mathbf{w}})$ satisfy the condition in Table 2 for Lemma 2.4 (hardness of MSIS).

5.2 Useful Lemmata

Looking ahead, we will show the unforgeability of TRaccoon-IA by reducing it to the unforgeability of TRaccoon [DPKM+24, KRT24] since the signing protocol and verification algorithm remain unchanged. However, the unforgeability for TRaccoon holds under different asymptotic parameters (cf. [KRT24, App. D]) than the ones described above, and it is proven under a slightly different unforgeability game in [KRT24]. Before moving to the unforgeability of TRaccoon-IA, we thus show that TRaccoon is unforgeable with the above asymptotic parameters under our definition from Section 3.

[9] This comes from the fact that we prove unforgeability by using unforgeability of TRaccoon. If we prove it directly from Hint-MLWE and SelfTargetMSIS assumption, this does not appear.

Lemma 5.1. TRaccoon *with parameters* $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ *given in Section 5.1 is unforgeable as per Definition 3.2. In particular, for any adversary* $\mathcal{A}$ *against the unforgeability game of* TRaccoon *making at most* $Q_{\mathsf{H}_{\mathsf{msk}}}, Q_{\mathsf{H}_c}, Q_{\mathsf{H}_{\mathsf{com}}}$ *and* Q_{S} *queries to the random oracles* $\mathsf{H}_{\mathsf{msk}}, \mathsf{H}_c, \mathsf{H}_{\mathsf{com}}$ *and the signing oracle, respectively, there exist adversaries* $\mathcal{B}$ *and* $\mathcal{B}'$ *against the* $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,T\cdot Q,\sigma_{\mathbf{t}},\sigma_{\mathbf{w}},\mathcal{C}}$ *and* $\mathsf{SelfTargetMSIS}_{q,\ell+1,k,\mathsf{H}_c,\mathcal{C},B_{\mathsf{stmsis}}}$ *problems, respectively, such that*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{A}}(\lambda,N,T,\perp) \leqslant\ & \mathsf{Adv}^{\mathsf{Hint\text{-}MLWE}}_{\mathcal{B}}(\lambda) + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}'}(\lambda) \\ & + \frac{T\cdot Q_{\mathsf{S}}\cdot(Q_{\mathsf{H}_{\mathsf{com}}} + Q_{\mathsf{H}_c} + 2T\cdot Q_{\mathsf{S}})}{2^{n-1}} \\ & + \frac{Q_{\mathsf{H}_{\mathsf{msk}}}}{2^{\lambda}} + \frac{(Q_{\mathsf{H}_{\mathsf{com}}} + T\cdot Q_{\mathsf{S}})^2 + Q_{\mathsf{H}_{\mathsf{com}}}}{2^{2\lambda}} + \mathsf{negl}(\lambda) \qquad (1)\end{aligned}$$

where $\mathsf{Time}(\mathcal{B}), \mathsf{Time}(\mathcal{B}') \approx \mathsf{Time}(\mathcal{B}')$ *and parameters given in Section 5.1.*

Proof. We prove this lemma in two steps. We first show that (a) TRaccoon remains unforgeable under the definition in Section 3, by substituting Q_{S} by $T\cdot Q_{\mathsf{S}}$ in parameters from [KRT24]. After that, we show that (b) we can increase the bound B in the asymptotic parameters of TRaccoon to tackle a factor increase from $\sqrt{T}$ to T in TRaccoon-IA, while still satisfying unforgeability.

Let us show the first statement (a). For the unforgeability, we observe that the signing oracles in our definition from Section 3 executes the signing algorithm for all honest users at once, whereas in [KRT24], it is performed for only one honest user at once. Hence, one signing oracle query in our definition can be simulated with T queries to oracles in the definition of [KRT24]. We deduce that TRaccoon remains unforgeable in our definition by replacing Q_{S} by $T\cdot Q_{\mathsf{S}}$ in the asymptotic parameters from [KRT24], formally defined as $\mathsf{hard\text{-}param}'_{\mathsf{TRaccoon}}$. From the security theorem [KRT24, Theorem 4.1], we immediately obtain that there exist adversaries $\mathcal{B}$ and $\mathcal{B}'$ against the $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,T\cdot Q,\sigma_{\mathbf{t}},\sigma_v ecw,\mathcal{C}}$ and $\mathsf{SelfTargetMSIS}_{q,\ell+1,k,\mathsf{H}_c,\mathcal{C},B_{\mathsf{stmsis}}}$ problems, respectively, such that Eq. (1) holds and $\mathsf{Time}(\mathcal{B}), \mathsf{Time}(\mathcal{B}') \approx \mathsf{Time}(\mathcal{B}')$.

Let us show the second statement (b), i.e. replacing the bound B in $\mathsf{hard\text{-}param}'_{\mathsf{TRaccoon}}$ with the above formula used in $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon}}$ stills provides unforgeability. Plugging $B_{\mathbf{r}} = e^{1/4}\cdot\sigma_{\mathbf{w}}\cdot\sqrt{n\ell}$ and $B_{\mathbf{e}} = e^{1/4}\cdot\sigma_{\mathbf{w}}\cdot\sqrt{nk}$ into the new formula for B, we have

$$B = e^{1/4}(W\cdot\sigma_{\mathbf{t}} + T\sigma_{\mathbf{w}})\sqrt{n(k+\ell)} + (W\cdot 2^{\nu_{\mathbf{t}}} + 2^{\nu_{\mathbf{w}}+1})\cdot\sqrt{nk}.$$

Recall that B in $\mathsf{hard\text{-}param}'_{\mathsf{TRaccoon}}$, denoted by $\overline{B}$, is set to $e^{1/4}(W\cdot\sigma_{\mathbf{t}} + \sqrt{T}\cdot\sigma_{\mathbf{w}})\sqrt{n(k+\ell)} + (W\cdot 2^{\nu_{\mathbf{t}}} + 2^{\nu_{\mathbf{w}}+1})\cdot\sqrt{nk}$. The only difference with the asymptotic parameters for TRaccoon-IA is a factor of $\sqrt{T}$. Observing the relations of B with other parameters, having a larger B only impacts the lower bound on q imposed by B_{stmsis}. Successively, q only constrains the value of $\sigma_{\mathbf{w}}$. It is thus sufficient to show that the condition $\sigma_{\mathbf{w}} > 2n\cdot q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$ still holds.

From the above observations, B is at most $\sqrt{T}\cdot\overline{B}$. Since $T\in\mathsf{poly}(\lambda)$, the updated q in our parameters then still satisfies $q = \mathsf{poly}(\lambda)$. Also, since we set $n,\ell,k = \mathsf{poly}(\lambda)$, $q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$ is asymptotically bounded by some constant close to 1. Therefore, $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ satisfies $\sigma_{\mathbf{w}} > 2n\cdot q^{\frac{1}{k+\ell}+\frac{2}{n\ell}}$. This completes the proof. □

The relation R_{z} contains an condition on the norm of $\mathbf{e}'$ and $\mathbf{r}$. This condition does not necessarily hold with probability 1, even for honest users, since $(\mathbf{e}',\mathbf{r})$ is chosen following the distribution $\mathcal{D}^{\ell}_{\mathbf{w}}\times\mathcal{D}^{k}_{\mathbf{w}}$. The following lemma ensures that $(\mathbb{x},\mathbb{w})\in\mathsf{R}_{\mathsf{z}}$ except for negligible probability if $\mathbb{x}$ and $\mathbb{w}$ are setup honestly as in the IA protocol (cf. Fig. 5).

Lemma 5.2. *For parameters given in Section 5.1, we have*

$$\Pr[(\mathbf{r},\mathbf{e}') \xleftarrow{\$} \mathcal{D}^{\ell}_{\mathbf{w}}\times\mathcal{D}^{k}_{\mathbf{w}} : \|\mathbf{r}_i\|_2 > B_{\mathbf{r}} \vee \|\mathbf{e}'_i\|_2 > B_{\mathbf{e}}] \leqslant 2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}$$

Proof. Recall that $B_{\mathbf{r}}$ and $B_{\mathbf{e}}$ are set to $e^{1/4}\cdot\sigma_{\mathbf{w}}\cdot\sqrt{n\ell}$ and $e^{1/4}\cdot\sigma_{\mathbf{w}}\cdot\sqrt{nk}$, respectively. Then, since $\mathbf{r}$ and $\mathbf{e}'$ are independently generated, we have

$$\Pr[(\mathbf{r},\mathbf{e}') \xleftarrow{\$} \mathcal{D}^{\ell}_{\mathbf{w}}\times\mathcal{D}^{k}_{\mathbf{w}} : \|\mathbf{r}_i\|_2 > B_{\mathbf{r}} \vee \|\mathbf{e}'_i\|_2 > B_{\mathbf{e}}]$$

$$
\begin{aligned}
&= \Pr[\mathbf{r} \xleftarrow{\$} \mathcal{D}_{\mathbf{w}}^{\ell} : \|\mathbf{r}_i\|_2 > B_{\mathbf{r}}] + \Pr[\mathbf{e}' \xleftarrow{\$} \mathcal{D}_{\mathbf{w}}^{k} : \|\mathbf{e}'_i\|_2 > B_{\mathbf{e}}] \\
&\leqslant 2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}
\end{aligned}
$$

where the inequality follows from Lemma 2.1. The proof is completed. □

5.3 Unforgeability

The following theorem establishes the unforgeability of TRaccoon-IA.

Theorem 5.1. *The threshold signature with identifiable abort* TRaccoon-IA *in Figs. 4 and 5 instantiated with* $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ *given in Section 5.1 is unforgeable under the* Hint-MLWE *and* SelfTargetMSIS *assumptions, zero-knowledge of NIZK* Π_{z}*, and hiding of the commitment schemes.*

Formally, for any N, T, T_{ia} *with* $T = T_{\mathsf{ia}} \leqslant N$ *and an adversary* $\mathcal{A}$ *against the unforgeability game making at most* $Q_{\mathsf{H}_{\mathsf{msk}}}, Q_{\mathsf{H}_c}, Q_{\mathsf{H}_{\mathsf{seed}}}, Q_{\mathsf{H}_{\mathsf{com}}}, Q_{\mathsf{H}_{\mathsf{rnd}}}$ *and* Q_{S} *queries to the random oracles* $\mathsf{H}_{\mathsf{msk}}, \mathsf{H}_c, \mathsf{H}_{\mathsf{seed}}, \mathsf{H}_{\mathsf{com}}, \mathsf{H}_{\mathsf{rnd}}$ *and the signing, aggregation, and IA oracles, respectively, there exists adversaries* $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{B}_4$ *and* $\mathcal{B}_5$ *against zero-knowledge of NIZK* Π_{z}*, hiding of the commitment schemes* $\mathsf{C}_{\mathsf{msk}}$ *and* $\mathsf{C}_{\mathsf{share}}$*, and the* $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,T\cdot Q,\sigma_{\mathbf{t}},\sigma_{\mathbf{w}},\mathcal{C}}$ *and* $\mathsf{SelfTargetMSIS}_{q,\ell+1,k,\mathsf{H}_c,\mathcal{C},B_{\mathsf{stmsis}}}$ *problems, respectively, such that*

$$
\begin{aligned}
&\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon\text{-}IA},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}}) \\
&\leqslant \mathsf{Adv}^{\mathsf{zk}}_{\mathcal{B}_1,\Pi_{\mathsf{z}}}(\lambda) + \mathsf{Adv}^{\mathsf{hide}}_{\mathcal{B}_2,\mathsf{C}_{\mathsf{msk}}}(\lambda) + \mathsf{Adv}^{\mathsf{hide}}_{\mathcal{B}_3,\mathsf{C}_{\mathsf{share}}}(\lambda) + \mathsf{Adv}^{\mathsf{Hint\text{-}MLWE}}_{\mathcal{B}_4}(1^\lambda) \\
&\quad + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_5}(1^\lambda) + \frac{T \cdot Q_{\mathsf{S}} \cdot (Q_{\mathsf{H}_{\mathsf{com}}} + Q_{\mathsf{H}_c} + 2T \cdot Q_{\mathsf{S}})}{2^{n-1}} + \frac{Q_{\mathsf{H}_{\mathsf{seed}}} + Q_{\mathsf{H}_{\mathsf{rnd}}} + Q_{\mathsf{H}_{\mathsf{msk}}}}{2^\lambda} \\
&\quad + \frac{(Q_{\mathsf{H}_{\mathsf{com}}} + T \cdot Q_{\mathsf{S}})^2 + Q_{\mathsf{H}_{\mathsf{com}}}}{2^{2\lambda}} + T \cdot Q_{\mathsf{S}} \cdot \left(2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}\right) + \mathsf{negl}(\lambda)
\end{aligned}
$$

where $\mathsf{Time}(\mathcal{B}_1), \mathsf{Time}(\mathcal{B}_2), \mathsf{Time}(\mathcal{B}_3), \mathsf{Time}(\mathcal{B}_4), \mathsf{Time}(\mathcal{B}_5) \approx \mathsf{Time}(\mathcal{A})$.

Before showing the formal proof, we provide the overview of it.

Overview. The unforgeability of TRaccoon-IA is naturally inherited from the unforgeability of TRaccoon since the added IA protocol leaks no information of the honest signers' partial signing keys. Recall that during our IA protocol, the secret information of the signers are included in the commitments and NIZK proofs generated in $\mathsf{IA}_{4,1}$ and proofs in $\mathsf{IA}_{4,2}$. It is easy to see that they do not leak any secret information as the commitment scheme is hiding and the NIZK is zero-knowledge. While $\mathsf{IA}_{4,3}$ requires the signers to disclose the seeds, this will not affect the security when one of the signers is malicious. This is because a malicious signer holds the same seed as the honest signer. Moreover, a pair of honest signers are never asked to disclose the seeds by construction.

Our proof consists in formalizing this argument and directly reducing the unforgeability of TRaccoon to the unforgeability of TRaccoon-IA. Concretely, we first modify the original game so that it aborts when any of the following three bad events occur in the IA protocol.

1. The first event is when the honest signer's $(\mathbb{x}_i, \mathbb{w}_i)$ is not in R_{z}. It is easy to check that this event occurs with negligible probability due to the tail-cut bound (see Lemma 5.2).

2. The second event is when seeds for a pair of honest signers are opened. Given that honest users deterministically generate commitments from the same seed, this event never occurs.

3. The final event is when an adversary $\mathcal{A}$ succeeds in guessing the seed shared between a pair of honest signers. This occurs with negligible probability thanks to the high-entropy of the seeds.

We then change the game so that the challenger no longer uses the secret information of the honest signers during the IA protocol. As explained above, this modification is possible thanks to the hiding property of the commitment scheme $\mathsf{C}_{\mathsf{msk}}$ and zero-knowledge property of the NIZK. We also need to remove the information pertaining to seeds and shares of secret key from the commitments in KeyGen. This modification is possible due to the hiding property of the commitment scheme $\mathsf{C}_{\mathsf{share}}$ and the hash commitment instantiated by the random oracle $\mathsf{H}_{\mathsf{seed}}$.

We now can reduce from the unforgeability of the original TRaccoon to complete the proof since the modified game ensures no use of secret information except for signing protocol, which is the same with the original one. But, we cannot directly rely on the security theorem of TRaccoon because of the differences of the unforgeability game and parameters (see Remark 3.1 and Section 5.1). Thus, we rely on Lemma 5.1, stating the original TRaccoon is unforgeable under the our unforgeability game and our parameters $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$. Due to this lemma, we can bound the advantage of $\mathcal{A}$ in the modified game by the advantage given in this supporting lemma and complete the proof. □

Proof. We prove the above theorem with a series of hybrids starting from the unforgeability game. Before we give the formal argument, let us sketch our approach. Observe that the signing protocol is identical to the signing protocol in TRaccoon [KRT24]. Roughly, we move to a hybrid game where the challenger simulates the IA oracles and key generation output using only the secret keys of malicious users $i \in \mathsf{CS}$ and verification key vk. Then, we can simply reduce unforgeability of our scheme TRaccoon-IA from unforgeability of TRaccoon [KRT24].

Let $\mathcal{A}$ be an adversary against unforgeability of TRaccoon-IA. Denote by $\mathsf{sHS} := \mathsf{HS} \cap \mathsf{SS}$ and $\mathsf{sCS} := \mathsf{CS} \cap \mathsf{SS}$. For clarity, when introducing abort conditions within the hybrids, instead of saying the challenger aborts, we introduce a flag $\mathsf{f}_{\mathsf{fail}}$ (cf. Fig. 6) which is initially set to $\bot$. At the end of the game, the challenger checks whether $\mathsf{f}_{\mathsf{fail}} = \bot$, and outputs 0 if not. When we introduce an abort condition in a hybrid, we say that the challenger sets $\mathsf{f}_{\mathsf{fail}} = \top$, in which case the adversary cannot win anymore. (Note that we ensure that $\mathsf{f}_{\mathsf{fail}}$ is never reset to $\bot$ within the game).

Hybrid_1. The first hybrid corresponds to the unforgeability game $\mathsf{Game}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon\text{-}IA},\mathcal{A}}$ from Fig. 2. Note that $\mathsf{H}_{\mathsf{msk}}, \mathsf{H}_c, \mathsf{H}_{\mathsf{seed}}, \mathsf{H}_{\mathsf{com}}$ and $\mathsf{H}_{\mathsf{rnd}}$ are modeled as random oracles. We have

$$\mathsf{Adv}^{\mathsf{Hybrid}_1}_{\mathcal{A}}(\lambda) = \mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon\text{-}IA},\mathcal{A}}(\lambda, N, T, T_{\mathsf{ia}}).$$

Hybrid_2. In this hybrid, the challenger introduces two additional abort conditions. First, the challenger aborts in $\mathcal{O}_{\mathsf{IA}_{4,1}}$ if $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}_{\mathsf{z}}$. Second, the challenger aborts in $\mathcal{O}_{\mathsf{IA}_{4,2}}$ if $\mathsf{proof}_j = (i, j, \mathsf{seed}_{i,j}, \mathsf{seed}_{j,i}) \in \mathsf{proofs}$ is output for some $j \in \mathsf{sHS}$. This is formalized in Fig. 6.

Let us analyze the probability of setting $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$ in Hybrid_2. First, let us show that in $\mathcal{O}_{\mathsf{IA}_{4,2}}$ when invoked for $i \in \mathsf{sHS}$, it holds that $D^{(i)}_{i,j} = D^{(j)}_{i,j}$ and $D^{(i)}_{j,i} = D^{(j)}_{j,i}$ for $j \in \mathsf{sHS}$. (Only if this does not hold, the challenger outputs proof_j within proofs.) Recall that the adversary only controls protocol messages of corrupted signers $j \in \mathsf{CS}$ in the game. Thus, it suffices to show that both user i and j output identical commitments in $\mathcal{O}_{\mathsf{IA}_{4,1}}$. Notice that both $D^{(i)}_{i,j}$ and $D^{(j)}_{i,j}$ are obtained via $\mathsf{DetMaskCom}$ on input $(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathsf{z}})$. Equality of both commitments follows because $\mathsf{DetMaskCom}$ is deterministic, and the same argument yields $D^{(i)}_{j,i} = D^{(j)}_{j,i}$. Second, observe that due Lemma 5.2, we have $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}_{\mathsf{z}}$ except with probability $2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}$. A union bound yields

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_1}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_2}_{\mathcal{A}}(\lambda)\right| \leqslant T \cdot Q_{\mathsf{S}} \cdot \left(2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}\right).$$

Hybrid_3. In this hybrid, the challenger simulates the NIZK π_i in $\mathcal{O}_{\mathsf{IA}_{4,1}}$. Let $\mathsf{Sim} = (\mathsf{Sim}_0, \mathsf{Sim}_1)$ be the zero-knowledge simulator of Π_{z}. The challenger simulates H_{z} via Sim_0 and proofs π_i by invoking Sim_1 on statement $\mathbb{x}$. This is formalized in Fig. 7.

Note that $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}_{\mathsf{z}}$ due to the abort condition introduced in the previous hybrid. Thus, it is straightforward to construct an adversary $\mathcal{B}_1$ on the zero-knowledge property of Π_{z} such that $\mathsf{Time}(\mathcal{B}_1) \approx \mathsf{Time}(\mathcal{A})$ and

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_2}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_3}_{\mathcal{A}}(\lambda)\right| \leqslant \mathsf{Adv}^{\mathsf{zk}}_{\mathcal{B}_1,\Pi_{\mathsf{z}}}(\lambda)$$

Hybrid_4. In this hybrid, the challenger sets $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$ if the adversary queries $\mathsf{H}_{\mathsf{seed}}$ or $\mathsf{H}_{\mathsf{rnd}}$ with input $\mathsf{seed}_{i,j}$ for $i, j \in \mathsf{HS}$. This is formalized in Fig. 8.

We bound the probability of setting $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$ due to the added abort conditions. Let $Q^{(i,j)}_{\mathsf{H}_{\mathsf{seed}}}$ and $Q^{(i,j)}_{\mathsf{H}_{\mathsf{rnd}}}$ be the number of the random oracle queries with users $i, j \in \mathsf{HS}$ for $\mathsf{H}_{\mathsf{seed}}$ and $\mathsf{H}_{\mathsf{rnd}}$, respectively. Note

$\mathsf{Game}^{\mathsf{ts\text{-}uf}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}})$

$\mathsf{f}_{\mathsf{fail}} := \bot$
$\mathsf{Q}_{\mathsf{M}} := \varnothing, \mathsf{Q}_{\mathsf{Sign}}[\cdot] := \varnothing, \mathsf{Q}_{\mathsf{ia}}[\cdot] := \varnothing, \mathsf{Round}_\bot[\cdot] := \varnothing$
$\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}})$
$(\mathsf{CS}, \mathsf{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(\mathsf{tspar})$
req $(\mathsf{CS} \subseteq [N]) \wedge (|\mathsf{CS}| \leqslant T-1)$
$\mathsf{HS} := [N] \backslash \mathsf{CS}$
for $i \in \mathsf{HS}$ **do** $\mathsf{st}_i := \varnothing$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$
$\mathsf{oracles} := \Big((\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}, \mathcal{O}_{\mathsf{Agg}}, (\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\bot,r}})^{\mathsf{rnd}_\bot\in[R+1],}_{r\in[R_{\mathsf{ia}}^{(\mathsf{rnd}_\bot)}]}, \mathsf{H}\Big)$
$(\mathsf{sig}^*, \mathsf{M}^*) \xleftarrow{\$} \mathcal{A}^{\mathsf{oracles}}(\mathsf{vk}, \mathsf{inf}, (\mathsf{sk}_i)_{i\in\mathsf{CS}}, \mathsf{st}_{\mathcal{A}})$
req $\big(\mathsf{M}^* \notin \mathsf{Q}_{\mathsf{M}}\big) \wedge (\mathsf{f}_{\mathsf{fail}} = \bot)$
return $\mathsf{Verify}(\mathsf{vk}, \mathsf{M}^*, \mathsf{sig}^*)$

$\mathsf{IA}_{4,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[4]}, \mathsf{sk}_i, \mathsf{st}_i)$

// Aggregated signature does not verify
req $(\mathsf{IAS} = \mathsf{SS})$
parse $(\mathbf{z}_j)_{j\in\mathsf{SS}\backslash\{i\}} \leftarrow (\mathsf{pm}_j^{(3)})_{j\in\mathsf{SS}\backslash\{i\}}$
parse $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
$\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS}||\mathsf{M}||(\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}$
for $j \in \mathsf{SS}\backslash\{i\}$ **do**
  $(D_{i,j}^{(i)}, \delta_{i,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$
  $(D_{j,i}^{(i)}, \delta_{j,i}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,i}, \mathsf{ctnt}_{\mathbf{z}})$
$\mathbb{x} := (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}}, C_i)$
$\mathbb{w} := (\llbracket \mathbf{s} \rrbracket_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j\in\mathsf{SS}})$
$\pi_i \leftarrow \Pi_{\mathsf{z}}.\mathsf{Prove}_{\mathsf{z}}^{\mathsf{H}}(\mathbb{x}, \mathbb{w})$
if $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}_{\mathsf{z}}$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}})$
$\mathsf{pmia}_{4,i}^{(1)} := ((D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}}, \pi_i)$
return $(\mathsf{pmia}_{4,i}^{(1)}, \mathsf{st}_i)$

$\mathsf{IA}_{4,2}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pmia}_{4,j}^{(1)})_{j\in\mathsf{IAS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $((D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}}, \pi_j)_{j\in\mathsf{SS}\backslash\{i\}} \leftarrow (\mathsf{pmia}_{4,j}^{(1)})_{j\in\mathsf{SS}\backslash\{i\}}$
parse $(\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}}) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
$\mathsf{CS}^{\mathsf{IA}} := \varnothing, \mathsf{proofs}_i := \varnothing$
for $j \in \mathsf{SS}\backslash\{i\}$ **do**
  $\mathbb{x}_j := (L_{\mathsf{SS},j}, \mathbf{z}_j, \mathbf{w}_j, c, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}}, C_j)$
  if $\Pi_{\mathsf{z}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{z}}}(\mathbb{x}_j, \pi_j) = 0$ **then**
    $\mathsf{CS}^{\mathsf{IA}} \leftarrow \mathsf{CS}^{\mathsf{IA}} \cup \{j\}$
  if $\Big(D_{i,j}^{(i)} \neq D_{i,j}^{(j)}\Big) \vee \Big(D_{j,i}^{(i)} \neq D_{j,i}^{(j)}\Big)$ **then**
    $\mathsf{CS}^{\mathsf{IA}} \leftarrow \mathsf{CS}^{\mathsf{IA}} \cup \{j\}$
    if $j \in \mathsf{sHS}$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
    // Proof honest commitment behaviour
    $\mathsf{proof}_j \leftarrow (i, j, \mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})$
    $\mathsf{proofs} \leftarrow \mathsf{proofs} \cup \{\mathsf{proof}_j\}$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{(j,k)\in\mathsf{SS}}, \mathsf{CS}^{\mathsf{IA}})$
return $(\mathsf{pmia}_{4,i}^{(2)} := \mathsf{proofs}, \mathsf{st}_i)$

Figure 6: Second hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

$\mathsf{IA}_{4,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}_j^{(k)})_{j\in\mathsf{SS},k\in[4]}, \mathsf{sk}_i, \mathsf{st}_i)$

1 : **req** $(\mathsf{IAS} = \mathsf{SS})$
2 : **parse** $(\mathbf{z}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pm}_j^{(3)})_{j\in\mathsf{SS}\setminus\{i\}}$
3 : **parse** $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
4 : $\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS}||\mathsf{M}||(\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}$
5 : **for** $j \in \mathsf{SS}\setminus\{i\}$ **do**
6 : $\quad (D_{i,j}^{(i)}, \delta_{i,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$
7 : $\quad (D_{j,i}^{(i)}, \delta_{j,i}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,i}, \mathsf{ctnt}_{\mathbf{z}})$
8 : $\mathbb{x} := (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}}, C_i)$
9 : $\mathbb{w} := (\llbracket \mathbf{s} \rrbracket_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j\in\mathsf{SS}})$
10 : $\pi_i \leftarrow \mathsf{Sim}_1(\mathbb{x})$
11 : **if** $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}_{\mathsf{z}}$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
12 : $\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}})$
13 : $\mathsf{pmia}_{4,i}^{(1)} := ((D_{i,j}^{(i)}, D_{j,i}^{(i)})_{j\in\mathsf{SS}}, \pi_i)$
14 : **return** $(\mathsf{pmia}_{4,i}^{(1)}, \mathsf{st}_i)$

$\mathsf{H}_{\mathsf{z}}(x)$

1 : **return** $\mathsf{Sim}_0(x)$

Figure 7: Third hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

$\mathsf{H}_{\mathsf{seed}}(\mathsf{seed})$

1 : **if** $i\|j\|\mathsf{rand} \leftarrow \mathsf{seed}$ correctly parses **then**
2 : $\quad$ **if** $\big((i,j) \in \mathsf{HS}^2\big) \wedge (\mathsf{rand} = \mathsf{rand}_{i,j})$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
3 : **if** $\mathsf{Q}_{\mathsf{H}_{\mathsf{seed}}}[\mathsf{seed}] = \bot$ **then**
4 : $\quad C^{\mathsf{seed}} \xleftarrow{\$} \{0,1\}^{2\lambda}$
5 : $\quad \mathsf{Q}_{\mathsf{H}_{\mathsf{seed}}}[\mathsf{seed}] \leftarrow C^{\mathsf{seed}}$
6 : **return** $\mathsf{Q}_{\mathsf{H}_{\mathsf{msk}}}[\mathsf{seed}]$

$\mathsf{H}_{\mathsf{rnd}}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}})$

1 : **if** $i\|j\|\mathsf{rand} \leftarrow \mathsf{seed}$ correctly parses **then**
2 : $\quad$ **if** $\big((i,j) \in \mathsf{HS}^2\big) \wedge (\mathsf{rand} = \mathsf{rand}_{i,j})$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
3 : **if** $\mathsf{Q}_{\mathsf{H}_{\mathsf{rnd}}}[\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}}] = \bot$ **then**
4 : $\quad \delta \xleftarrow{\$} \mathcal{R}_{\mathsf{msk}}$
5 : $\quad \mathsf{Q}_{\mathsf{H}_{\mathsf{rnd}}}[\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}}] \leftarrow C^{\mathsf{seed}}$
6 : **return** $\mathsf{Q}_{\mathsf{H}_{\mathsf{rnd}}}[\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}}]$

Figure 8: Fourth hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

that $\sum_{i,j\in\mathsf{HS}} Q^{(i,j)}_{\mathsf{H}_{\mathsf{seed}}} \leqslant Q_{\mathsf{H}_{\mathsf{seed}}}$ and $\sum_{i,j\in\mathsf{HS}} Q^{(i,j)}_{\mathsf{H}_{\mathsf{rnd}}} \leqslant Q_{\mathsf{H}_{\mathsf{rnd}}}$. In the initial phase, $\mathsf{rand}_{i,j}$ is picked uniformly at random from $\{0,1\}^\lambda$ within KeyGen. Since $\mathsf{rand}_{i,j}$ for $i,j \in \mathsf{HS}$ is information theoretically hidden from $\mathcal{A}$, the probability that a queried rand is equal to $\mathsf{rand}_{i,j}$ in one query is at most $1/2^\lambda$. Thus, the probability of aborting in $\mathsf{H}_{\mathsf{seed}}$ and $\mathsf{H}_{\mathsf{rnd}}$ for $i,j \in \mathsf{HS}$ are at most $Q^{(i,j)}_{\mathsf{H}_{\mathsf{seed}}}/2^\lambda$ and $Q^{(i,j)}_{\mathsf{H}_{\mathsf{rnd}}}/2^\lambda$, respectively. Due to the union bound across all honest users pairs, the probability of setting $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$ is bounded by $\sum_{i,j\in\mathsf{HS}} Q^{(i,j)}_{\mathsf{H}_{\mathsf{seed}}}/2^\lambda + Q^{(i,j)}_{\mathsf{H}_{\mathsf{rnd}}}/2^\lambda \leqslant (Q_{\mathsf{H}_{\mathsf{seed}}} + Q_{\mathsf{H}_{\mathsf{rnd}}})/2^\lambda$. Therefore, we have

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_3}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_4}_{\mathcal{A}}(\lambda)\right| \leqslant \frac{Q_{\mathsf{H}_{\mathsf{seed}}} + Q_{\mathsf{H}_{\mathsf{rnd}}}}{2^\lambda}.$$

Hybrid_5. In this hybrid, the challenger changes the way it samples δ in $\mathsf{DetMaskCom}$ for inputs $(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$ with $i,j \in \mathsf{HS}$. In more detail, the challenger initially sets up an empty table $\mathsf{T}_{\mathsf{msk}}[\cdot,\cdot,\cdot] := \bot$. When $\mathsf{DetMaskCom}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}})$ is invoked for $\mathsf{seed} = \mathsf{seed}_{i,j}$ or $\mathsf{seed} = \mathsf{seed}_{j,i}$ with $i,j \in \mathsf{HS}$, then the challenger checks if $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] = \bot$. If so, it samples $\delta \leftarrow \mathcal{R}_{\mathsf{msk}}$ and stores $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] \leftarrow \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m},\delta)$, where $\mathbf{m} := \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}, \mathsf{ctnt}_{\mathbf{z}})$ is as before. Then, it outputs $D := \mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}]$. This is formalized in Fig. 9. Note that to capture the change in output and input values, an additional helper algorithm $\mathsf{DetMaskComMod}$ is introduced for the case when $\mathsf{DetMaskCom}$ is called for $\mathsf{seed}_{i,j}$ or $\mathsf{seed}_{j,i}$ with $i,j \in \mathsf{HS}$.

$\mathsf{DetMaskComMod}(i,j,\mathsf{ctnt}_{\mathbf{z}})$

// Helper algorithm for commitments $D_{i,j}$ with $i,j \in \mathsf{HS}$

$\mathbf{m} := \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$
if $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] = \bot$ **then**
  $\delta \leftarrow \mathcal{R}_{\mathsf{msk}}$
  $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] \leftarrow \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m};\delta)$
return $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}]$

$\mathsf{IA}_{4,1}(\mathsf{vk},\mathsf{inf},\mathsf{SS},\mathsf{IAS},\mathsf{sid},\mathsf{M},i,(\mathsf{pm}^{(k)}_j)_{j\in\mathsf{SS},k\in[4]},\mathsf{sk}_i,\mathsf{st}_i)$

req $(\mathsf{IAS} = \mathsf{SS})$
parse $(\mathbf{z}_j)_{j\in\mathsf{SS}\setminus\{i\}} \leftarrow (\mathsf{pm}^{(3)}_j)_{j\in\mathsf{SS}\setminus\{i\}}$
parse $(\mathsf{SS},\mathsf{M},(\mathsf{cmt}_j,\mathbf{w}_j)_{j\in\mathsf{SS}},\mathbf{r}_i,\mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
$\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS}||\mathsf{M}||(\mathsf{cmt}_j,\mathbf{w}_j)_{j\in\mathsf{SS}}$
for $j \in \mathsf{SS}\setminus\{i\}$ **do**
  if $j \in \mathsf{sHS}$ **then**
    $D^{(i)}_{i,j} := \mathsf{DetMaskComMod}(i,j,\mathsf{ctnt}_{\mathbf{z}})$
    $D^{(i)}_{j,i} := \mathsf{DetMaskComMod}(j,i,\mathsf{ctnt}_{\mathbf{z}})$
  else
    $(D^{(i)}_{i,j},\delta_{i,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{i,j},\mathsf{ctnt}_{\mathbf{z}})$
    $(D^{(i)}_{j,i},\delta_{j,i}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,i},\mathsf{ctnt}_{\mathbf{z}})$
$\mathbb{x} := (L_{\mathsf{SS},i},\mathbf{z}_i,\mathbf{w}_i,c,(D^{(i)}_{i,j},D^{(i)}_{j,i})_{j\in\mathsf{SS}},C_i)$
$\pi_i \leftarrow \mathsf{Sim}_1(\mathbb{x})$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS},\mathsf{M},(\mathbf{w}_j,\mathbf{z}_j,D^{(i)}_{i,j},D^{(i)}_{j,i})_{j\in\mathsf{SS}})$
$\mathsf{pmia}^{(1)}_{4,i} := ((D^{(i)}_{i,j},D^{(i)}_{j,i})_{j\in\mathsf{SS}},\pi_i)$
return $(\mathsf{pmia}^{(1)}_{4,i},\mathsf{st}_i)$

Figure 9: Fifth hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

To analyze the advantage of $\mathcal{A}$ in Hybrid_5, let us start with some observations.

1. There is a one-to-one mapping from $\mathsf{seed}_{i,j}$ to (i,j).
2. The helper algorithm $\mathsf{DetMaskCom}$ is invoked in Hybrid_4 on input $(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$ iff the helper algorithm $\mathsf{DetMaskComMod}$ is invoked in Hybrid_5 on input $(i,j,\mathsf{ctnt}_{\mathbf{z}})$.
3. The abort condition introduced in Hybrid_4 ensures that the oracle $\mathsf{H}_{\mathsf{rnd}}$ is never invoked on input $(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$ by $\mathcal{A}$, where $i,j \in \mathsf{HS}$.

Due to the above facts, we conclude that the randomness δ is identically distributed in invocations of $\mathsf{DetMaskCom}$ and $\mathsf{DetMaskComMod}$ for $i,j \in \mathsf{HS}$ in Hybrid_4 and Hybrid_5, respectively. Consequently, also $D^{(i)}_{i,j}$ and $D^{(i)}_{j,i}$ are identically distributed, and we have

$$\mathsf{Adv}^{\mathsf{Hybrid}_4}_{\mathcal{A}}(\lambda) = \mathsf{Adv}^{\mathsf{Hybrid}_5}_{\mathcal{A}}(\lambda).$$

Hybrid_6. In this hybrid, the challenger outputs commitments D to $\mathbf{0}$ in $\mathsf{DetMaskComMod}$. That is, when $\mathsf{DetMaskComMod}$ is invoked on input $(i,j,\mathsf{ctnt}_{\mathbf{z}})$ with $i,j \in \mathsf{HS}$, then the challenger proceeds as in Hybrid_5, except it sets $\mathbf{m} := \mathbf{0}$. This is formalized in Fig. 10.

$\mathsf{DetMaskComMod}(i,j,\mathsf{ctnt}_{\mathbf{z}})$

// Helper algorithm for commitments $D_{i,j}$ with $i,j \in \mathsf{HS}$

1 : **if** $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] = \bot$ **then**
2 : $\quad \delta \leftarrow \mathcal{R}_{\mathsf{msk}}$
3 : $\quad \mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] \leftarrow \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{0},\delta)$
4 : **return** $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}]$

Figure 10: Sixth hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

It is straightforward to construct an adversary $\mathcal{B}_2$ on the hiding property of $\mathsf{C}_{\mathsf{msk}}$ that succeeds if $\mathcal{A}$ distinguishes Hybrid_5 and Hybrid_6. Initially, the adversary $\mathcal{B}_2$ obtains $\mathsf{pp}_{\mathsf{msk}}$ and access to a commitment oracle $\mathcal{C}_b$ from the hiding challenger, where $b \in \{0,1\}$. First, $\mathcal{B}_2$ invokes $\mathcal{A}$ on input $\mathsf{tspar} = (N,T,T_{\mathsf{ia}})$ and obtains $\mathsf{state}_{\mathcal{A}}$ and $\mathsf{CS} \subseteq [N]$ with $|\mathsf{CS}| \leqslant T-1$. Next, $\mathcal{B}_2$ runs $\mathsf{KeyGen}(\mathsf{tspar})$ as in Hybrid_5 to obtain $(\mathsf{vk},(\mathsf{sk}_i)_{i\in[N]},\mathsf{inf})$, except it uses the obtained $\mathsf{pp}_{\mathsf{msk}}$ in inf. Now, $\mathcal{B}_2$ runs $\mathcal{A}$ on input $(\mathsf{vk},\mathsf{inf},(\mathsf{sk})_{i\in\mathsf{CS}},\mathsf{st}_{\mathcal{A}})$, where the oracles are simulated as in Hybrid_5, except that $\mathsf{DetMaskComMod}$ is simulated as follows by $\mathcal{B}_2$. $\mathcal{B}_2$ checks if $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] = \bot$. If so, it sets $m_0 := \mathbf{m}$ and $m_1 := \mathbf{0}$, where $\mathbf{m} := \mathsf{H}_{\mathsf{msk}}(\mathsf{seed}_{i,j},\mathsf{ctnt}_{\mathbf{z}})$. Then, it sets $\mathsf{T}_{\mathsf{msk}}[i,j,\mathsf{ctnt}_{\mathbf{z}}] := \mathcal{C}(m_0,m_1)$. Finally, $\mathcal{B}_2$ checks if $\mathcal{A}$'s provided forgery is valid and outputs 1 if so, else outputs 0.

Observe that $\mathcal{B}_2$ perfectly simulates Hybrid_5 if $b=0$ and Hybrid_6 if $b=1$. Consequently, we have $\mathsf{Time}(\mathcal{B}_2) \approx \mathsf{Time}(\mathcal{A})$ and

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_5}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_6}_{\mathcal{A}}(\lambda)\right| \leqslant \mathsf{Adv}^{\mathsf{hide}}_{\mathcal{B}_2,\mathsf{C}_{\mathsf{msk}}}(\lambda).$$

Note that as consequence, the IA oracles are now simulated exclusively based on information known to the unforgeability adversary $\mathcal{A}$. Next, we move towards a game where also the additional KeyGen outputs (with respect to $\mathsf{TRaccoon}$ [KRT24], *i.e.*, the values inf and γ_i) are also simulated exclusively with information known to $\mathcal{A}$.

Hybrid_7. In this hybrid, the challenger simulates KeyGen as follows. The challenger samples $C^{\mathsf{seed}}_{i,j} \leftarrow \{0,1\}^{2\lambda}$ at random, instead of $C^{\mathsf{seed}}_{i,j} := \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{i,j})$. Also, the challenger sets $C_i \leftarrow \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\mathbf{0};\gamma_i)$ for $i \in \mathsf{HS}$, where $\gamma_i \xleftarrow{\$} \mathcal{R}_{\mathsf{share}}$. This is formalized in Fig. 11.

KeyGen(tspar)

$(\mathbf{s}, \mathbf{e}) \xleftarrow{\$} \mathcal{D}_{\mathbf{t}}^{\ell} \times \mathcal{D}_{\mathbf{t}}^{k}$

$\mathbf{t} := \lfloor \mathbf{A}\mathbf{s} + \mathbf{e} \rceil_{\nu_{\mathbf{t}}} \in \mathcal{R}_{q_{\nu_{\mathbf{t}}}}^{k}$

for $(i, j) \in [N]^2$ **do**

  $\mathsf{rand}_{i,j} \xleftarrow{\$} \{0,1\}^{\lambda}$

  $\mathsf{seed}_{i,j} := i \| j \| \mathsf{rand}_{i,j}$

  if $(i, j) \in \mathsf{sHS}^2$ **then**

    $C_{i,j}^{\mathsf{seed}} \xleftarrow{\$} \{0,1\}^{2\lambda}$

  else

    $C_{i,j}^{\mathsf{seed}} := \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{i,j})$

$(\vec{\mathsf{seed}}_i)_{i \in [N]} := \left((\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j \in [N]} \right)_{i \in [N]}$

$\vec{P} \xleftarrow{\$} \mathcal{R}_q^{\ell}[X]$ with $\deg(\vec{P}) = T - 1, \vec{P}(0) = \mathbf{s}$

$(\llbracket \mathbf{s} \rrbracket_i)_{i \in [N]} := (\vec{P}(i))_{i \in [N]}$

for $i \in [N]$ **do**

  $\gamma_i \xleftarrow{\$} \mathcal{R}_{\mathsf{share}}$

  if $i \in \mathsf{HS}$ **then**

    $C_i \leftarrow \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\mathbf{0}; \gamma_i)$

  else

    $C_i \leftarrow \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\llbracket \mathbf{s} \rrbracket_i; \gamma_i)$

$\mathsf{vk} := (\mathsf{tspar}, \mathbf{t})$

$\mathsf{inf} := ((C_{i,j}^{\mathsf{seed}})_{i,j \in [N]}, (C_i)_{i \in [N]})$

$(\mathsf{sk}_i)_{i \in [N]} := \left(\llbracket \mathbf{s} \rrbracket_i, \vec{\mathsf{seed}}_i, \gamma_i \right)_{i \in [N]}$

return $(\mathsf{vk}, (\mathsf{sk}_i)_{i \in [N]}, \mathsf{inf})$

Figure 11: Seventh hybrid of the unforgeability proof of the TRaccoon-IA scheme. Differences with the previous hybrid are highlighted in blue.

Recall that $\mathsf{seed}_{i,j}$ is never queried to $\mathsf{H}_{\mathsf{seed}}$ by $\mathcal{A}$ due to the abort condition added in Hybrid_4. Thus, it is straightforward to construct an adversary $\mathcal{B}_3$ on hiding of $\mathsf{C}_{\mathsf{share}}$ such that $\mathsf{Time}(\mathcal{B}_3) \approx \mathsf{Time}(\mathcal{A})$ and

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_6}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda)\right| \leqslant \mathsf{Adv}^{\mathsf{hide}}_{\mathcal{B}_3,\mathsf{C}_{\mathsf{share}}}(\lambda).$$

Finally, we bound $\mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda)$. Using Lemma 5.3, which we will prove later, we have

$$\mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T, \bot)$$

and $\mathsf{Time}(\mathcal{B}) \approx \mathsf{Time}(\mathcal{A})$ where $\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T, \bot)$ is an advantage of $\mathcal{B}$ against the unforgeability game of $\mathsf{TRaccoon}$ with $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ given in Section 5.1. Moreover, due to Lemma 5.1, there exist adversaries $\mathcal{B}_4$ and $\mathcal{B}_5$ against the $\mathsf{Hint\text{-}MLWE}_{q,\ell,k,T\cdot Q,\sigma_{\mathbf{t}},\sigma_{\mathbf{w}},\mathcal{C}}$ and $\mathsf{SelfTargetMSIS}_{q,\ell+1,k,\mathsf{H}_c,\mathcal{C},B_{\mathsf{stmsis}}}$ problems, respectively, such that

$$\begin{aligned}\mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T, \bot) \leqslant \mathsf{Adv}^{\mathsf{Hint\text{-}MLWE}}_{\mathcal{B}_4}(\lambda) + \mathsf{Adv}^{\mathsf{SelfTargetMSIS}}_{\mathcal{B}_5}(\lambda) + \frac{T\cdot Q_{\mathsf{S}}\cdot(Q_{\mathsf{H}_{\mathsf{com}}} + Q_{\mathsf{H}_c} + 2T\cdot Q_{\mathsf{S}})}{2^{n-1}} \\ + \frac{Q_{\mathsf{H}_{\mathsf{msk}}}}{2^{\lambda}} + \frac{(Q_{\mathsf{H}_{\mathsf{com}}} + T\cdot Q_{\mathsf{S}})^2 + Q_{\mathsf{H}_{\mathsf{com}}}}{2^{2\lambda}} + \mathsf{negl}(\lambda),\end{aligned}$$

where $\mathsf{Time}(\mathcal{B}_4), \mathsf{Time}(\mathcal{B}_5) \approx \mathsf{Time}(\mathcal{B}')$ and parameters given in Section 5.1. We can conclude this proof by combining all arguments. □

Below, we prove Lemma 5.3

Lemma 5.3. *There exist an adversary against the unforgeability game of* $\mathsf{TRaccoon}$ *with* $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ *given in Section 5.1 such that*

$$\mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T, \bot)$$

where $\mathsf{Time}(\mathcal{B}) \approx \mathsf{Time}(\mathcal{A})$.

Proof. We cannot immediately construct $\mathcal{B}'$ from $\mathcal{A}$ against Hybrid_7 since the abort conditions in $\mathsf{IA}_{4,1}$, in $\mathsf{H}_{\mathsf{seed}}$, and in $\mathsf{H}_{\mathsf{rnd}}$ that we added in second and fourth hybrids require the secret information for honest users. Thus, to prove this lemma, we first consider an intermediate game Hybrid'_7. The game Hybrid'_7 is identical to Hybrid_7 except that the aforementioned abort conditions are removed. Clearly, this can at most improve the advantage of $\mathcal{A}$, and we have

$$\mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{Hybrid}'_7}_{\mathcal{A}}(\lambda). \tag{2}$$

Now we construct an adversary $\mathcal{B}$ on the unforgeability of $\mathsf{TRaccoon}$ using the adversary $\mathcal{A}$ on Hybrid'_7. Adversary $\mathcal{B}$ simulates the challenger in Hybrid'_7 by accessing the random and signing oracles of the unforgeability game of $\mathsf{TRaccoon}$. More concretely, $\mathcal{B}$ is given $\mathsf{tspar} = (\mathbf{A}, N, T)$ by the $\mathsf{TRaccoon}$ unforgeability challenger. First, $\mathcal{B}$ sets $T_{\mathsf{ia}} = T$ and runs $\mathcal{A}$ on input $(\mathbf{A}, N, T, T_{\mathsf{ia}})$. After receiving a set of corrupted users CS, $\mathcal{B}$ outputs CS and obtains vk and $\mathsf{sk}_i = (\llbracket \mathbf{s} \rrbracket_i, \vec{\mathsf{seed}}_i)$ for $i \in \mathsf{CS}$. Then, $\mathcal{B}$ generates $C^{\mathsf{seed}}_{i,j}$ and C_i as in Hybrid'_7 and runs $\mathcal{A}$ by simulating oracle queries as follows. Adversary $\mathcal{B}$ simulates random oracles $\mathsf{H}_{\mathsf{seed}}, \mathsf{H}_{\mathsf{rnd}}$, and the IA oracles as in Hybrid'_7. Note that this is possible as their simulation does not require any secret information of honest users. For $\mathsf{H}_{\mathsf{msk}}, \mathsf{H}_c, \mathsf{H}_{\mathsf{com}}$, and signing oracles, it forwards the query to the oracle provided by the $\mathsf{TRaccoon}$ unforgeability challenger, and returns the received responses to $\mathcal{A}$. Because the signing protocol of $\mathsf{TRaccoon\text{-}IA}$ is identical to that of $\mathsf{TRaccoon}$, adversary $\mathcal{B}$ perfectly simulates Hybrid'_7. Finally, it outputs $\mathcal{A}$'s output as forgery. Since $\mathcal{B}$ does not make a singing query on M^* as long as $\mathsf{M}^* \in \mathsf{Q}_{\mathsf{M}}$, a valid $\mathcal{A}$'s output satisfies the winning conditions of unforgeability game of $\mathsf{TRaccoon}$. Therefore, we have

$$\mathsf{Adv}^{\mathsf{Hybrid}'_7}_{\mathcal{A}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T)$$

where $\mathsf{Time}(\mathcal{B}) \approx \mathsf{Time}(\mathcal{A})$. Via Eq. (2), we obtain

$$\mathsf{Adv}^{\mathsf{Hybrid}_7}_{\mathcal{A}}(\lambda) \leqslant \mathsf{Adv}^{\mathsf{ts\text{-}uf}}_{\mathsf{TRaccoon},\mathcal{B}}(\lambda, N, T)$$

where $\mathsf{Time}(\mathcal{B}) \approx \mathsf{Time}(\mathcal{A})$. □

5.4 Identifiable Abort

The following theorem establishes identifiable abort of TRaccoon-IA.

Theorem 5.2. *The threshold signature with identifiable abort* TRaccoon-IA *in Figs. 4 and 5 instantiated with* $\mathsf{hard\text{-}param}_{\mathsf{TRaccoon\text{-}IA}}$ *is identifiable abort under the correctness and knowledge soundness of NIZK* Π_{z} *and the binding of commitment schemes.*

Formally, for any N, T, T_{ia} *with* $T = T_{\mathsf{ia}} \leqslant N$ *and an adversary* $\mathcal{A}$ *against the identifiable abort game making at most* $Q_{\mathsf{H}_{\mathsf{z}}}$ *and* Q_{S} *queries to the random oracle* H_{z} *for* Π_{z} *and the signing, aggregation, and IA oracles, respectively, there exists adversaries* $\mathcal{B}_{\mathsf{msk}}$ *and* $\mathcal{B}_{\mathsf{share}}$ *against the binding of commitment schemes* $\mathsf{C}_{\mathsf{msk}}$ *and* $\mathsf{C}_{\mathsf{share}}$*, respectively, such that*

$$\begin{aligned}\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TRaccoon\text{-}IA},\mathcal{A}}(\lambda, N, T, T_{\mathsf{ia}}) \leqslant\ & \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_{\mathsf{msk}},\mathsf{C}_{\mathsf{msk}}}(\lambda) + \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_{\mathsf{share}},\mathsf{C}_{\mathsf{share}}}(\lambda) + \kappa(\lambda, Q_{\mathsf{H}_{\mathsf{z}}}) \\ & + T \cdot Q_{\mathsf{S}} \cdot \left(2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}\right) + 2^{-\frac{n(\ell+k)}{10}} + \mathsf{negl}(\lambda)\end{aligned}$$

where κ *is the knowledge error of* Π_{z}*,* $\mathsf{Time}(\mathcal{B}_{\mathsf{msk}}), \mathsf{Time}(\mathcal{B}_{\mathsf{share}}) \approx \mathsf{poly}(\lambda, Q_{\mathsf{H}_{\mathsf{z}}}) \cdot T_{\mathcal{A}}$*, and* $\mathsf{poly}(\lambda, Q_{\mathsf{H}_{\mathsf{z}}})$ *denotes the runtime overhead of* Ext.

Proof. Let $\mathcal{A}$ be a PPT adversary against the identifiable abort game. Denote $\mathsf{sCS} = \mathsf{CS} \cap \mathsf{SS}$ as the corrupt signers within the signer set, and similarly denote $\mathsf{sHS} = \mathsf{HS} \cap \mathsf{SS}$. As the proof of Theorem 5.1, for clarity, when introducing abort conditions within the hybrids, instead of saying the challenger aborts, we introduce a flag $\mathsf{f}_{\mathsf{fail}}$ (cf. Fig. 12) which is initially set to $\bot$. If some abort condition holds, then the challenger sets $\mathsf{f}_{\mathsf{fail}} = \top$. At the end of the game, the challenger checks whether $\mathsf{f}_{\mathsf{fail}} = \bot$, and outputs 0 if not. Note that we ensure that $\mathsf{f}_{\mathsf{fail}}$ is never reset to $\bot$ within the game.

Looking ahead, it is worth highlighting that we extensively rely on a synchronous authenticated broadcast channel, implicit in the identifiable abort game. Concretely, this is used to argue that the view of the honest signers are consistent and that an honest signer cannot be detected as misbehaving by another honest signer. For clarity, we highlight this in the proof below.

Hybrid_1**.** This hybrid corresponds to the original game $\mathsf{Game}^{\mathsf{ts\text{-}ia}}_{\mathsf{TRaccoon\text{-}IA},\mathcal{A}}$ from Fig. 3. Note that $\mathsf{H}_{\mathsf{msk}}, \mathsf{H}_c$, $\mathsf{H}_{\mathsf{seed}}$ and $\mathsf{H}_{\mathsf{rnd}}$ are modeled as random oracles.

Hybrid_2**.** In this hybrid, the challenger introduces the aforementioned flag $\mathsf{f}_{\mathsf{fail}}$. The challenger sets $\mathsf{f}_{\mathsf{fail}} = \top$ if after $\mathcal{A}$ outputs sid^*, it holds that $\mathsf{rnd}^*_\bot \in \{1, 2, 3\}$. Moreover, it sets $\mathsf{f}_{\mathsf{fail}} = \top$ if $\|(\mathbf{s}, \mathbf{e})\|_2$ is larger than $e^{1/4} \cdot \sigma_{\mathbf{t}} \cdot \sqrt{n(k+\ell)}$ in KeyGen. This is formalized in Fig. 12.

It suffices to prove that the probability of $\mathsf{f}_{\mathsf{fail}}$ being set to $\top$ is negligible in case there exists $i, j \in \mathsf{sHS}^*$ such that $\mathsf{CS}^{\mathsf{IA}*}_i \neq \mathsf{CS}^{\mathsf{IA}*}_j$, $\mathsf{CS}^{\mathsf{IA}*}_i = \varnothing$, or $\mathsf{CS}^{\mathsf{IA}*}_i \not\subseteq \mathsf{CS}$ (*i.e.*, the original winning condition of the identifiable abort game). We refer to Fig. 12 for the notations.

We first consider the cases $\mathsf{rnd}^*_\bot \in \{1, 2\}$ (*i.e.*, an abort occurred in rounds 1 or 2 of the signing protocol). By construction, an honest user never aborts in the first two rounds, implying that the identified malicious signers are included in the corrupted signer set: $\mathsf{CS}^{\mathsf{IA}*}_i \subseteq \mathsf{CS}$ for all $i \in \mathsf{sHS}^*$. Moreover, since an abort occurred, we also have $\mathsf{CS}^{\mathsf{IA}*}_i \neq \varnothing$ for all $i \in \mathsf{sHS}^*$. Lastly, by construction, we also have $\mathsf{CS}^{\mathsf{IA}*}_i = \mathsf{CS}^{\mathsf{IA}*}_j$ for $i, j \in \mathsf{sHS}^*$ as the view of the honest signers are consistent. This establishes that $\mathsf{f}_{\mathsf{fail}}$ cannot be set to $\top$ in case $\mathsf{rnd}^*_\bot \in \{1, 2\}$.

The case for $\mathsf{rnd}^*_\bot = 3$ consists of the same check. The only difference is that there exists an honest signer $i \in \mathsf{sHS}^*$ such that for some $j \in \mathsf{SS}^*$, $\mathsf{pm}^{(3)}_j = \bot$ or $\mathsf{cmt}_j \neq \mathsf{H}_{\mathsf{com}}(j, \mathbf{w}_j)$; namely, we have an additional check on the hash commitment. Given that the views of the honest signers are consistent, an honest signer can never trigger these condition. As such, j must be included in sCS^*. Following the same argument as above, this establishes that $\mathsf{f}_{\mathsf{fail}}$ cannot be set to $\top$ in case $\mathsf{rnd}^*_\bot = 3$.

Lastly, due to standard Gaussian tail bounds (cf. Lemma 2.1), the probability of setting $\mathsf{f}_{\mathsf{fail}} = \top$ in KeyGen is at most $2^{-\frac{n(\ell+k)}{10}}$. By the above discussion, the advantage of $\mathcal{A}$ is not impacted, and we have

$$\left|\mathsf{Adv}^{\mathsf{Hybrid}_1}_{\mathcal{A}}(\lambda) - \mathsf{Adv}^{\mathsf{Hybrid}_2}_{\mathcal{A}}(\lambda)\right| \leqslant 2^{-\frac{n(\ell+k)}{10}}.$$

Hybrid_2

$\mathsf{Game}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(1^\lambda, N, T, T_{\mathsf{ia}})$

$\mathsf{f}_{\mathsf{fail}} := \bot$
$\mathsf{Q}_{\mathsf{Sign}}[\cdot] := \varnothing, \mathsf{Q}_{\mathsf{ia}}[\cdot] := \varnothing, \mathsf{Round}_\bot[\cdot] := \varnothing$
$\mathsf{tspar} \xleftarrow{\$} \mathsf{Setup}(1^\lambda, N, T, T_{\mathsf{ia}})$
$(\mathsf{CS}, \mathsf{st}_{\mathcal{A}}) \xleftarrow{\$} \mathcal{A}^{\mathsf{H}}(\mathsf{tspar})$
req $(\mathsf{CS} \subseteq [N]) \wedge (|\mathsf{CS}| \leqslant T-1)$
$\mathsf{HS} := [N] \backslash \mathsf{CS}$
for $i \in \mathsf{HS}$ **do** $\mathsf{st}_i := \varnothing$
$(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf}) \xleftarrow{\$} \mathsf{KeyGen}(\mathsf{tspar})$
// All Oracles are the same as in $\mathsf{Game}^{\mathsf{ts\text{-}uf}}$
$\mathsf{oracles} := \left((\mathcal{O}_{\mathsf{Sign}_r})_{r\in[R]}, \mathcal{O}_{\mathsf{Agg}}, (\mathcal{O}_{\mathsf{IA}_{\mathsf{rnd}_\bot,r}})^{\mathsf{rnd}_\bot\in[R+1],}_{r\in[R^{(\mathsf{rnd}_\bot)}_{\mathsf{ia}}]}, \mathsf{H}\right)$
$\mathsf{sid}^* \xleftarrow{\$} \mathcal{A}^{\mathsf{oracles}}(\mathsf{vk}, \mathsf{inf}, (\mathsf{sk}_i)_{i\in\mathsf{CS}}, \mathsf{st}_{\mathcal{A}})$
req $\left(\mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}^*] = (R^{(\mathsf{rnd}^*_\bot)}_{\mathsf{ia}}, \mathsf{rnd}^*_\bot, \cdot, \cdot, \cdot)\right)$
parse $(R^{(\mathsf{rnd}^*_\bot)}_{\mathsf{ia}}, \mathsf{rnd}^*_\bot, \mathsf{SS}^*, \mathsf{IAS}^*, \mathsf{M}^*, (\mathsf{CS}^{\mathsf{IA}*}_i)_{i\in\mathsf{HS}\cap\mathsf{IAS}*}) \leftarrow \mathsf{Q}_{\mathsf{ia}}[\mathsf{sid}^*]$
$\mathsf{sHS}^* \leftarrow \mathsf{HS} \cap \mathsf{IAS}^*$
if $\mathsf{rnd}^*_\bot \in \{1,2,3\}$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
if $(\exists i, j \in \mathsf{sHS}^*, \mathsf{CS}^{\mathsf{IA}*}_i \neq \mathsf{CS}^*_j)$
$\quad \vee (\exists i \in \mathsf{sHS}^*, \mathsf{CS}^{\mathsf{IA}*}_i = \varnothing) \vee (\exists i \in \mathsf{sHS}^*, \mathsf{CS}^{\mathsf{IA}*}_i \nsubseteq \mathsf{CS})$
  return $(\mathsf{f}_{\mathsf{fail}} = \bot)$
return 0

$\mathsf{KeyGen}(\mathsf{tspar})$

$(\mathbf{s}, \mathbf{e}) \xleftarrow{\$} \mathcal{D}^\ell_{\mathbf{t}} \times \mathcal{D}^k_{\mathbf{t}}$
if $\|(\mathbf{s}, \mathbf{e})\|_2 > e^{1/4} \cdot \sigma_{\mathbf{t}} \cdot \sqrt{n(k+\ell)}$ **then**
  $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
$\mathbf{t} := \lfloor \mathbf{As} + \mathbf{e} \rceil_{\nu_{\mathbf{t}}} \in \mathcal{R}^k_{q_{\nu_{\mathbf{t}}}}$
for $(i,j) \in [N]^2$ **do**
  $\mathsf{rand}_{i,j} \xleftarrow{\$} \{0,1\}^\lambda$
  $\mathsf{seed}_{i,j} := i \| j \| \mathsf{rand}_{i,j}$
  $C^{\mathsf{seed}}_{i,j} := \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{i,j})$
$(\vec{\mathsf{seed}}_i)_{i\in[N]} := \left((\mathsf{seed}_{i,j}, \mathsf{seed}_{j,i})_{j\in[N]}\right)_{i\in[N]}$
$\vec{P} \xleftarrow{\$} \mathcal{R}^\ell_q[X]$ with $\deg(\vec{P}) = T-1, \vec{P}(0) = \mathbf{s}$
$(\llbracket \mathbf{s} \rrbracket_i)_{i\in[N]} := (\vec{P}(i))_{i\in[N]}$
$\mathsf{pp}_{\mathsf{msk}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{msk}}}, \mathsf{pp}_{\mathsf{share}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{share}}}$
for $i \in [N]$ **do**
  $\gamma_i \xleftarrow{\$} \mathcal{R}_{\mathsf{share}}$
  $C_i \leftarrow \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\llbracket \mathbf{s} \rrbracket_i; \gamma_i)$
$\mathsf{crs}_{\mathsf{z}} \xleftarrow{\$} \{0,1\}^{\ell_{\mathsf{z}}}$
$\mathsf{vk} := (\mathsf{tspar}, \mathbf{t})$
$\mathsf{inf} := (\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}, (C^{\mathsf{seed}}_{i,j})_{i,j\in[N]}, (C_i)_{i\in[N]})$
$(\mathsf{sk}_i)_{i\in[N]} := (\llbracket \mathbf{s} \rrbracket_i, \vec{\mathsf{seed}}_i, \gamma_i)_{i\in[N]}$
return $(\mathsf{vk}, (\mathsf{sk}_i)_{i\in[N]}, \mathsf{inf})$

Hybrid_3

$\mathsf{IA}_{4,1}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pm}^{(k)}_j)_{j\in\mathsf{SS}, k\in[4]}, \mathsf{sk}_i, \mathsf{st}_i)$

// Aggregated signature does not verify
req $(\mathsf{IAS} = \mathsf{SS})$
parse $(\mathbf{z}_j)_{j\in\mathsf{SS}\backslash\{i\}} \leftarrow (\mathsf{pm}^{(3)}_j)_{j\in\mathsf{SS}\backslash\{i\}}$
parse $(\mathsf{SS}, \mathsf{M}, (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}, \mathbf{r}_i, \mathbf{z}_i) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
$\mathsf{ctnt}_{\mathbf{z}} := \mathsf{SS} || \mathsf{M} || (\mathsf{cmt}_j, \mathbf{w}_j)_{j\in\mathsf{SS}}$
for $j \in \mathsf{SS}\backslash\{i\}$ **do**
  $(D^{(i)}_{i,j}, \delta_{i,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{i,j}, \mathsf{ctnt}_{\mathbf{z}})$
  $(D^{(i)}_{j,i}, \delta_{j,i}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,i}, \mathsf{ctnt}_{\mathbf{z}})$
$\mathbb{x} := (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D^{(i)}_{i,j}, D^{(i)}_{j,i})_{j\in\mathsf{SS}}, C_i)$
$\mathbb{w} := (\mathbf{s}_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j\in\mathsf{SS}})$
$\pi_i \leftarrow \Pi_{\mathsf{z}}.\mathsf{Prove}^{\mathsf{H}}_{\mathsf{z}}(\mathbb{x}, \mathbb{w})$
if $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}_{\mathsf{z}}$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
if $\Pi_{\mathsf{z}}.\mathsf{Verify}^{\mathsf{H}_{\mathsf{z}}}(\mathbb{x}, \pi_i) = \bot$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
$\mathsf{st}_i[\mathsf{sid}] \leftarrow (\mathsf{SS}, \mathsf{M}, (\mathbf{w}_j, \mathbf{z}_j, D^{(i)}_{i,j}, D^{(i)}_{j,i})_{j\in\mathsf{SS}})$
$\mathsf{pmia}^{(1)}_{4,i} := ((D^{(i)}_{i,j}, D^{(i)}_{j,i})_{j\in\mathsf{SS}}, \pi_i)$
return $(\mathsf{pmia}^{(1)}_{4,i}, \mathsf{st}_i)$

Hybrid_4

$\mathsf{IA}_{4,3}(\mathsf{vk}, \mathsf{inf}, \mathsf{SS}, \mathsf{IAS}, \mathsf{sid}, \mathsf{M}, i, (\mathsf{pmia}^{(2)}_{4,j})_{j\in\mathsf{IAS}}, \mathsf{sk}_i, \mathsf{st}_i)$

parse $(\mathsf{proofs}_j)_{j\in\mathsf{SS}} \leftarrow (\mathsf{pmia}^{(2)}_{4,j})_{j\in\mathsf{SS}}$
parse $(\mathsf{SS}, \mathsf{M}, (D^{(j)}_{j,k}, D^{(j)}_{k,j})_{(j,k)\in\mathsf{SS}}, \mathsf{CS}^{\mathsf{IA}}) \leftarrow \mathsf{st}_i[\mathsf{sid}]$
for $(j,k) \in (\mathsf{SS}\backslash\{i\})^2$ s.t. $j \neq k$ **do**
  if $\left(D^{(j)}_{j,k} \neq D^{(k)}_{j,k}\right) \vee \left(D^{(j)}_{k,j} \neq D^{(k)}_{k,j}\right)$ **then**
    if $(j,k) \in \mathsf{HS}^2$ **then** $\mathsf{f}_{\mathsf{fail}} \leftarrow \top$
    // Check behaviour of user j during IA
    $\mathsf{bad\text{-}proof} := (j, k, \mathsf{seed}_{j,k}, \mathsf{seed}_{k,j}) \notin \mathsf{proofs}_j$
    $\mathsf{bad\text{-}seed} := C^{\mathsf{seed}}_{j,k} \neq \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{j,k})$
    $\mathsf{bad\text{-}seed} \leftarrow \mathsf{bad\text{-}seed} \vee (C^{\mathsf{seed}}_{k,j} \neq \mathsf{H}_{\mathsf{seed}}(\mathsf{seed}_{k,j}))$
    $(\overline{D}^{(j)}_{j,k}, \overline{\delta}_{j,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{j,k}, \mathsf{ctnt}_{\mathbf{z}})$
    $(\overline{D}^{(j)}_{k,j}, \overline{\delta}_{k,j}) := \mathsf{DetMaskCom}(\mathsf{seed}_{k,j}, \mathsf{ctnt}_{\mathbf{z}})$
    $\mathsf{bad\text{-}vm} := ((\overline{D}^{(j)}_{j,k}, \overline{D}^{(j)}_{k,j}) \neq (D^{(j)}_{j,k}, D^{(j)}_{k,j}))$
    if bad-proof $\vee$ bad-seed $\vee$ bad-vm **then**
      $\mathsf{CS}^{\mathsf{IA}} \leftarrow \mathsf{CS}^{\mathsf{IA}} \cup \{j\}$
return $\mathsf{pmia}^{(3)}_{4,i} := \mathsf{CS}^{\mathsf{IA}}, \mathsf{st}_i$

Figure 12: Second, third, and forth hybrids of the identifiable abort proof. Differences with the previous hybrid are highlighted in blue.

Hybrid_3. In this hybrid, the challenger sets $\mathsf{f}_{\mathsf{fail}} = \top$ in $\mathcal{O}_{\mathsf{IA}_{4,1}}$ if $(\mathbb{x}, \mathbb{w}) \notin \mathsf{R}_{\mathsf{z}}$ or if the proof π_i does not verify for $i \in \mathsf{sHS}$. This is formalized in Fig. 12.

It is easy to check that the probability $\mathsf{f}_{\mathsf{fail}}$ being set to $\top$ only changes negligibly. Application of the standard Gaussian tail bound shows that we have $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}_{\mathsf{z}}$ except with probability $2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}$ (cf. Lemma 5.2). Furthermore, conditioned on $(\mathbb{x}, \mathbb{w}) \in \mathsf{R}_{\mathsf{z}}$, π_i verifies correctly except with negligible probability due to correctness of Π_{z}. In total, two applications of a union bound yields that

$$\left|\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_2}(\lambda) - \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_3}(\lambda)\right| \leqslant T \cdot Q_S \cdot \left(2^{-\frac{n\ell}{10}} + 2^{-\frac{nk}{10}}\right) + \mathsf{negl}(\lambda).$$

Hybrid_4. Lastly, in this hybrid, the challenger sets $\mathsf{f}_{\mathsf{fail}} = \top$ if in $\mathcal{O}_{\mathsf{IA}_{4,3}}$, we have that $D_{j,k}^{(j)} \neq D_{j,k}^{(k)}$ or $D_{k,j}^{(j)} \neq D_{k,j}^{(k)}$ with $j, k \in \mathsf{sHS}$. This is formalized in Fig. 12.

To prove that the probability of $\mathsf{f}_{\mathsf{fail}}$ being set to $\top$ only changes negligibly, we rely on the fact that these Ajtai commitments D are generated deterministically. Recall that the honest signers j and k generate the commitments $D_{j,k}^{(k)}$ and $D_{j,k}^{(j)}$ via $\mathsf{DetMaskCom}$ on input $(\mathsf{seed}_{j,k}, \mathsf{ctnt}_{\mathsf{z}})$. Here, $\mathsf{DetMaskCom}$ generates these commitments deterministically using the randomness derived from the seed $\mathsf{seed}_{j,k}$. In particular, if the view $\mathsf{ctnt}_{\mathsf{z}}$ is identical for both honest signers, then the generated Ajtai commitments are identical. Since the view is defined as $\mathsf{ctnt}_{\mathsf{z}} = \mathsf{SS}||\mathsf{M}||(\mathsf{cmt}_j, \mathbf{w}_j)_{j \in \mathsf{SS}}$, the identifiable abort game (or more specifically, the synchronous authenticated broadcast channel) guarantees that the views of any honest signers are consistent. We thus have

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_3}(\lambda) = \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_4}(\lambda).$$

Lastly, we establish below that no adversary can win in Hybrid_4. Combined this fact with all the bounds, we arrive at the statement in Theorem 5.2. It remains to prove the following lemma.

Lemma 5.4. *There are adversaries $\mathcal{B}_{\mathsf{msk}}$ and $\mathcal{B}_{\mathsf{share}}$ on $\mathsf{C}_{\mathsf{msk}}$ and $\mathsf{C}_{\mathsf{share}}$, respectively, with runtime $\mathsf{poly}(\lambda, Q_{\mathsf{H}_{\mathsf{z}}}) \cdot T_{\mathcal{A}}$ such that*

$$\mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_4}(\lambda) \leqslant \mathsf{Adv}_{\mathcal{B}_{\mathsf{msk}}, \mathsf{C}_{\mathsf{msk}}}^{\mathsf{bind}}(\lambda) + \mathsf{Adv}_{\mathcal{B}_{\mathsf{share}}, \mathsf{C}_{\mathsf{share}}}^{\mathsf{bind}}(\lambda) + \kappa(\lambda, Q_{\mathsf{H}_{\mathsf{z}}}),$$

where κ is the knowledge error of Π_{z} and $\mathsf{poly}(\lambda, Q_{\mathsf{H}_{\mathsf{z}}})$ is the polynomial that describes the runtime overhead of Ext.

Proof. On a high level, we apply knowledge soundness of Π_{z} to extract witnesses for relation R_{z} from the proofs of each malicious signer $j \in \mathsf{sCS}^*$ from session sid^*. Note that the challenger in Hybrid_4 already knows statement-witness pairs for honest signers $j \in \mathsf{sHS}^*$. Given these witnesses, we show that if aggregation fails, then we can construct an adversary breaking the binding properties of $\mathsf{C}_{\mathsf{msk}}$ or $\mathsf{C}_{\mathsf{share}}$. Let us first define a wrapper algorithm that outputs the proof-statement pairs of malicious signers in session sid^*.

Description of Wrapper Algorithm $\mathcal{B}$. We now present a wrapper algorithm $\mathcal{B}$ that simulates the interaction between the challenger and $\mathcal{A}$ in Hybrid_4. We denote by $\mathcal{H}$ the list of H_{z} outputs and $\mathsf{coin}_{\mathcal{A}}$ the random coins of $\mathcal{A}$. We further denote $\mathsf{coin}_{\mathcal{C}}$ the random coins of the challenger, excluding the random coins to sample H_{z} output; to generate $\mathsf{crs}_{\mathsf{z}}$; and to generate the public parameters $\mathsf{pp}_{\mathsf{msk}}$ and $\mathsf{pp}_{\mathsf{share}}$ in KeyGen. Lastly, we denote $\mathsf{coin} = (\mathsf{coin}_{\mathcal{C}}, \mathsf{coin}_{\mathcal{A}})$, where note that given $\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}, \mathcal{H}, \mathsf{coin}_{\mathcal{C}}$ and $\mathsf{coin}_{\mathcal{A}}$, the interaction between the challenger and $\mathcal{A}$ in Hybrid_4 is deterministic. We then define $\mathcal{B}$ as an algorithm with oracle access to H as follows:

$\mathcal{B}^{\mathsf{H}}(\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}; \mathsf{coin})$: On input $\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}$ and $\mathsf{pp}_{\mathsf{share}}$, $\mathcal{B}$ simulates the interaction between the challenger and $\mathcal{A}$ in Hybrid_4. That is, $\mathcal{B}$ invokes $\mathcal{A}$ on the randomness $\mathsf{coin}_{\mathcal{A}}$ included in coin and simulates Hybrid_4, where it runs the same code as Hybrid_4 with random coins $\mathsf{coin}_{\mathcal{C}}$ except for the following differences:

- It uses the provided $\mathsf{crs}_{\mathsf{z}}$, $\mathsf{pp}_{\mathsf{msk}}$ and $\mathsf{pp}_{\mathsf{share}}$ rather than generating them on its own in KeyGen;
- All random oracle queries to H_{z} are answered via the provided oracle H.

At the end of the game when $\mathcal{A}$ outputs a session identifier sid^*, $\mathcal{B}$ checks if $\mathcal{A}$ was successful in Hybrid_4. If the check does not pass, then $\mathcal{B}$ outputs $\perp$. Otherwise, we have $\mathsf{rnd}_{\perp}^* = 4$

and for all honest signers $i \in \mathsf{sHS}^*$, we have $\mathsf{CS}_i^{\mathsf{IA}*} = \varnothing$. In this case, $\mathcal{B}$ retrieves the proofs $(\pi_j)_{j\in\mathsf{sCS}*}$ from the protocol messages $\mathsf{pmia}_{4,j}^{(1)}$ of the corrupt signers $j \in \mathsf{sCS}^*$ in session sid^*, with corresponding statements $\mathbb{x}_j = (L_{\mathsf{SS},j}, \mathbf{z}_j, \mathbf{w}_j, c, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}}, C_j)$. Similarly, $\mathcal{B}$ retrieves the proofs $(\pi_j)_{j\in\mathsf{sHS}}$ from the protocol messages $\mathsf{pmia}_{4,j}^{(1)}$ of honest signers $j \in \mathsf{sHS}^*$ in session sid^*, with corresponding statements $\mathbb{x}_j = (L_{\mathsf{SS},j}, \mathbf{z}_j, \mathbf{w}_j, c, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}}, C_j)$ and witness $\mathbb{w}_j = ([\![\mathbf{s}]\!]_j, \gamma_j, \mathbf{r}_j, \mathbf{e}'_j, (\mathbf{m}_{j,k}^{(j)}, \mathbf{m}_{k,j}^{(j)}, \delta_{j,k}^{(j)}, \delta_{k,j}^{(j)})_{k\in\mathsf{SS}})$. Sets $\mathsf{aux} := ((\mathbb{x}_j, \mathbb{w}_j)_{j\in\mathsf{sHS}*}, ([\![\mathbf{s}]\!]_j, \gamma_j)_{j\in\mathsf{SS}*})$ and outputs

$$R := ((\mathbb{x}_j, \pi_j)_{j\in\mathsf{sCS}*}, \mathsf{aux}).$$

It can be checked that $\mathcal{B}$ perfectly simulates the view of the challenger in Hybrid_4 if $\mathsf{crs}_\mathsf{z}, \mathsf{pp}_\mathsf{msk}$ and $\mathsf{pp}_\mathsf{share}$ are distributed uniform. Therefore, $\mathcal{B}$ outputs $R \neq \bot$ with probability $\mu(\lambda) := \mathsf{Adv}_{\mathcal{A}}^{\mathsf{Hybrid}_4}(1^\lambda)$. Also, note that since $\mathsf{CS}_i^{\mathsf{IA}*} = \varnothing$, the statement-proof pairs $(\mathbb{x}_j, \pi_j)$ verify correctly.

Let Ext be the knowledge extractor of NIZK Π_z (cf. Definition 2.9). As explained above, we have for uniform pp_msk and $\mathsf{pp}_\mathsf{share}$ and $R \leftarrow \mathcal{B}^{\mathsf{H}}(\mathsf{crs}_\mathsf{z}, \mathsf{pp}_\mathsf{msk}, \mathsf{pp}_\mathsf{share}; \mathsf{coin})$, where coin is sampled at random (as specified above), that $R \neq \bot$ with probability $\mu(\lambda)$. Denote by $\mathcal{H}$ the random choices of H during the execution of $\mathcal{B}$. In case $R \neq \bot$, then $R = ((\mathbb{x}_j, \pi_j)_{j\in\mathsf{sCS}*}, \mathsf{aux})$, where each statement-proof pair $(\mathbb{x}_j, \pi_j)$ verifies correctly. By $|\mathsf{sCS}^*|$-adaptive knowledge soundness, we have that

$$\Pr[(\mathbb{w}_j)_{j\in\mathsf{sCS}*} \leftarrow \mathsf{Ext}^{\mathcal{B}(\mathsf{crs}_\mathsf{z}, \mathsf{pp}_\mathsf{msk}, \mathsf{pp}_\mathsf{share}; \cdot)}(\mathsf{crs}_\mathsf{z}, \rho, \mathcal{H}) : \forall j \in \mathsf{sCS}^*, (\mathbb{x}_j, \mathbb{w}_j) \in \mathsf{R}_\mathsf{z}] \geqslant \mu(\lambda, Q_{\mathsf{H}_\mathsf{z}}) - \kappa(\lambda, Q_{\mathsf{H}_\mathsf{z}}), \quad (3)$$

where $\kappa(\lambda, Q_{\mathsf{H}_\mathsf{z}})$ is the knowledge error of Π_z, and $(\mathbb{x}_j, \pi_j)$ is defined as above. Note that the runtime of Ext is $\mathsf{poly}(\lambda, Q_{\mathsf{H}_\mathsf{z}}) \cdot T_\mathcal{B}$. (Note that $T_\mathcal{B} \approx T_\mathcal{A}$.)

Parse Ext's output as $\mathbb{w}_j = ([\![\mathbf{s}]\!]'_j, \gamma'_j, \mathbf{r}_j, \mathbf{e}'_j, (\mathbf{m}_{j,k}^{(j)}, \mathbf{m}_{k,j}^{(j)}, \delta_{j,k}^{(j)}, \delta_{k,j}^{(j)})_{k\in\mathsf{SS}*})$ and $\mathbb{x}_j = (L_{\mathsf{SS}*,j}, \mathbf{z}_j, \mathbf{w}_j, c, (D_{j,k}^{(j)}, D_{k,j}^{(j)})_{k\in\mathsf{SS}}, C_j)$ as above for $j \in \mathsf{SS}^*$, where $\mathbb{w}_j$ and $\mathbb{x}_j$ are taken from aux for $j \in \mathsf{sHS}^*$, and for $j \in \mathsf{sCS}^*$ taken from the $\mathcal{B}$ and Ext's output as specified above. Let us define some events (in which public parameters $\mathsf{crs}_\mathsf{z}, \mathsf{pp}_\mathsf{msk}$ and $\mathsf{pp}_\mathsf{share}$ are chosen uniformly and independently at random).

- The event E_Ext occurs iff extraction succeeds, *i.e.*, for all $j \in \mathsf{sCS}^*$ it holds that $(\mathbb{x}_j, \mathbb{w}_j) \in \mathsf{R}_\mathsf{z}$, where $(\mathbb{w}_j)_{j\in\mathsf{sCS}*} \leftarrow \mathsf{Ext}^{\mathcal{B}(\mathsf{crs}_\mathsf{z}, \mathsf{pp}_\mathsf{msk}, \mathsf{pp}_\mathsf{share}; \cdot)}(\mathsf{crs}_\mathsf{z}, \rho, \mathcal{H})$.
- The event E_msk occurs iff E_Ext occurs, and there are $(j,k) \in (\mathsf{SS}^*)^2$ with $j \neq k$ and $\mathbf{m}_{j,k}^{(j)} \neq \mathbf{m}_{j,k}^{(k)}$.
- The event E_share occurs iff E_Ext occurs and there is a signer $j \in \mathsf{sCS}^*$ such that $[\![\mathbf{s}]\!]'_j \neq [\![\mathbf{s}]\!]_j$.

Then, by Eq. (3) and Lemmata 5.5 to 5.7, there are PPT adversaries $\mathcal{B}_\mathsf{msk}$ and $\mathcal{B}_\mathsf{share}$ such that

$$\begin{aligned}
\mu(\lambda, Q_{\mathsf{H}_\mathsf{z}}) - \kappa((\lambda, Q_{\mathsf{H}_\mathsf{z}})) &\leqslant \Pr[E_\mathsf{Ext}] && \text{(Eq. (3))}\\
&= \Pr[E_\mathsf{msk} \ \vee \ E_\mathsf{share}] && \text{(Lemma 5.5)}\\
&\leqslant \Pr[E_\mathsf{msk}] + \Pr[E_\mathsf{share}] && \text{(Union bound)}\\
&\leqslant \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_\mathsf{msk}, \mathsf{C}_\mathsf{msk}}(\lambda) + \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_\mathsf{share}, \mathsf{C}_\mathsf{share}}(\lambda) && \text{(Lemmata 5.6 and 5.7).}
\end{aligned}$$

This shows the statement. Below, we prove the aforementioned lemmata.

Lemma 5.5. *We have* $E_\mathsf{Ext} = E_\mathsf{msk} \vee E_\mathsf{share}$.

Proof. Assume for the sake of contradiction that E_Ext occurs and neither E_msk nor E_share occurs. Because E_Ext occurs, we have $(\mathbb{x}_j, \mathbb{w}_j) \in \mathsf{R}_\mathsf{z}$ for corrupted signers $j \in \mathsf{sCS}^*$. Let us recall some facts that follow by construction of the wrapper algorithm $\mathcal{B}^\mathsf{H}$. First, we know that $\mathsf{rnd}_\bot^* = 4$, and thus Agg failed in session sid^*. Due to the condition introduced in Hybrid_3, we have $(\mathbb{x}_j, \mathbb{w}_j) \in \mathsf{R}_\mathsf{z}$ for honest signers $j \in \mathsf{sHS}^*$. Because E_msk does not occur, we have $\mathbf{m}_{j,k} := \mathbf{m}_{j,k}^{(j)} = \mathbf{m}_{j,k}^{(k)}$ for $(j,k) \in (\mathsf{SS}^*)^2$. Similarly, because E_share does not occur, we have $\mathbf{s}_j = \mathbf{s}'_j$ for $j \in \mathsf{sCS}^*$. By $(\mathbb{x}_j, \mathbb{w}_j) \in \mathsf{R}_\mathsf{z}$, we have the identities

$$\mathbf{z}_i = c \cdot L_{\mathsf{SS}*,i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \mathbf{\Delta}_i, \quad \mathbf{w}_i = \mathbf{A} \cdot \mathbf{r}_i + \mathbf{e}'_i, \quad (4)$$

where $\|\mathbf{r}_i\|_2 \leqslant B_{\mathbf{r}} \ \wedge \ \|\mathbf{e}'_i\|_2 \leqslant B_{\mathbf{e}}$ and $\boldsymbol{\Delta}_i = \sum_{j \in \mathsf{SS}^* \setminus \{i\}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i}$. Let us show

$$\sum_{i \in \mathsf{SS}^*} \boldsymbol{\Delta}_i = \mathbf{0}. \tag{5}$$

By plugging the above identity of $\boldsymbol{\Delta}_i$ into the equation, we have

$$\begin{aligned}
\sum_{i \in \mathsf{SS}^*} \boldsymbol{\Delta}_i &= \sum_{i \in \mathsf{SS}^*} \sum_{j \in \mathsf{SS}^* \setminus \{i\}} (\mathbf{m}_{i,j} - \mathbf{m}_{j,i}) \\
&= \sum_{i \in \mathsf{SS}^*} \sum_{j \in \mathsf{SS}^*} (\mathbf{m}_{i,j} - \mathbf{m}_{j,i}) \\
&= \sum_{i \in \mathsf{SS}^*} \sum_{j \in \mathsf{SS}^*} \mathbf{m}_{i,j} - \sum_{i \in \mathsf{SS}^*} \sum_{j \in \mathsf{SS}^*} \mathbf{m}_{j,i} \\
&= \mathbf{0}.
\end{aligned}$$

Note that by correctness of Shamir's secret sharing, we have $\sum_{i \in \mathsf{SS}^*} L_{\mathsf{SS}^*,i} \cdot [\![\mathbf{s}]\!]_i = \mathbf{s}$. Thus, we have

$$\begin{aligned}
\sum_{i \in \mathsf{SS}^*} \mathbf{z}_i &= \sum_{i \in \mathsf{SS}^*} c \cdot L_{\mathsf{SS}^*,i} \cdot [\![\mathbf{s}]\!]_i + \mathbf{r}_i + \boldsymbol{\Delta}_i && \text{(Eq. (4))} \\
&= c \cdot \mathbf{s} + \sum_{i \in \mathsf{SS}^*} \mathbf{r}_i + \boldsymbol{\Delta}_i && \text{(Sharmir's SS)} \\
&= c \cdot \mathbf{s} + \sum_{i \in \mathsf{SS}^*} \mathbf{r}_i. && \text{(Eq. (5))}
\end{aligned}$$

Similarly, we have by Eq. (4) that

$$\sum_{i \in \mathsf{SS}^*} \mathbf{w}_i = \sum_{i \in \mathsf{SS}^*} \mathbf{A} \cdot \mathbf{r}_i + \mathbf{e}'_i = \mathbf{A} \cdot \left(\sum_{i \in \mathsf{SS}^*} \mathbf{r}_i \right) + \sum_{i \in \mathsf{SS}^*} \mathbf{e}'_i.$$

Combining all above equation, the condition $c = c'$ in the signature verification obviously holds. Thus, if the condition for norm of a signature, i.e., $\|(\mathbf{z}, 2^{\nu_{\mathbf{w}}} \cdot \mathbf{h})\|_2 \leqslant B$, also hold, the aggregated signature is valid. We can show that this holds by almost following the proof of correctness of the original $\mathsf{TRaccoon}$ [DPKM+24, Lemma 7.1]. Because the difference of norm evaluation from the original $\mathsf{TRaccoon}$ is the term $T(B_{\mathbf{r}} + B_{\mathbf{e}})$ corresponding to the sum of randomness of commitment in B, it is sufficient to confirm that the norm of $(\sum_{i \in \mathsf{SS}} \mathbf{r}_i, \sum_{i \in \mathsf{SS}} \mathbf{e}'_i)$ is bounded by $T(B_{\mathbf{r}} + B_{\mathbf{e}})$ conditioned on E_{Ext} for proving the aggregated signatures pass the verification. Since E_{Ext} occurs, all signers' $(\mathbf{r}_i, \mathbf{e}'_i)$ satisfy $\|\mathbf{r}_i\|_2 \leqslant B_{\mathbf{r}}$ and $\|\mathbf{e}'_i\|_2 \leqslant B_{\mathbf{e}}$ from the relation R_{z}. By Minkowski's inequality, we have $\|(\sum_{i \in \mathsf{SS}} \mathbf{r}_i, \sum_{i \in \mathsf{SS}} \mathbf{e}'_i)\|_2 \leqslant T(B_{\mathbf{r}} + B_{\mathbf{e}})$. Consequently, the aggregated signature verifies and Agg does *not* abort. Thus, $\mathsf{rnd}^*_{\perp} \neq 4$ which contradicts that E_{Ext} occurs. □

Lemma 5.6. *There exists a PPT adversary* $\mathcal{B}_{\mathsf{msk}}$ *such that* $\Pr[E_{\mathsf{msk}}] \leqslant \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_{\mathsf{msk}}, \mathsf{C}_{\mathsf{msk}}}(\lambda)$.

Proof. Let us construct an adversary $\mathcal{B}_{\mathsf{msk}}$ on $\mathsf{C}_{\mathsf{msk}}$'s binding property. First, $\mathcal{B}_{\mathsf{msk}}$ obtains public parameters $\mathsf{pp}_{\mathsf{msk}}$ from the binding challenger and samples $\mathsf{pp}_{\mathsf{share}}$ and $\mathsf{crs}_{\mathsf{z}}$ at random. Then, $\mathcal{B}_{\mathsf{msk}}$ runs $R \leftarrow \mathcal{B}^{\mathsf{H}}(\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}; \mathsf{coin})$, where coin is sampled at random and checks that $R \neq \perp$. If so, $\mathcal{B}_{\mathsf{msk}}$ parses $R = ((\mathbb{x}_j, \pi_j)_{j \in \mathsf{sCS}^*}, \mathsf{aux})$, where each statement-proof pair $(\mathbb{x}_j, \pi_j)$ verifies correctly. Then, $\mathcal{B}_{\mathsf{msk}}$ invokes $(\mathbb{w}_j)_{j \in \mathsf{sCS}^*} \leftarrow \mathsf{Ext}^{\mathcal{B}(\mathsf{crs}_{\mathsf{z}}, \mathsf{pp}_{\mathsf{msk}}, \mathsf{pp}_{\mathsf{share}}; \cdot)}(\mathsf{crs}_{\mathsf{z}}, \rho, \mathcal{H})$ and checks if E_{msk} occurs. (Here, $\mathcal{H}$ denotes the random oracle outputs when $\mathcal{B}^{\mathsf{H}}$ is invoked as above.) If E_{msk} occurs, then there are indices $(j, k) \in \mathsf{SS}^*$ with $j \neq k$ such that $\mathbf{m}^{(j)}_{j,k} \neq \mathbf{m}^{(k)}_{j,k}$. Then, $\mathcal{B}_{\mathsf{msk}}$ retrieves openings $\delta^{(j)}_{j,k}$ and $\delta^{(k)}_{j,k}$ from $\mathbb{w}_j$ and $\mathbb{w}_k$. Finally, $\mathcal{B}_{\mathsf{msk}}$ outputs $(D^{(j)}_{j,k}, \mathbf{m}^{(j)}_{j,k}, \mathbf{m}^{(k)}_{j,k}, \delta^{(j)}_{i,j}, \delta^{(k)}_{i,j})$ to its challenger [10].

[10] Technically, $\delta^{(j)}_{i,j}$ and $\delta^{(k)}_{i,j}$ is the randomness used to construct $D^{(j)}_{j,k}$ and $D^{(k)}_{k,j}$, respectively. By correctness, we can derive appropriate opening information from the randomness.

Let us analyze the success probability of $\mathcal{B}_{\mathsf{msk}}$. By construction of the wrapper algorithm $\mathcal{B}^{\mathsf{H}}$, we know that the checks in $\mathcal{O}_{\mathsf{IA}_{4,3}}$ passed due to $\mathsf{CS}_i^{\mathsf{IA}*} = \varnothing$ for all honest signers $i \in \mathsf{sHS}^*$. Consequently, we have that $D_{j,k}^{(j)} = D_{j,k}^{(k)}$ for all signers $(j,k) \in (\mathsf{SS}^*)^2$. Also, we have $\mathbf{m}_{j,k}^{(j)} \neq \mathbf{m}_{j,k}^{(k)}$ and that $\delta_{i,j}^{(j)}$ and $\delta_{i,j}^{(k)}$ are valid if E_{msk} occurs. Thus, $\mathcal{B}_{\mathsf{msk}}$ succeeds with probability $\Pr[E_{\mathsf{msk}}]$ and $\mathsf{Time}(\mathcal{B}_{\mathsf{msk}}) \approx \mathsf{Time}(\mathsf{Ext})$. □

Lemma 5.7. *There exists a PPT adversary* $\mathcal{B}_{\mathsf{share}}$ *such that* $\Pr[E_{\mathsf{share}}] \leqslant \mathsf{Adv}^{\mathsf{bind}}_{\mathcal{B}_{\mathsf{share}},\mathsf{C}_{\mathsf{share}}}(\lambda)$.

Proof. This follows similarly to Lemma 5.6. The adversary $\mathcal{B}_{\mathsf{share}}$ on $\mathsf{C}_{\mathsf{share}}$'s binding property extracts $R = ((\mathbb{x}_j, \pi_j)_{j \in \mathsf{sCS}*}, \mathsf{aux})$ similar to $\mathcal{B}_{\mathsf{msk}}$ except that it uses $\mathsf{pp}_{\mathsf{share}}$ provided by its challenger and samples $\mathsf{pp}_{\mathsf{msk}}$ at random. it then checks that E_{share} occurs, and if so, there exists some $j \in \mathsf{sCS}^*$ such that $[\![\mathbf{s}]\!]'_j \neq [\![\mathbf{s}]\!]_j$. $\mathcal{B}_{\mathsf{share}}$ retrieves opening γ' from $\mathbb{w}_j$ and outputs $(C_j, [\![\mathbf{s}]\!]_j, [\![\mathbf{s}]\!]'_j, \gamma, \gamma')$ to its challenger, where $([\![\mathbf{s}]\!]_j, \gamma)$ is the opening of C_j retrieved from $\mathsf{aux} = ((\mathbb{x}_j, \mathbb{w}_j)_{j \in \mathsf{sHS}*}, (\mathbf{s}_j, \gamma_j)_{j \in \mathsf{SS}*})$.

Note that by construction, $([\![\mathbf{s}]\!]_j, \gamma_j)$ is an opening for C_j. As E_{share} occurs, also $([\![\mathbf{s}]\!]'_j, \gamma'_j)$ is an opening for C_j with $[\![\mathbf{s}]\!]'_j \neq [\![\mathbf{s}]\!]_j$. Moreover, we have $\mathsf{Time}(\mathcal{B}_{\mathsf{share}}) \approx \mathsf{Time}(\mathsf{Ext})$. The statements follows. □

This concludes the proof of Lemma 5.4. □

□

6 Analysis of LaBRADOR with Zero-Knowledge

We give an overview of our analysis of our proof system for relation R_{z} (cf. Section 4.2). We refer to Section B for the detailed analysis.

Overview of LaBRADOR and LNP. We give a brief overview of LaBRADOR [BS23] and LNP [LNP22]. LaBRADOR is a 9-move interactive proof system for lattice-related relations, including quadratic equations and norm bounds. It has a slight soundness slack for norm bounds, *i.e.*, if the input witness is bounded by B, the extracted witness is guaranteed to be smaller than B_s, where $B < B_s$. By recursion of a base protocol, the proof size can be compressed through *folding*, resulting in a *succinct* proof system. The proof system is *not* zero-knowledge, *i.e.*, a proof leaks information about the witness.

On the other hand, LNP is a 7-move interactive proof system for lattice-related relations. LNP is an *exact* proof, *i.e.*, there is no slack in the norm bound. While LNP is not succinct, it is zero-knowledge by design. An undesired characteristic of LNP is that the last flow (sent from the prover to the verifier) scales with the size of the *witness*, while prior flows scale with the size of the *proven statement*, both of which are very large for our relation R_{z}.

Both proof systems can be compiled into a non-interactive proof via Fiat-Shamir. It was shown in [LNP22] that LNP is secure, but the proof only covers their specific protocol. Also, [AAB+24] proves that LaBRADOR compiled via Fiat-Shamir is secure within a framework coined *predicate special soundness* (PSS), a natural extension of special soundness that is well-suited to lattice-specific problems within extraction. Importantly, as shown in [AAB+24], PSS is composition-friendly, *i.e.*, when composing two PSS interactive protocols, their composition is PSS, and therefore can be compiled via Fiat-Shamir in a sound manner.

Limitation of Prior Work. Before we detail our approach of combining LaBRADOR and LNP, we note that the approach of using LNP [LNP22] in combination with LaBRADOR [BS23] has been mentioned in the literature. In [BS23, Section 6], the authors suggest using LNP with LaBRADOR to obtain zero-knowledge. For this, the authors give two approaches:

1. Prove the desired statement via LNP and then prove knowledge of the final prover message via LaBRADOR. Note that intermediate prover messages cannot be omitted from the proof (or compressed via LaBRADOR), as these are required to derive challenges from the random oracle after applying the Fiat-Shamir transformation.

2. Blind the prover messages within LaBRADOR (via standard lattice-based Σ-protocol techniques) and prove knowledge of the witness via LNP after the final LaBRADOR prover message.

Beullens and Seiler provide a sketch of the second approach in [BS23]. We also note that the first approach is employed in [ADDG24], as in their protocol, the proof cost is dominated by the final prover message of LNP.

Unfortunately, prior works [BS23, ADDG24] only provide a sketch of these approaches, and no formal security proof is provided. Considering that even proving security of standard LaBRADOR was non-trivial [AAB+24], the security of these approaches must be substantiated with a formal proof. Secondly, the statement proven in our protocol is rather large (Section 4.2). As mentioned above, the size of the intermediate prover messages of LNP scale with the statement size; therefore compressing the final flow via LaBRADOR is only a marginal improvement over simply using LNP alone. Consequently, the first approach is not an option without further modification for our instantiation.

On the other hand, the second approach requires analyzing soundness of the modified LaBRADOR protocol and does not guarantee exact proofs since LaBRADOR includes a slack in the extraction's norm. Proving soundness of this approach is less modular, as we cannot employ the result that LaBRADOR is PSS from [AAB+24].

Our Approach. As in prior works [BS23, ADDG24], we also employ the first approach: we employ LNP to prove the desired relation R_z (cf. Section 4.2) and then prove knowledge of the final prover-message of LNP via LaBRADOR. The advantage of this approach is that as PSS protocols compose well, we can rely on the fact that LaBRADOR is PSS from [AAB+24] in a black-box manner, leading to a modular security proof.

As mentioned, this leads to a rather large proof size. To compress the LNP proof further, we employ the same optimization as LaBRADOR. Instead of sending the prover-messages in plain, the prover sends *compressing* Ajtai commitments of the messages to the verifier. The commitments are then opened in the final prover message. While this increases total communication, now the bulk of the communication is in the final prover message, which will be compressed using LaBRADOR, *even* for large statements. Note that the relation R_z is not natively supported by LNP as it uses multiple commitments with the same randomness to enable mask comparison (where LNP commits to all values within a single commitment), we further extend the relations proven by LNP to cover R_z. The final non-interactive proof is then obtained by compiling the composed protocol using Fiat-Shamir. This leads to a succinct proof system with zero-knowledge, which we coin LNPLab.

Security Analysis. As LNP is zero-knowledge, it is straightforward to verify that the composed proof is zero-knowledge.[11] The core technical challenge is analyzing soundness of our proof system. For this, we employ the PSS framework from [AAB+24]. In particular, we revisit the security proof of LNP in [LNP22] and formally show that our LNP variant is PSS. As mentioned, PSS protocols compose well, so soundness of this approach follows from our LNP analysis and the fact that LaBRADOR is PSS [AAB+24]. Note that through a careful analysis of the extraction of the composition of LNP and LaBRADOR, we can prove that combining an exact proof (LNP) with a non-exact proof (LaBRADOR) we can obtain an exact proof, meaning that LNPLab obtains a tighter extraction than LaBRADOR. On a high level, given a tree of transcripts[12], we need to analyze under which conditions we can compute either a witness for the desired relation or break a hard problem embedded in the common reference string. These conditions are formalized as *predicates* per layer of the tree. By analyzing the failure density of the predicates, *i.e.*, the probability that a predicate fails to be satisfied, we can derive a lower bound on the extraction probability. As [AAB+24] proposes a rewinding-based extractor $\mathcal{E}_{\mathsf{FS}}$ for PSS protocols compiled via Fiat-Shamir, this yields soundness of LNPLab compiled via Fiat-Shamir transformation.

Multi-proof Extraction Let us denote by Π the non-interactive proof system obtained by compiling LNPLab via the Fiat-Shamir transformation. As mentioned, [AAB+24] proposes an extractor $\mathcal{E}_{\mathsf{FS}}$ for Π. Their extractor is based on rewinding, and unfortunately only works for a single proof: given oracle-access to a prover $\mathcal{P}$ that outputs a proof π, $\mathcal{E}_{\mathsf{FS}}$ can extract a witness $\mathbb{w}$ for the relation R_z. For our security

[11] Formally, we simply employ the LNP simulator to simulate the LNP transcript, and then run the honest LaBRADOR prover to compute the full transcripts.

[12] A tree of transcripts is a tree, where each node represents a protocol message. Each path represents a transcript and each branch diverges if the protocol message differs.

proof of identifiable abort in Section 5.4, we crucially need an extractor that given many proofs extracts a witness for each proof. In general, rewinding-based extractors for many proofs run in exponential time in the number of proofs [SG98, BFW15]. A crucial observation is that in our case, all proofs $(\pi_i)_{i\in[N]}$ are output at once. This property allows us to give a natural extractor that, assuming a single proof extractor has knowledge error κ, then the knowledge error of our multi-proof extractor is only $N \cdot \kappa$. The analysis follows from a careful application of a union bound. We refer to Section B.3 for details.

7 Concrete Parameters

In this section, we provide concrete parameters for our proposed scheme TRaccoon-IA.We consider two approaches, (i) choose them to be compatible with the verification procedure of TRaccoon by reducing the number of supported signatures Q_S and vary private parameters $\sigma_{\mathbf{t}}$, $\sigma_{\mathbf{w}}$, or (ii) select other parameters to support $Q_S = 2^{60}$ like TRaccoon. In particular, for the first case, we enforce the bound B used for verification and maintain the same signature size as TRaccoon. For our parameter selection, we assume at most $N = 1024$ parties.

We select parameters based on the security reductions from Section 5. We provide here constraints on parameters induced by *unforgeability* and *identifiable abort*, and explain our parameter selection strategy.

NIZK Components. The NIZK used in TRaccoon-IA can be instantiated independently from the other parameters and do not influence signature size. The bounds on concrete parameters are given in Lemma B.10, computing an optimized and exact instantiation is outside of the scope of this work but a high level analysis (cf. Section B.6.3) show that a conservative estimate gives proof size of less than 60 kB and commitment size of around 3 kB (with two mask commitment per user for each other user).

Identifiable Abort. IA is ensured when using the bounds $B_{\mathbf{r}}, B_{\mathbf{e}}, B$ from the asymptotic parameters (Section 5.1).

For the scheme compatible with TRaccoon, we additionally need to ensure that B remains below the verification bound of TRaccoon. This in turn provides an upper bound on the standard deviation $\sigma_{\mathbf{w}}$. Taking into accounts the new bounds (cf. Section 5.1), $\sigma_{\mathbf{w}}$ needs to be rougly $\sqrt{T}$ times smaller than in TRaccoon for TRaccoon-IA.

Unforgeability. Following the security reduction of Theorem 5.1 and Lemma 5.1, the unforgeability of our scheme reduces to the Hint-MLWE and SelfTargetMSIS problems.

TRaccoon-IA is unforgeable under $\mathsf{Hint\text{-}MLWE}_{q,\ell,k}$ with $T \cdot Q_S$ hints of the form $c \cdot \mathbf{s} + \mathbf{r}$. The need for $T \cdot Q_S$ hints as opposed to Q_S is an artifact of the proof as mentioned in Remark 3.1, and we evaluate the security for Q_S hints. In our case, all distributions are discrete Gaussians so that Hint-MLWE reduces from $\mathsf{MLWE}_{q,\ell,k,\sigma_{\mathsf{red}}}$ by Lemma 2.6with

$$\frac{1}{\sigma_{\mathsf{red}}^2} = \frac{1}{2} \cdot \left(\frac{1}{\sigma_{\mathbf{t}}^2} + \frac{B_{\mathsf{sign}}}{\sigma_{\mathbf{w}}^2} \right),$$

where $B_{\mathsf{sign}} = Q_S \cdot W \cdot (1 + o(1))$. See [DPKM+24, Lemma B.2] and the bound on $s_1(c) \leqslant W$. We then use the lattice-estimator (https://github.com/malb/lattice-estimator) for concrete security.

We also require the $\mathsf{SelfTargetMSIS}_{q,\ell+1,k,\mathsf{H}_c,\mathcal{C},B_{\mathsf{stmsis}}}$ problem to be hard. Following the footsteps of Dilithium [LDK+22, §C.3] and Raccoon [dEK+23, §4.3.5], we evaluate the hardness of SelfTargetMSIS based on the best known attacks. According to [LDK+22, dEK+23], the best known attacks against SelfTargetMSIS are either to (i) break the second preimage resistance of the underlying hash function, or (ii) to find a short vector $\mathbf{z}$ such that:

$$\mathbf{A} \cdot \bar{\mathbf{z}} = \mathbf{w} + \mathbf{b} \cdot c + \alpha, \tag{6}$$

where the term $\alpha = \mathbf{A} \cdot \mathbf{z} - 2^{\nu_{\mathbf{w}}} \lfloor \mathbf{A} \cdot \mathbf{z} \rceil_{\nu_{\mathbf{w}}}$ satisfies $\|\alpha\| \leqslant 2^{\nu_{\mathbf{w}}} \sqrt{k \cdot n}$. Solving (i) is hard as long as $|\mathcal{C}| = \binom{n}{W} 2^W \geqslant 2^\lambda$. On the other hand, solving (ii) requires solving Eq. (6), which is an instance of the (inhomogeneous) MSIS problem. We again evaluate the hardness of this MSIS problem using the lattice-estimator.

We explored the space of parameters to provide two parameter sets in Table 3 aiming for NIST level I. The first is compatible with TRaccoon at the cost of a lower Q_S.

(k,ℓ)	$\sigma_{\mathbf{t}}$	$\sigma_{\mathbf{w}}$	$\nu_{\mathbf{t}}$	$\nu_{\mathbf{w}}$	B	Q_{S}	Unforg. C/Q	$\lvert\mathsf{vk}\rvert$	$\lvert\mathsf{Sig}\rvert$	comm. per party sign	ia	Compatible TRaccoon
$(5,4)$	2^{20}	2^{32}	37	40	$2^{49.2}$	2^{50}	135/119	3.9kB	12.4kB	28.2	$60+6.4T$	Yes
$(5,5)$	2^{8}	2^{31}	37	43	$2^{50.1}$	2^{60}	134/118	3.9kB	14.9kB	31.4	$60+6.4T$	No

Table 3: Selected parameters of TRaccoon-IA aiming for the same bit-security as TRaccoon-128 and $\lambda = 128$. We take $n = 512$, $W = 19$, $q = 549824583172097 = 16515073 \cdot 33292289 \approx 2^{49}$. We indicate the core-SVP hardness of unforgeability, a metric which ignores several polynomial factors. We also give communication cost per participant for the signature generation protocol, and estimates for the abort identification. Sizes are in kilobytes (kB).

8 Acknowledgments

We would like to thank Michael Klooß for insightful discussions on multi-proof extraction and his valuable help in the proof of Lemma B.4. This paper was partially based on results obtained from a project, JPNP24003, commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

References

[AAB+24] Marius A. Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, and Akira Takahashi. Aggregating falcon signatures with labrador. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part I*, volume 14920 of *Lecture Notes in Computer Science*, pages 71–106. Springer, 2024.

[ADDG24] Martin R. Albrecht, Alex Davidson, Amit Deo, and Daniel Gardham. Crypto dark matter on the torus - oblivious PRFs from shallow PRFs and TFHE. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VI*, volume 14656 of *LNCS*, pages 447–476. Springer, Cham, May 2024.

[AF04] Masayuki Abe and Serge Fehr. Adaptively secure feldman VSS and applications to universally-composable threshold cryptography. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 317–334. Springer, Berlin, Heidelberg, August 2004.

[AFK22] Thomas Attema, Serge Fehr, and Michael Klooß. Fiat-shamir transformation of multi-round interactive proofs. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part I*, volume 13747 of *LNCS*, pages 113–142. Springer, Cham, November 2022.

[Ajt96] Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pages 99–108. ACM Press, May 1996.

[ASY22] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022.

[BCK+22] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 517–550. Springer, Cham, August 2022.

[BD22] LTAN Brandão and Michael Davidson. Notes on threshold eddsa/schnorr signatures. National Institute of Standards and Technology, 2022. https://doi.org/10.6028/NIST.IR.8214B.ipd.

[BDL+18] Carsten Baum, Ivan Damgård, Vadim Lyubashevsky, Sabine Oechsner, and Chris Peikert. More efficient commitments from structured lattice assumptions. In Dario Catalano and Roberto De Prisco, editors, *SCN 18*, volume 11035 of *LNCS*, pages 368–385. Springer, Cham, September 2018.

[BDLR25] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from ddh with full adaptive security. *To Appear in EUROCRYPT 2024. Available at https://eprint.iacr.org/2023/1628*, 2025.

[BFW15] David Bernhard, Marc Fischlin, and Bogdan Warinschi. Adaptive proofs of knowledge in the random oracle model. In Jonathan Katz, editor, *PKC 2015*, volume 9020 of *LNCS*, pages 629–649. Springer, Berlin, Heidelberg, March / April 2015.

[BGG+18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018.

[BHK+24] Fabrice Benhamouda, Shai Halevi, Hugo Krawczyk, Yiping Ma, and Tal Rabin. SPRINT: High-throughput robust distributed Schnorr signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part V*, volume 14655 of *LNCS*, pages 62–91. Springer, Cham, May 2024.

[BKL+25] Cecilia Boschini, Darya Kaviani, Russell Lai, Giulio Malavolta, Akira Takahashi, and Mehdi Tibouchi. Ringtail: Practical Two-Round Threshold Signatures from Learning with Errors . In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 70–70, Los Alamitos, CA, USA, May 2025. IEEE Computer Society.

[BKP13] Rikke Bendlin, Sara Krehbiel, and Chris Peikert. How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE. In Michael J. Jacobson Jr., Michael E. Locasto, Payman Mohassel, and Reihaneh Safavi-Naini, editors, *ACNS 13International Conference on Applied Cryptography and Network Security*, volume 7954 of *LNCS*, pages 218–236. Springer, Berlin, Heidelberg, June 2013.

[BL22] Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207. ACM Press, November 2022.

[BLSW25] Renas Bacho, Julian Loss, Gilad Stern, and Benedikt Wagner. Harts: High-threshold, adaptively secure, and robust threshold schnorr signatures. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 104–140, Singapore, 2025. Springer Nature Singapore.

[BLT+24] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 429–459. Springer, Cham, May 2024.

[BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 390–399. ACM Press, October / November 2006.

[Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.

[BS23] Ward Beullens and Gregor Seiler. LaBRADOR: Compact proofs for R1CS from module-SIS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 518–548. Springer, Cham, August 2023.

[BTZ22] Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Report 2022/833, 2022.

[CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. Cryptology ePrint Archive, Paper 2024/467, 2024. https://eprint.iacr.org/2024/467.

[CCL+23] Guilhem Castagnos, Dario Catalano, Fabien Laguillaumie, Federico Savasta, and Ida Tucker. Bandwidth-efficient threshold EC-DSA revisited: Online/offline extensions, identifiable aborts proactive and adaptive security. *Theor. Comput. Sci.*, 939:78–104, 2023.

[CGG+20] Ran Canetti, Rosario Gennaro, Steven Goldfeder, Nikolaos Makriyannis, and Udi Peled. UC non-interactive, proactive, threshold ECDSA with identifiable aborts. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1769–1787. ACM Press, November 2020.

[CKM23] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023.

[DD05] Ivan Damgård and Kasper Dupont. Efficient threshold RSA signatures with general moduli and no extra assumptions. In Serge Vaudenay, editor, *PKC 2005*, volume 3386 of *LNCS*, pages 346–361. Springer, Berlin, Heidelberg, January 2005.

[dEK+23] Rafael del Pino, Thomas Espitau, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, Mélissa Rossi, and Markku-Juhani Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023. available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[dKPR24] Rafaël del Pino, Shuichi Katsumata, Thomas Prest, and Mélissa Rossi. Raccoon: A masking-friendly signature proven in the probing model. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 409–444. Springer, Cham, August 2024.

[DOTT21] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 99–130. Springer, Cham, May 2021.

[DOTT22] Ivan Damgård, Claudio Orlandi, Akira Takahashi, and Mehdi Tibouchi. Two-round n-out-of-n and multi-signatures and trapdoor commitment from lattices. *Journal of Cryptology*, 35(2):14, April 2022.

[dPEK+23] Rafaël del Pino, Thomas Espitau, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, Mélissa Rossi, and Markku-Juhani Saarinen. Raccoon. Technical report, National Institute of Standards and Technology, 2023. Available at https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures.

[dPENP25] Rafael del Pino, Thomas Espitau, Guilhem Niot, and Thomas Prest. Simple and efficient lattice threshold signatures with identifiable aborts. Cryptology ePrint Archive, Paper 2025/871, 2025.

[DPKM+24] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 219–248. Springer, 2024.

[DR24] Sourav Das and Ling Ren. Adaptively secure BLS threshold signatures from DDH and co-CDH. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 251–284. Springer, Cham, August 2024.

[EEN+24] Muhammed F. Esgin, Thomas Espitau, Guilhem Niot, Thomas Prest, Amin Sakzad, and Ron Steinfeld. Plover: Masking-friendly hash-and-sign lattice signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 316–345. Springer, Cham, May 2024.

[EKT24] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 387–424. Springer, Cham, August 2024.

[ENP24] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerse: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 425–458. Springer, Cham, August 2024.

[FGMY97] Yair Frankel, Peter Gemmell, Philip D. MacKenzie, and Moti Yung. Proactive RSA. In Burton S. Kaliski Jr., editor, *CRYPTO'97*, volume 1294 of *LNCS*, pages 440–454. Springer, Berlin, Heidelberg, August 1997.

[FGY96] Yair Frankel, Peter Gemmell, and Moti Yung. Witness-based cryptographic program checking and robust function sharing. In *28th ACM STOC*, pages 499–508. ACM Press, May 1996.

[FMM+24] Offir Friedman, Avichai Marmor, Dolev Mutzari, Omer Sadika, Yehonatan C. Scaly, Yuval Spiizer, and Avishay Yanai. 2PC-MPC: Emulating two party ECDSA in large-scale MPC. Cryptology ePrint Archive, Report 2024/253, 2024.

[GG20] Rosario Gennaro and Steven Goldfeder. One round threshold ECDSA with identifiable abort. Cryptology ePrint Archive, Report 2020/540, 2020.

[GJKR96a] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Robust and efficient sharing of RSA functions. In Neal Koblitz, editor, *CRYPTO'96*, volume 1109 of *LNCS*, pages 157–172. Springer, Berlin, Heidelberg, August 1996.

[GJKR96b] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Robust threshold DSS signatures. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 354–371. Springer, Berlin, Heidelberg, May 1996.

[GJKR07] Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Secure distributed key generation for discrete-log based cryptosystems. *Journal of Cryptology*, 20(1):51–83, January 2007.

[GKPV10] Shafi Goldwasser, Yael Tauman Kalai, Chris Peikert, and Vinod Vaikuntanathan. Robustness of the learning with errors assumption. In *Innovations in Computer Science - ICS 2010, Tsinghua University, Beijing, China, January 5-7, 2010. Proceedings*, pages 230–240. Tsinghua University Press, 2010.

[GKS24] Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 266–300. Springer, Cham, June 2024.

[GKSŚ20] Adam Gągol, Jędrzej Kula, Damian Straszak, and Michał Świętek. Threshold ECDSA for decentralized asset custody. Cryptology ePrint Archive, Report 2020/498, 2020.

[GMPW20] Nicholas Genise, Daniele Micciancio, Chris Peikert, and Michael Walter. Improved discrete gaussian and subgaussian analysis for lattice cryptography. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part I*, volume 12110 of *LNCS*, pages 623–651. Springer, Cham, May 2020.

[GPV08] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008.

[GS23] Jens Groth and Victor Shoup. Fast batched asynchronous distributed key generation. *IACR Cryptol. ePrint Arch.*, page 1175, 2023.

[GVW15] Sergey Gorbunov, Vinod Vaikuntanathan, and Daniel Wichs. Leveled fully homomorphic signatures from standard lattices. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *47th ACM STOC*, pages 469–477. ACM Press, June 2015.

[Har94] Lein Harn. Group-oriented (t, n) threshold digital signature scheme and digital multisignature. *IEE Proceedings-Computers and Digital Techniques*, 141(5):307–313, 1994.

[KG20] Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65. Springer, Cham, October 2020.

[KLSS23] Duhyeong Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-mlwe. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023*, pages 549–580, Cham, 2023. Springer Nature Switzerland.

[KRT24] Shuichi Katsumata, Michael Reichle, and Kaoru Takemure. Adaptively secure 5 round threshold signatures from MLWE/MSIS and DL with rewinding. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part VII*, volume 14926 of *LNCS*, pages 459–491. Springer, Cham, August 2024.

[LDK+22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[Lin22] Yehuda Lindell. Simple three-round multiparty schnorr signing with full simulatability. Cryptology ePrint Archive, Report 2022/374, 2022.

[LJY14] Benoît Libert, Marc Joye, and Moti Yung. Born and raised distributively: fully distributed non-interactive adaptively-secure threshold signatures with short shares. In Magnús M. Halldórsson and Shlomi Dolev, editors, *33rd ACM PODC*, pages 303–312. ACM, July 2014.

[LNP22] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 71–101. Springer, Cham, August 2022.

[LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-LWE cryptography. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013.

[LS15] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Designs, Codes and Cryptography*, 75(3):565–599, 2015.

[Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009.

[Lyu12] Vadim Lyubashevsky. Lattice signatures without trapdoors. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 738–755. Springer, Berlin, Heidelberg, April 2012.

[Mak22] Nikolaos Makriyannis. On the classic protocol for MPC schnorr signatures. Cryptology ePrint Archive, Report 2022/1332, 2022.

[MR04] Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on Gaussian measures. In *45th FOCS*, pages 372–381. IEEE Computer Society Press, October 2004.

[NIS22] NIST. Call for additional digital signature schemes for the post-quantum cryptography standardization process. https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/call-for-proposals-dig-sig-sept-2022.pdf, 2022.

[PB23] René Peralta and Luís T.A.N. Brandão. Nist first call for multi-party threshold schemes. National Institute of Standards and Technology, 2023. https://doi.org/10.6028/NIST.IR.8214C.ipd.

[PS00] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, June 2000.

[Rab98] Tal Rabin. A simplified approach to threshold and proactive RSA. In Hugo Krawczyk, editor, *CRYPTO'98*, volume 1462 of *LNCS*, pages 89–104. Springer, Berlin, Heidelberg, August 1998.

[RRJ+22] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: Robust asynchronous schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 2551–2564. ACM Press, November 2022.

[SG98] Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. In Kaisa Nyberg, editor, *EUROCRYPT'98*, volume 1403 of *LNCS*, pages 1–16. Springer, Berlin, Heidelberg, May / June 1998.

[Sha79] Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.

[Sho00] Victor Shoup. Practical threshold signatures. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 207–220. Springer, Berlin, Heidelberg, May 2000.

[Sho23] Victor Shoup. The many faces of schnorr. Cryptology ePrint Archive, Report 2023/1019, 2023.

[SS01] Douglas R. Stinson and Reto Strobl. Provably secure distributed Schnorr signatures and a (t, n) threshold scheme for implicit certificates. In Vijay Varadharajan and Yi Mu, editors, *ACISP 01*, volume 2119 of *LNCS*, pages 417–434. Springer, Berlin, Heidelberg, July 2001.

[TPCZ23] Guofeng Tang, Bo Pang, Long Chen, and Zhenfeng Zhang. Efficient lattice-based threshold signatures with functional interchangeability. *IEEE Trans. Inf. Forensics Secur.*, 18:4173–4187, 2023.

[TZ23] Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 628–658. Springer, Cham, April 2023.

[WMYC23] Harry W. H. Wong, Jack P. K. Ma, Hoover H. F. Yin, and Sherman S. M. Chow. Real threshold ECDSA. In *NDSS 2023*. The Internet Society, February 2023.

A Correctness from Threshold Signatures with Identifiable Abort

Lemma A.1. *If a threshold signature scheme with identifiable abort* TS *is identifiable abort, then* TS *satisfies correctness.*

Proof. If TS is identifiable abort, for any PPT adversary $\mathcal{A}$, we have $\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}}(\lambda, N, T, T_{\mathsf{ia}}) \leqslant \mathsf{negl}(\lambda)$. Thus, to show this lemma, it is sufficient to show that there exist an adversary $\mathcal{A}'$ against $\mathsf{Game}^{\mathsf{ts\text{-}ia}}$ such that, for any SS and M,

$$\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}'}(\lambda, N, T, T_{\mathsf{ia}}) \geqslant \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 0\right]$$

since $\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 1\right] = 1 - \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 0\right]$ and $\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}'}(\lambda, N, T, T_{\mathsf{ia}}) \leqslant \mathsf{negl}(\lambda)$.

Let us consider an $\mathcal{A}'$ against $\mathsf{Game}^{\mathsf{ts\text{-}ia}}$ who outputs $\mathsf{CS} = \varnothing$ in the initial phase and makes a signing query with SS' and M' such that we have $\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}', \mathsf{SS}') = 0\right]$ is equal to $\max_{\mathsf{SS},\mathsf{M}} \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 0\right]$. From the description of the game $\mathsf{Game}^{\mathsf{ts\text{-}ia}}$, $\mathcal{A}'$ can win the game only when there exists at least one signing query in which the signing protocol failed. Since $\mathcal{A}'$ no longer corrupt any users, SS' includes only honest users. Thus, the probability of failure of the signing protocol in a signing query is equal to $\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}', \mathsf{SS}') = 0\right]$. Then we have

$$\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}'}(\lambda, N, T, T_{\mathsf{ia}}) = \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}', \mathsf{SS}') = 0\right].$$

As we have $\Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}', \mathsf{SS}') = 0\right] \geqslant \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 0\right]$ for any SS and M, we obtain

$$\mathsf{Adv}^{\mathsf{ts\text{-}ia}}_{\mathsf{TS},\mathcal{A}'}(\lambda, N, T, T_{\mathsf{ia}}) \geqslant \Pr\left[\mathsf{Game}^{\mathsf{ts\text{-}cor}}_{\mathsf{TS}}(1^\lambda, N, T, T_{\mathsf{ia}}, \mathsf{M}, \mathsf{SS}) = 0\right]$$

for any SS and M. This completes the proof. □

B Analysis of LaBRADOR with Zero-Knowledge

In this section, we provide a lattice-based succinct NIZK (or ZK-SNARK). Our construction is an instantiation of the folklore paradigm that combines a SNARK (without zero-knowledge) with an appropriate zero-knowledge proof. In particular, we combine the proof system in [LNP22] with LaBRADOR [BS23]. First, we will recall some additional preliminaries, including the framework in which non-interactive LaBRADOR is proven secure [AAB+24]. Then, we recall LaBRADOR and its relevant security properties. Then, we provide a zero-knowledge proof system $\mathrm{LNP}_{\mathsf{z}}$ for our relation R_{z} (which we wish to prove) via [LNP22]. Finally, we combine $\mathrm{LNP}_{\mathsf{z}}$ with LaBRADOR to obtain our ZK-SNARK LNPLab. To analyze security of our construction, we revisit the security of [LNP22] in the analysis framework of [AAB+24] to prove soundness in a modular manner.

B.1 Interactive Proof Systems

For an NP relation $\mathcal{R}$, we denote $\mathcal{R}(\mathbb{x}) = \{\mathbb{w} \mid (\mathbb{x}, \mathbb{w}) \in \mathcal{R}\}$. We denote by $\mathcal{L}_{\mathcal{R}} = \{\mathbb{x} : \exists \mathbb{w} \in \mathcal{R}(\mathbb{x})\}$ the language induced by $\mathcal{R}$. An interactive proof allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ that $\mathbb{x} \in \mathcal{L}_{\mathcal{R}}$ interactively. We recall the definition from [AFK22].

Definition B.1 (Interactive Proof). *An interactive proof* $\Pi = (\mathcal{P}, \mathcal{V})$ *for relation* $\mathcal{R}$ *is an interactive protocol between two probabilistic machines, a prover* $\mathcal{P}$ *and a polynomial time verifier* $\mathcal{V}$*. Both* $\mathcal{P}$ *and* $\mathcal{V}$ *take as public input a statement* $\mathbb{x}$ *and, additionally,* $\mathcal{P}$ *takes as private input a witness* $\mathbb{w} \in \mathcal{R}(\mathbb{x})$*. The verifier* $\mathcal{V}$ *either accepts or rejects and its output is denoted as* $(\mathcal{P}(\mathbb{w}), \mathcal{V})(\mathbb{x})$*. Accordingly, we say the corresponding transcript (i.e., the set of all messages exchanged in the protocol execution) is accepting or rejecting.*

As in [AFK22], we assume that prover $\mathcal{P}$ sends the first and last message. If the interactive proof proceeds in $(2\mu+1)$ moves, we refer to it as $(2\mu+1)$-move protocol. This property is important to transform the interactive proof into an NIZK (cf. Section 2.5).

Definition B.2 (Public coin). *An interactive proof $\Pi = (\mathcal{P}, \mathcal{V})$ is public-coin if all of $\mathcal{V}$'s random choices are made public. The message $\mathsf{chal}_i \xleftarrow{\$} \mathcal{C}_i$ of $\mathcal{V}$ in the $2i$-th move is called the i-th challenge, and $\mathcal{C}_i$ is the challenge set.*

Definition B.3 (Correctness). *An interactive proof $\Pi = (\mathcal{P}, \mathcal{V})$ is correct if for any $(\mathbb{x}, \mathbb{w}) \in \mathcal{R}$, we have $\Pr[(\mathcal{P}(\mathbb{w}), \mathcal{V})(\mathbb{x}) = 1] = 1 - \mathsf{negl}(\lambda)$, where the probability is over the random choices of $\mathcal{P}$ and $\mathcal{V}$.*

Definition B.4 (Fiat-Shamir). *The (adaptive) Fiat-Shamir transformation allows to transform a public coin interactive proof $\Pi = (\mathcal{P}, \mathcal{V})$ into an NIZK. To derive the challenges, the prover simply hashes the transcript so far (including the challenge). We refer to [AFK22] a more formal description. In general, we write $\mathsf{FS}[\Pi]$ for the Fiat-Shamir compiled proof system.*

B.2 Coordinate-Wise Predicate Special Soundness

Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(2\mu+1)$-move public-coin interactive proof. We recall the notion of coordinate-wise predicate special soundness (PSS) [AAB+24]. This framework enables the analysis of Fiat-Shamir compiled proofs $\mathsf{FS}[\Pi]$ and is in particular suitable for lattice-based proofs. Let $\mathcal{S}_m$ be some base challenge set for the m-th move and the challenge set is $\mathcal{C}_m = \mathcal{S}_m^{r_m}$. These definitions are taken almost verbatim from [AAB+24], and we follow their convention that $\vec{s}$ denotes vectors over $\mathcal{S}_m$, $\vec{c} = (\vec{s}_1, \ldots, \vec{s}_n)$ denotes vectors over $\mathcal{C}_m$ for some n.

Definition B.5 (Coordinate-wise Tree of Transcripts). *Let $\mu, k_1, \ldots, k_\mu, r_1, \ldots, r_\mu \in \mathbb{N}_{\geqslant 0}$ and let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(2\mu+1)$-move public-coin interactive proof for a relation $\mathsf{R}_{\mathsf{snd}}$ such that the m-th challenge set is $\mathcal{C}_m = \mathcal{S}_m^{r_m}$ for some set $\mathcal{S}_m$. Additionally, let $m \in [\mu]$ and $\ell \in [k]$.*

- *Let $\mathbb{T}_{\mu+1}$ be the set of possible accepting transcripts for Π.*
- *Let $\mathbb{T}_{m+1}^{(\ell)}$ be the set of possible accepting $(r_1, \ldots, r_\mu)$-coordinate-wise $(1, \ldots, 1, \ell, k_{m+1}, \ldots, k_\mu)$-trees of transcripts, and denote $\mathbb{T}_m = \mathbb{T}_{m+1}^{(k_m)}$. We arrange $t \in \mathbb{T}_{m+1}^{\ell}$ such that $t = (t_1, \vec{t}_2, \ldots, \vec{t}_\ell)$, with $t_1 \in \mathbb{T}_{m+1}$ and $\vec{t}_i \in \mathbb{T}_{m+1}^{r_m}$ for $i > 1$. For any $i \in [r_m]$, the m-th challenges of $t_1, t_{2,i}, \ldots, t_{\ell,i}$ in $\mathcal{S}_m^{r_m}$ are pairwise distinct in the i-th coordinate and equal in other coordinates.*
- *For $t \in \mathbb{T}_{m+1}$, we define $\mathsf{trunk}(t)$ to be the prefix $(a_1, c_1, a_2, c_2, \ldots, a_m)$ shared by all the transcripts in t, $\mathsf{chal}_m(t)$ to be their shared m-th challenge $\vec{s} \in \mathcal{S}_m^{r_m}$, and $\mathsf{chal}_{m,i}(t) = \vec{s}_i$ for $i \in [r_m]$. Furthermore, for $\vec{t} \in \mathbb{T}_{m+1}^{r_m}$, we define $\mathsf{chal}_m^{(}\vec{t})$ to be the vector $\vec{c} = (\vec{s}_1, \ldots, \vec{s}_n) \in \mathcal{C}_m^n$ with $\vec{s}_i = \mathsf{chal}_m(t_i)$.*
- *Finally, let $\mathcal{C}_m^\ell$ denote the set of tuples $(\vec{s}_1, \vec{c}_2, \vec{c}_\ell)$ with $\vec{s}_1 \in \mathcal{C}_m$ and $\vec{c}_i = (\vec{s}_{i,1}, \ldots, \vec{s}_{i,r_m}) \in \mathcal{C}_m^{r_m}$ such that $\vec{s}_1, \vec{s}_{2,i}, \ldots, \vec{s}_{\ell,i}$ are pairwise distinct in the i-th coordinate and equal in the other coordinates. This is the set of all tuples that may occur as $(\mathsf{chal}_m(t_1), \mathsf{chal}_m(\vec{t}_2), \ldots, \mathsf{chal}_m(\vec{t}_\ell))$ for some $(t_1, \vec{t}_2, \ldots, \vec{t}_\ell) \in \mathbb{T}_{m+1}^{(\ell)}$.*

For a $(t_1, \vec{t}_2, \ldots, \vec{t}_\ell) \in \mathbb{T}_{m+1}^{(\ell)}$ and a coordinate $i \in [r_m]$, we refer to t_1 as the first tree for the i-th coordinate, $t_{2,i}$ as the second tree for that coordinate, $t_{3,i}$ as the third, and so on. Likewise, for a $(\vec{s}_1, \vec{c}_2, \ldots, \vec{c}_\ell) \in \mathcal{C}_m^{(\ell)}$, we refer to $\vec{s}_1$ as the first challenge vector for the i-th coordinate, $\vec{s}_{2,i}$ the second, and so on. Furthermore, we say that $s_{1,i}$ is the first challenge for the i-th coordinate, $s_{2,i,i}$ the second, and so on. Because of the asymmetry with the indexing, we will sometimes write $s_{1,i,i}$ to denote $s_{1,i}$ and $t_{1,i}$ to denote t_1.

Definition B.6 (Coordinate-wise Predicates). *Let $m \in [\mu]$ and $\ell \in [k_m]$.*

- *A challenge predicate on level m for the ℓ-th challenge is a polynomial-time computable function*

$$\Phi_{m,\ell}^{\mathsf{chal}} : \mathcal{C}_m^{(\ell-1)} \times [r_m] \times \mathcal{S} \to \{0,1\}.$$

- *A commitment predicate on level m is a pair of polynomial-time computable functions*

$$\Phi_m^{\mathsf{prop}} : \mathbb{T}_{m+1}^{(k_m-1)} \to \{0,1\} \quad \textit{and} \quad \Phi_{m+1}^{\mathsf{bind}} : \mathbb{T}_{m+1}^{(k_m-1)} \times [r_m] \times \mathbb{T}_{m+1} \to \{0,1\}$$

Definition B.7. *Coordiante-wise Pedicate System A predicate system $\mathbf{\Phi}$ for a $(r_1,\ldots,r_\mu)$-coordinate wise $(k_1,\ldots,k_\mu)$-tree structure is a collection of predicates for each level in the tree. The m-th level has one commitment predicate $(\Phi_m^{\mathsf{prop}}, \Phi_m^{\mathsf{bind}})$, and k_m challenge predicates $\Phi_{m,1}^{\mathsf{chal}},\ldots,\Phi_{m,k_m}^{\mathsf{chal}}$. We recursively define a series of boolean functions $\mathbf{\Phi}_m$ for $m \in [\mu+1]$, describing whether a partial tree of transcripts satisfies the predicate system. For a single accepting transcripts $t \in \mathbb{T}_{\mu+1}$, let*

$$\mathbf{\Phi}_{\mu+1}(t) = 1.$$

For all larger subtrees $t = (t_1, \vec{t}_2, \ldots, \vec{t}_{k_m}) \in \mathbb{T}_m$ with $m \in [\mu]$, let $\mathbf{\Phi}_m(t) = 1$ if and only if

$$\bigwedge_{\ell\in[k_m]} \bigwedge_{i\in[r_m]} (\mathbf{\Phi}_{m+1}(t_{\ell,i})) \ \wedge \ \Phi_{m,\ell}^{\mathsf{chal}}((\vec{s}_1,\vec{c}_2,\ldots,\vec{c}_{\ell-1}), i, s_{\ell,i,i} = 1) \ \wedge$$
$$\bigwedge_{i\in[r_m]} \Phi_m^{\mathsf{bind}}((t_1,\vec{t}_2,\ldots,\vec{t}_{k_m-1}), i, t_{k_m,i}) = 1 \ \wedge$$
$$\Phi_m^{\mathsf{prop}}(t_1,\vec{t}_2,\ldots,\vec{t}_{k_m-1}) = 1,$$

where $\vec{s}_1 = \mathsf{chal}_m(t_1)$ and $\vec{c}_j = \mathsf{chal}_m(\vec{t}_j)$ for $j > 1$. For notational convenience, we let $\mathbf{\Phi} = \mathbf{\Phi}_1$.

Definition B.8 (Coordinate-wise Predicate Special Soundness). *Let $\Pi = (\mathcal{P},\mathcal{V})$ be a $2\mu+1$-message public-coin argument of knowledge for a relation $\mathsf{R}_{\mathsf{snd}}$, with m-th challenge space $\mathcal{C}_m = \mathcal{S}_m^{r_m}$. We say that Π is $(\mathbf{R},\mathbf{K},\mathbf{\Phi})$-coordinate-wise predicate-special-sound for $\mathbf{R} = (r_1,\ldots,r_\mu)$, $\mathbf{K} = (k_1,\ldots,k_\mu)$ and a coordinate-wise predicate system $\mathbf{\Phi}$ if there exists a polynomial time algorithm $\mathcal{E}_{\mathsf{pss}}^{\mathsf{cw}}$ which given a statement $\mathbb{x}$ and a $\mathbf{R}$-coordinate-wise $\mathbf{K}$-tree of transcripts t for this statement with $\mathbf{\Phi}(t) = 1$ always outputs a witness $\mathbb{w}$ such that $(\mathbb{x},\mathbb{w}) \in \mathsf{R}_{\mathsf{snd}}$.*

Definition B.9 (Failure Density of Challenge Predicates). *Let $m \in [\mu]$ and $\ell \in [k_m]$. Let $(\vec{s}_1,\vec{c}_2,\ldots,\vec{c}_{\ell-1}) \in \mathcal{C}_m^{(\ell-1)}$ be any collection of challenges such that the first $\ell-1$ challenge predicates are satisfied in every coordinate, meaning*

$$\forall i \in [\ell-1], j \in [r_m] : \Phi_{m,i}^{\mathsf{chal}}((\vec{s}_1,\vec{c}_2,\ldots,\vec{c}_{\ell-1}), j, s_{i,j,j}) = 1.$$

For each coordinate $j \in [r_m]$, consider the set of possible ℓ-th challenges such that $\Phi_{m,\ell}^{\mathsf{chal}}$ fails, that is

$$\mathsf{BadChal}_{m,\ell}(\vec{s}_1,\vec{c}_2,\ldots,\vec{c}_{\ell-1},j) = \left\{ \ s \in \mathcal{S}_m \ \middle| \ \begin{array}{l} s \notin \{s_{1,j}, s_{1,j,j,j},\ldots,s_{\ell-1,j,j}\}, \\ \Phi_{m,\ell}^{\mathsf{chal}}((\vec{c}_1,\ldots,\vec{c}_{\ell-1},j,s)) = 0 \end{array} \ \right\}.$$

The challenge predicate $\Phi_{m,\ell}^{\mathsf{chal}}$ has failure density $p_{m,\ell}^{\mathsf{chal}}$ if for all $j \in [r_m]$ it always holds that

$$|\mathsf{BadChal}_{m,\ell}(\vec{s}_1,\vec{c}_2,\ldots,\vec{c}_{\ell-1},j)| \leqslant p_{m,\ell}^{\mathsf{chal}} \cdot |\mathcal{S}_m|.$$

Definition B.10 (Failure Density of Commitment Predicates). *Let $m \in [\mu]$. Define a set of bad subtrees*

$$\mathsf{PropUnsat}_m = \left\{ \ (t_1,\vec{t}_2,\ldots,\vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)} \ \middle| \ \begin{array}{l} \forall i \in [k_m-1], j \in [r_m] : \\ \mathbf{\Phi}_{m+1}(t_{i,j}) = 1, \\ \Phi_{m,i}^{\mathsf{chal}}(\vec{s}_1,\vec{c}_1,\ldots,\vec{c}_{i-1},j,\vec{s}_{i,j,j}) = 1, \\ \Phi_{t_1,\vec{t}_2,\ldots,\vec{t}_{k_m-1}}^{\mathsf{prop}} = 0 \end{array} \ \right\},$$

using the shorthand $\vec{s}_1 = \mathsf{chal}_m(t_1)$ and $\vec{c}_i = \mathsf{chal}_m(\vec{t}_i)$ for $i > 1$. That is, subtrees in $\mathsf{PropUnsat}_m$ fail to satisfy the property predicate but otherwise satisfy the constraints. For each $(t_1,\vec{t}_2,\ldots,\vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{(k_m-1)}$,

we define $\mathsf{BindTree}_m(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j)$ *to be the set of possible* k_m*-th subtrees that satisfy the binding predicate and the other constraints, using the shorthand* $s = \mathsf{chal}_{m,j}(t)$*.*

$$\mathsf{BindTree}_m(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j) = \left\{ t \in \mathbb{T}_{m+1} \;\middle|\; \begin{array}{l} \forall i \in [k_m - 1] : s \neq s_{i,j,j}, \\ \mathsf{trunk}(t) = \mathsf{trunk}(t_1), \\ \mathbf{\Phi}_{m+1}(t) = 1, \\ \Phi^{\mathsf{chal}}_{m,k_m}((\vec{s}_1, \vec{c}_2, \ldots, \vec{c}_{k_m-1}, j, s)) = 1 \\ \Phi^{\mathsf{bind}}_m t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j, t = 1 \end{array} \right\}.$$

Consider the m*-th level challenges for the* j*-th coordinate occurring for some tree in this set,*

$$\mathsf{BindTree}_m(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j) = \left\{ \mathsf{chal}_{m,j}(t) \mid t \in \mathsf{BindTree}_m(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j) \right\}.$$

The commitment predicate $(\Phi^{\mathsf{prop}}_m, \Phi^{\mathsf{bind}}_m)$ *has failure density* p^{com}_m *if it always holds that there is some coordinate* $j \in [r_m]$ *such that*

$$\left|\mathsf{BindTree}_m(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}, j)\right| \leqslant p^{\mathsf{com}}_m \cdot |\mathcal{S}_m|,$$

for all $t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1} \in \mathsf{PropUnsat}_m$*.*

Definition B.11 (Coordinate-wise Binding Relation). *Let* $\Pi = (\mathcal{P}, \mathcal{V})$ *be a* $(\mathbf{R}, \mathbf{K}, \mathbf{\Phi})$*-coordinate-wise predicate-special-sound argument of knowledge for a relation* $\mathsf{R}_{\mathsf{snd}}$*, and let* $\mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}$ *be an additional relation. We say that* $\mathbf{\Phi}$ *admits* $\mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}$ *as a coordinate-wise binding relation if there exists a polynomial time algorithm* $\mathcal{E}_{\mathsf{bind}}$ *with the following property.*

Algorithm $\mathcal{E}_{\mathsf{bind}}$ *gets as input some statement* $\mathbb{x}$*, trees* $(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}) \in \mathbb{T}^{k_m-1}_{m+1}$ *and* $t \in \mathbb{T}_{m+1}$ *for statement* $\mathbb{x}$*, and an index* $j \in [r_m]$ *such that*

$$\begin{aligned} \forall \ell \in [k_m], i \in [r_m] : \mathbf{\Phi}_{m+1}(t_{\ell,i}) = 1 \;\wedge\; \Phi^{\mathsf{chal}}_{m,\ell}((\vec{s}_1, \vec{c}_2, \ldots, \vec{c}_{\ell-1}), i, s_{\ell,i,i}) = 1 \;\wedge \\ \Phi^{\mathsf{bind}}_m((t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}), j, t) = 0, \end{aligned} \tag{7}$$

where $\vec{s}_1 = \mathsf{chal}_m(t_1)$ *and* $\vec{c}_1 = \mathsf{chal}_m(\vec{t}_i)$ *for* $i > 1$*. Then it always holds that*

$$\mathcal{E}_{\mathsf{bind}}((t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}), j, t) \in \mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}(\mathbb{x}).$$

B.3 Extractor for Fiat-Shamir Compiled Coordinate-wise PSS Protocols

Equipped with the notion of coordinate-wise predicate special soundness (see Section B.2), we can use an extractor to prove knowledge soundness of $\mathsf{FS}[\Pi]$. In particular, [AAB+24] provides an extractor that allows to extract a witness for a single proof-statement pair $(\pi, \mathbb{x})$. As we require a multi-proof variant of knowledge soundness that works for N pairs $(\pi_i, \mathbb{x}_i)$, we first have to extend the extractor to work for many proof systems. Our strategy is to run the single-proof extractor from [AAB+24] N-times. Then, success of our extractor follows via a simple union bound. We elaborate below. Let $\Pi = (\mathcal{P}, \mathcal{V})$ be an interactive proof that is $(\mathbf{R}, \mathbf{K}, \mathbf{\Phi})-$coordinate-wise predicate-special-sound for relation $\mathsf{R}_{\mathsf{snd}}$ with binding relation $\mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}$ (cf. Definitions B.8 and B.11).

First, we recall the single proof extractor from [AAB+24]. (We simplify the description and omit details not required for our purpose.) Roughly, the extractor $\mathcal{E}_{\mathsf{FS}}$ outputs a witness given oracle access to some $\mathcal{A}$ that outputs proof-statement pairs $(\mathbb{x}, \pi)$ for $\mathsf{FS}[\Pi]$. The knowledge error depends on the tree structure and failure densities of the predicates of Π.

Let $\mathcal{A}$ be some adaptive Q-query random oracle prover for $\mathsf{FS}[\Pi]$. That is, after making at most Q queries to the random oracle H, the prover outputs $(\mathbb{x}, \pi, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}$, where $\mathbb{x}$ is a statement, $\pi = (a_1, \ldots, a_{\mu+1})$ is a proof, and aux is some auxiliary information. We assume that $\mathcal{A}$ is deterministic without loss of generality [AAB+24, Remark B.1]. Also, we assume a single random oracle H mapping into challenge space $\mathcal{C}$ (instead of potentially distinct random oracles and challenge spaces for each round) for ease of presentation. As noted in [AAB+24], this can be extended easily. As in [AAB+24], we reformat the output of $\mathcal{A}$ to be an index vector $\vec{I} = (I_1, \ldots, I_{\mu+1})$, where we denote for $i \in [\mu + 1]$

$$I_i = (\mathbb{x}, a_1, \ldots, a_i).$$

Further, let us define a $(Q+\mu)$-query algorithm $\mathcal{A}^*$ that verifies the proofs. That is, $\mathcal{A}^*$ runs $\mathcal{A}$ to obtain $\vec{I}$ and aux, and computes $c_i = \mathsf{H}(I_i)$ for $i \in [\mu]$. Algorithm $\mathcal{A}^*$ then outputs

$$\vec{I}, t = (a_1, \mathsf{chal}_1, \ldots, a_\mu, c_\mu, a_{\mu+1}), v := \mathcal{V}^{\mathsf{H}}(\mathbb{x}, t), b := 1, \mathsf{aux}.$$

Note that the bit b indicates whether t satisfies the binding constraints or not. Let $\mathcal{E}_{\mathsf{FS}}$ denote the extractor given in [AAB+24, Section H.3]. Note that formally, $\mathcal{E}_{\mathsf{FS}}$ is an algorithm with access to random oracle H and prover $\mathcal{A}^*$ with the same input and output format as $\mathcal{A}^*$. We recall two important properties of $\mathcal{E}_{\mathsf{FS}}$. Note that the first lemma summarizes further useful facts from [AAB+24, Section H.2] in addition to [AAB+24, Lemma H.4].

Lemma B.1 (Extension of Lemma H.4 [AAB+24]). *For any fixed choice of random oracle H, let $(\vec{I}, t, v, b) \leftarrow \mathcal{E}_{\mathsf{FS}}^{\mathsf{H},\mathcal{A}^*}$. If $v = 1$, then we have that $t \in \mathbb{T}_1$. Also, the index vector $\vec{I}$ and the auxiliary information aux are equal to those output by $\mathcal{A}$ querying H. Furthermore, if also $b = 1$, then we have that $\mathbf{\Phi}(t) = 1$. On the other hand, if $b = 0$, then there exists an index $m \in [\mu]$ such that t contains trees $(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}) \in \mathbb{T}_{m+1}^{k_m-1}$ and $t^* \in \mathbb{T}_{m+1}$ for statement $\mathbb{x}$, and an index $j \in [r_m]$ such that Eq. (7) holds (i.e., the preconditions for $\mathcal{E}_{\mathsf{bind}}$ in Definition B.11 are met).*

Denote by $\mathcal{T}$ the efficient algorithm that given t extracts $(t_1, \vec{t}_2, \ldots, \vec{t}_{k_m-1}), t^*$, and j as above (if $v = 1$ and $b = 0$).

Lemma B.2 (Lemma H.5 with $\sigma_i = 2$ [AAB+24]). *Let $N = |\mathcal{C}|$. The extractor $\mathcal{E}_{\mathsf{FS}}$ succeeds and outputs $v = 1$ with probability at least*

$$\epsilon(\mathcal{A}) - 2(Q+1)\sum_{i=1}^{\mu} \max\left(\frac{k_i - 1}{|\mathcal{C}_i|}, p_i^{\mathsf{com}} + \sum_{\ell=1}^{k_i} p_{i,\ell}^{\mathsf{chal}}\right),$$

where $\epsilon(\mathcal{A}) = \Pr[\mathcal{V}^{\mathsf{H}}(\mathbb{x}, \pi) = 1 \mid (x, \pi, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}]$. The number of times $\mathcal{E}_{\mathsf{FS}}$ invokes $\mathcal{P}$ is in expectation at most $K_m + Q(K_m - 1)$, where $K_m = \prod_{i=1}^{\mu} k_i$.

Multi-instance Extractor

In this section, we extend the extractor $\mathcal{E}_{\mathsf{FS}}$ from Section B.3 to the multi-instance setting. That is, we define an extractor $\mathcal{E}_{\mathsf{FS},N}$ that allows to extract N witnesses $\mathbb{w}_i$ from proofs π_i for statements $\mathbb{x}_i$ for $i \in [N]$. Without loss of generality, we make the same simplifying assumptions as before. That is, we assume that $\mathcal{A}$ is deterministic and that there is a single random oracle H and challenge space $\mathcal{C}$. Let $\Pi = (\mathcal{P}, \mathcal{V})$ be a $(2\mu+1)$ move proof system.

Preparations. Let $\mathcal{A}$ be some adaptive Q-query random oracle prover for $\mathsf{FS}[\Pi]$, except that $\mathcal{A}$ outputs N statement-proof pairs. That is, after making at most Q queries to the random oracle, the prover outputs $((\mathbb{x}_i, \pi_i)_{i\in[N]}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}$, where $\mathbb{x}_i$ is a statement, $\pi_i = (a_1, \ldots, a_{\mu+1})$ is a proof, and aux is some auxiliary information. Let us denote by $\mathcal{A}_i$ the prover that runs $\mathcal{A}$ but only outputs the pair $(\mathbb{x}_i, \pi_i, \mathsf{aux})$.

As before, we reformat the output of $\mathcal{A}_i$ to be an index vector $\vec{I}_i = (I_{i,1}, \ldots, I_{i,\mu+1})$, where we denote for $j \in [\mu+1]$

$$I_{i,j} = (\mathbb{x}_i, a_{i,1}, \ldots, a_{i,j}).$$

Further, let us define a $(Q+\mu)$-query algorithm $\mathcal{A}_i^*$ that verifies the proofs. That is, $\mathcal{A}_i^*$ runs $\mathcal{A}_i$ to obtain $\vec{I}_i$, and computes $c_{i,j} = \mathsf{H}(I_{i,j})$ for $j \in [\mu]$. Algorithm $\mathcal{A}_i^*$ then outputs

$$\vec{I}_i, t_i = (a_{i,1}, \mathsf{chal}_{i,1}, \ldots, a_{i,\mu}, c_{i,\mu}, a_{i,\mu+1}), v_i := \mathcal{V}^{\mathsf{H}}(\mathbb{x}_i, t_i), b_i := 1, \mathsf{aux}.$$

We assume that across all $\mathcal{A}_i^*$ the random oracle is answered consistently.

$\mathcal{E}^{\mathsf{H},\mathcal{A}}_{\mathsf{FS},N}$

Sample $Q + N \cdot \mu$ oracle outputs $\mathcal{H}$ // Sampling is performed lazily
$((\mathbb{x}_i, \pi_i)_{i\in[N]}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}$ // Oracle H is answered via $\mathcal{H}$
for $i \in [N]$ **do**
  Sample random coin ρ_i for $\mathcal{E}_{\mathsf{FS}}$
  $(\vec{I}_i, t_i, v_i, b_i, \mathsf{aux}) \leftarrow \mathcal{E}^{\mathsf{H},\mathcal{A}^*_i}_{\mathsf{FS}}(\rho_i)$ // Oracle H is answered via $\mathcal{H}$
  if $v_i = 0$ **then**
    return $\perp$
  else if $b_i = 1$ **then**
    $\mathbb{w}_i = \mathcal{E}^{\mathsf{cw}}_{\mathsf{pss}}(\mathbb{x}_i, t_i)$ // cf. Definition B.8
  else
    $\mathsf{in} \leftarrow \mathcal{T}(t)$ // cf. Lemma B.1
    $\mathbb{w}_i = \mathcal{E}_{\mathsf{bind}}(\mathsf{in})$ // cf. Definition B.11
return $(\mathsf{aux}, \mathbb{w}_1, \ldots, \mathbb{w}_N)$

Figure 13: The extractor for N proofs of $\mathsf{FS}[\Pi]$.

The extractor. Our multi-instance extractor $\mathcal{E}_{\mathsf{FS},N}$ is given Fig. 13. The extractor has oracle access to $\mathcal{A}$ (and thus implicitly to $\mathcal{A}^*_i$ defined above) and random oracle H, and outputs auxiliary information aux and N witnesses $\mathbb{w}_i$ for relation $\mathcal{R} = \mathsf{R}_{\mathsf{snd}} \cup \mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}$ if it succeeds. It is (implicitly) parameterized by the proof system Π and $Q \in \mathbb{N}$, the predicate system $\mathbf{\Phi}$, and the extractor $\mathcal{E}_{\mathsf{FS}}$.

Lemma B.3. *For any fixed choice of random oracle* H*, let* $\mathsf{out} \leftarrow \mathcal{E}^{\mathsf{H},\mathcal{A}}_{\mathsf{FS}}$*. If* $\mathsf{out} \neq \perp$*, then* $(\mathbb{x}_i, \mathbb{w}_i) \in \mathcal{R}$*. Also,* $\mathbb{x}_i$ *and* aux *is equal to those output by* $\mathcal{A}$ *querying* H*.*

Proof. This follows by Lemma B.1. □

Lemma B.4. *Let* $N = |\mathcal{C}|$*. The extractor* $\mathcal{E}_{\mathsf{FS}}$ *succeeds and outputs* $\mathsf{out} \neq \perp$ *with probability at least*

$$\epsilon(\mathcal{A}) - N \cdot \kappa,$$

where $\kappa = 2(Q+1)\sum_{i=1}^{\mu} \max\left(\frac{k_i - 1}{|\mathcal{C}_i|}, p^{\mathsf{com}}_i + \sum_{\ell=1}^{k_i} p^{\mathsf{chal}}_{i,\ell}\right)$ *and* $\epsilon(\mathcal{A}) = \Pr[\forall i \in [N] : \mathcal{V}^{\mathsf{H}}(\mathbb{x}_i, \pi_i) = 1 \mid ((\mathbb{x}_i, \pi_i)_{i\in[N]}, \mathsf{aux}) \leftarrow \mathcal{A}^{\mathsf{H}}]$*. The number of times* $\mathcal{E}_{\mathsf{FS}}$ *invokes* $\mathcal{A}$ *is in expectation at most* $N \cdot (K_m + Q(K_m - 1))$*, where* $K_m = \prod_{i=1}^{\mu} k_i$*.*

Proof. Let us analyze the success probability of $\mathcal{E}_{\mathsf{FS},N}$, *i.e.*, the probability that its output is non-$\perp$. Let $i \in [N]$. By Lemma B.1, if $v_i = 1$, then we have two cases.

1. If $b_i = 1$, then it holds that $\mathbf{\Phi}(t_i) = 1$. Then, by Definition B.8 it holds that $\mathbb{w}_i = \mathcal{E}^{\mathsf{cw}}_{\mathsf{pss}}(\mathbb{x}_i, t_i) \in \mathsf{R}_{\mathsf{snd}}(\mathbb{x}_i)$.
2. If $b_i = 0$, then extractor $\mathcal{E}_{\mathsf{bind}}$ outputs a witness for $\mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}(\mathbb{x}_i)$ on input $\mathcal{T}(t_i)$ by Definition B.11. Thus, $\mathbb{w}_i \in \mathsf{R}^{\mathsf{bind}}_{\mathsf{snd}}(\mathbb{x}_i)$.

Note that the witness $\mathbb{w}_i$ is consistent with the statement $\mathbb{x}_i$ that is defined by the initial run of $\mathcal{A}$ by Lemma B.1.

Let us define some events. Denote by $\mathsf{E}_{\mathsf{Ext}}$ the event that the extractor $\mathcal{E}^{\mathsf{H},\mathcal{A}}_{\mathsf{FS},N}$ succeeds, *i.e.*, it outputs some non-$\perp$ value. Further, denote by $\mathsf{E}_{\mathsf{Ver},i}$ the event that the i-th proof verifies in the initial run, and by $\mathsf{E}_{\mathsf{Ver}}$ the event that all proofs verify. Note that $\Pr[\mathsf{E}_{\mathsf{Ver}}] = \epsilon(\mathcal{A})$.

Due to Lemma B.2, we have that the extractor $\mathcal{E}^{\mathsf{H},\mathcal{A}^*_i}_{\mathsf{FS}}$ outputs $v_i = 1$ with probability $\epsilon(\mathcal{A}) - \kappa$. In particular, we have

$$\Pr[\mathsf{E}_{\mathsf{Ver},i} \wedge v_i = 1] \geqslant \Pr[\mathsf{E}_{\mathsf{Ver},i}] - \kappa.$$

Together with $\Pr[\mathsf{E}_{\mathsf{Ver},i} \wedge v_i = 1] + \Pr[\mathsf{E}_{\mathsf{Ver},i} \wedge v_i = 0] = \Pr[\mathsf{E}_{\mathsf{Ver},i}]$, this yields

$$\Pr[\mathsf{E}_{\mathsf{Ver},i} \wedge v = 0] \leqslant \kappa.$$

Then, the above equation together with a union bounds yields

$$\begin{aligned}
\Pr[\mathsf{E}_{\mathsf{Ext}}] &\geqslant \Pr[\mathsf{E}_{\mathsf{Ext}} \wedge \mathsf{E}_{\mathsf{Ver}}] \\
&= \Pr[\mathsf{E}_{\mathsf{Ver}}] - \Pr[\neg\mathsf{E}_{\mathsf{Ext}} \wedge \mathsf{E}_{\mathsf{Ver}}] \\
&\geqslant \epsilon(\mathcal{A}) - \Pr[\mathsf{E}_{\mathsf{Ver}} \wedge \exists i \in [N] : v_i = 0] \\
&\geqslant \epsilon(\mathcal{A}) - \sum_{i\in[N]} \Pr[\mathsf{E}_{\mathsf{Ver}} \wedge v_i = 0] \\
&\geqslant \epsilon(\mathcal{A}) - \sum_{i\in[N]} \Pr[\mathsf{E}_{\mathsf{Ver},i} \wedge v_i = 0] \\
&\geqslant \epsilon(\mathcal{A}) - N \cdot \kappa.
\end{aligned}$$

Also, the number of invocations of $\mathcal{A}$ follows by linearity of expectation and Lemma B.2. □

B.4 LaBRADOR

We give a brief overview of LaBRADOR and refer to [BS23, AAB+24] for more details. We make some simplifying assumptions for readability. Before, let us give an overview of parameters and their meaning in LaBRADOR Table 4. For consistency, we use notational conventions of [AAB+24].

Parameter	Explanation
$\mathcal{R}_q, d, l$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}[X]/(q, X^d + 1)$ of degree d and splitting factor l
q'	Alternative modulus with $q' > nd((q-1)/2)^2$ for computational efficiency
t_1, t_2	Decomposition parameters for commitments
β, σ	Norm bound β and slack $\sigma \geqslant 1$ for the witness
(r, n)	Number of vectors $\mathbf{s}_i \in \mathcal{R}_q^n$ in the witness $\mathbb{w} = (\mathbf{s}_i)_{i\in[r]}$
$\mathsf{pp} = (\mathbf{A}, \mathbf{B}, \mathbf{D})$	Public matrices
(κ, n)	Dimensions for $\mathbf{A} \in \mathcal{R}_{q'}^{\kappa\times n}$
(κ_1, n_1)	Dimensions for $\mathbf{B} \in \mathcal{R}_{q'}^{\kappa_1\times n_1}$ with $n_1 = (r\kappa + \frac{r^2+r}{2}t_2)$
(κ_1, n_2)	Dimensions for $\mathbf{D} \in \mathcal{R}_{q'}^{\kappa_1\times n_2}$ with $n_2 = (\frac{r^2+r}{2}t_1)$
(C_1, C_2)	Bounds for shortness checks chosen according to Lemma B.9.
B	B-well-spread challenge set Definition B.12
T_{op}	Bound on the operator norm of each challenge
b	Decomposition basis for the recursion witness $\mathbf{z} = \vec{\mathbf{z}}_0 + b\mathbf{z}_1$
β'	Norm of the decomposed recursion witness $\beta' = \|\mathbf{z}_0, \mathbf{z}_1\|_2$

Table 4: Overview of relevant parameters used in LaBRADOR.

Let $\mathsf{pp} = (\mathbf{A}, \mathbf{B}, \mathbf{D})$ chosen as in Table 4. LaBRADOR is a $(2\mu+1)$-move succinct proof system for $\mu = 4$. By recursive composition, it yields a succinct argument of knowledge (SNARK). For $\mathbf{r} = \sum_{i=1}^{d-1} r_i X^i \in \mathcal{R}_q$, we denote by $\mathsf{ct}(\mathbf{r})$ its constant coefficient $\mathbf{r}_0$. In particular, LaBRADOR is a proof system for the relation $\mathsf{R}_{\mathsf{LaB}}^{\sigma}$ with $\sigma = 1$ defined below.

$$\mathsf{R}_{\mathsf{LaB}}^{\sigma} := \left\{ (\mathbb{x}, \mathbb{w}) \;\middle|\; \begin{array}{l} \forall f \in \mathcal{F} : f(\mathbf{s}_i, \ldots, \mathbf{s}_r) = \mathbf{0}, \\ \forall f' \in \mathcal{F}' : \mathsf{ct}(f'(\mathbf{s}_i, \ldots, \mathbf{s}_r)) = 0, \\ \sum_{i=1}^{r} \|\mathbf{s}_i\|_2^2 \leqslant \sigma\beta^2. \end{array} \right\},$$
$$\mathbb{x} = (\mathcal{F}, \mathcal{F}', \beta), \mathbb{w} = (\mathbf{s}_1, \ldots, \mathbf{s}_r),$$

where $\mathcal{F}$ and $\mathcal{F}'$ is a family of quadratic dot product functions $f : (\mathcal{R}_q^n)^r \to \mathcal{R}_q$ of the form

$$f(\mathbf{s}_1, \ldots, \mathbf{s}_r) = \sum_{i,j=1}^{r} \mathbf{a}_{i,j} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + \sum_{i=1}^{r} \langle \boldsymbol{\varphi}_i, \mathbf{s}_i \rangle - \mathbf{b},$$

where $\mathbf{a}_{i,j}, \mathbf{b} \in \mathcal{R}_q$ and $\boldsymbol{\varphi}_i \in \mathcal{R}_q^n$. The soundness relation has some slack $\sigma = \sqrt{\lambda/C_2}$, where $\sqrt{\lambda/C_2}\beta < q/C_1$. Below, we recall some properties of LaBRADOR from [AAB+24].

Lemma B.5 ([BS23, AAB+24]). *LaBRADOR is correct.*

Lemma B.6 (Lemma I.5 and Lemma I.6, [AAB+24]). *LaBRADOR with parameters* $\mathsf{pp} = (\mathbf{A}, \mathbf{B}, \mathbf{D})$ *is* $(\mathbf{R}, \mathbf{K}, \boldsymbol{\Phi})$*-coordinate-wise predicate-special-sound, with* $\mathbf{R} = (1, 1, 1, r)$, $\mathbf{K} = (2, 2, 2, 3)$ *and* $\boldsymbol{\Phi}$ *consisting of predicates* $\Phi_{4,1}^{\mathsf{chal}}, \Phi_{4,2}^{\mathsf{chal}}, \Phi_1^{\mathsf{prop}}, \ldots, \Phi_4^{\mathsf{prop}}, \Phi_1^{\mathsf{bind}}, \ldots, \Phi_4^{\mathsf{bind}}$ *with failure densities* $p_{4,1}^{\mathsf{chal}} = p_{4,2}^{\mathsf{chal}} = lB$, $p_1^{\mathsf{com}} = 2^{-\lambda}$, $p_2^{\mathsf{com}} = q^{-\lceil \lambda / \log q \rceil}$, *and* $p_3^{\mathsf{com}} = q^{-d/l}$, $p_4^{\mathsf{com}} = 5B$, *respectively. Further, the predicate system* $\boldsymbol{\Phi}$ *admits binding relation*

$$\mathsf{R}_{\mathsf{msis}}^{\mathsf{LaB}} = \mathsf{R}_{\mathsf{msis}}^{\kappa,n,\mathbf{A},8T_{op}(b+1)\beta'} \cup \mathsf{R}_{\mathsf{msis}}^{\kappa_1,n_1,\mathbf{B},2\beta'} \cup \mathsf{R}_{\mathsf{msis}}^{\kappa_1,n_2,\mathbf{D},2\beta'},$$

where $\mathsf{R}_{\mathsf{msis}}^{\kappa,n,\mathbf{A},\beta^*} = \{\mathbf{v} \mid \mathbf{v} \in \mathcal{R}_q^n, \mathbf{A} \cdot \mathbf{v} = \mathbf{0}, 0 < \|\mathbf{v}\|_2 \leqslant \beta^*\}$ *with* $\beta^* = 8T_{op}(b+1)\beta'$. *Relations* $\mathsf{R}_{\mathsf{msis}}^{\kappa_1,n_1,\mathbf{B},2\beta'}$ *and* $\mathsf{R}_{\mathsf{msis}}^{\kappa_1,n_2,\mathbf{D},2\beta'}$ *are defined accordingly.*

B.5 Exact NIZK

Parameter	Explanation
$\mathcal{R}_q, d, l$	Polynomial ring $\mathcal{R}_q = \mathbb{Z}[X]/(q, X^d + 1)$ of degree $d = 256$ and splitting factor l
$B_{\mathbf{r}}, B_{\mathbf{e}}$	bounds on the norm of $\mathbf{r}_i$ and $\mathbf{e}'_i$
$\ell, \mu = 2(T-1)\ell \log q$	Dimension of $\mathbf{s}_i, \mathbf{r}_i, \mathbf{m}_{i,j} \in \mathcal{R}_q^{\ell}$, dimension of $\mathbf{m}^* = (\mathbf{m}^*_{i,j}, \mathbf{m}^*_{j,i})_{j \neq i} \in \mathcal{R}^{\mu}$
$\mathbf{A}_1, \mathbf{A}_2$	Commitment matrices for Ajtai and BDLOP commitments
$\kappa_{\mathrm{MSIS}}, \kappa_{\mathbf{A}}$	Dimensions of $\mathbf{A}_1 \in \mathcal{R}_q^{\kappa_{\mathrm{MSIS}} \times \ell \log q}$ and $\mathbf{A}_2 \in \mathcal{R}_q^{\kappa_{\mathrm{MSIS}} \times \kappa_{\mathbf{A}}}$
$\mathfrak{s}, \mathfrak{s}_1, \mathfrak{s}_2,$	Standard deviations used in projection proofs and quadratic proofs
(C_1, C_2)	Bounds for shortness checks chosen according to Lemma B.9.
B	B-well-spread challenge set Definition B.12
T_{op}	Bound on the operator norm of each challenge

Table 5: Overview of relevant parameters used in $\mathrm{LNP}_{\mathsf{z}}$.

We want to construct an exact NIZK for the relation R_{z}. To do so we combine the proof of [LNP22] with the LABRADOR SNARK.

$$\begin{aligned}
\mathsf{R}_{\mathsf{z}} := \Big\{ (\mathbb{x}, \mathbb{w}) \;\mid\; & \mathbf{z}_i := c \cdot L_{\mathsf{SS},i} \cdot \mathbf{s}_i + \mathbf{r}_i + \boldsymbol{\Delta}_i \;\wedge\; \mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i \;\wedge \\
& \forall j \in \mathsf{SS} \backslash \{i\}, (D_{i,j} = \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m}_{i,j}, \delta_{i,j}) \;\wedge\; D_{i,j} = \mathsf{C}_{\mathsf{msk}}.\mathsf{Com}(\mathbf{m}_{i,j}, \delta_{i,j})) \;\wedge \\
& \boldsymbol{\Delta}_i = \sum_{j \in \mathsf{SS} \backslash \{i\}} \mathbf{m}_{i,j} - \mathbf{m}_{j,i} \;\wedge\; C_i = \mathsf{C}_{\mathsf{share}}.\mathsf{Com}(\mathbf{s}_i, \gamma_i) \;\wedge \\
& \|\mathbf{r}_i\|_2 \leqslant B_{\mathbf{r}} \;\wedge\; \|\mathbf{e}'_i\|_2 \leqslant B_{\mathbf{e}} \Big\}, \\
\mathbb{x} = {} & (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (D_{i,j}, D_{j,i})_{j \in \mathsf{SS} \backslash \{i\}}, C_i), \\
\mathbb{w} = {} & (\mathbf{s}_i, \gamma_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}_{i,j}, \mathbf{m}_{j,i}, \delta_{i,j}, \delta_{j,i})_{j \in \mathsf{SS} \backslash \{i\}}).
\end{aligned}$$

Note that in Section 4 the shared secret is named $[\![\mathbf{s}]\!]_i$, as we will use this variable as a subscript we simplify this notation to $\mathbf{s}_i$ for clarity. As the proofs in [LNP22] are not proven in the framework of [AAB+24] we will make the NIZK used to prove R_{z} explicit and prove its soundness in this framework. To simplify notations and clarity we will not use the efficiency improvements of [LNP22] that consist in sampling

bimodal Gaussians or using the trace for constant term amortization. These improvements do not change the soundness of the protocol and are only used to reduce signature sizes. We will also present a protocol in which the communication is not small, i.e. the communication flows can scale in the size of the witness, which would prevent us from applying LaBRADOR effectively. We present in Section B.6 how to reduce this communication size simply by sending compressing commitments to each flow and the opening in the last round. The (large) opening can be used as the witness of the LaBRADOR protocol and does not have to be made public.

While we prove the soundness of the explicit NIZK used for $\mathsf{R_z}$, adapting our proof to the generic construction of [LNP22] is completely straightforward. We will start by making the commitments used explicit, we then show that all the constraints in $\mathsf{R_z}$ can be expressed as (the constant term of) quadratic equations over $\mathcal{R}_q$ as well as approximate norm bounds on the witness, finally we will present the proof protocol.

B.5.1 Conventions and Additional Preliminaries.

We consider a ring $\mathcal{R}_q$ of degree $d = 256$, for a prime modulus q. We assume that $\mathcal{R}$ splits l times modulo q, i.e.

$$\mathcal{R}_q = \prod_{i\in[l]} \mathbb{Z}_q[x]/(X^{d/l} - \xi_i)$$

with l such that $q^{d/l} \geqslant 2^\lambda$. As a convention for any $x \in \mathbb{Z}_q$, we will note $x^* \in \mathbb{Z}^{\lceil \log q \rceil}$ the binary decomposition of x, which we extend to polynomials $x \in \mathcal{R}_q$ as $x^* \in \mathcal{R}^{\lceil \log q \rceil}$ such that $x = \sum_1^{\lfloor \log q \rfloor} 2^i x_i^*$, and to vectors of polynomials $\mathbf{x} = (x_1, \ldots, x_k) \in \mathcal{R}_q^k$ as $\mathbf{x}^* = (x_1^*, \ldots, x_k^*) \in \mathcal{R}^{k\lceil \log q \rceil}$. Additionally we will sometimes need to consider vectors of polynomials as integer vectors: for $\mathbf{x} \in \mathcal{R}_q^k$ we will note $\vec{\mathbf{x}} \in \mathbb{Z}_q^{kd}$ the vector of its coefficients (for clarity we will in fact denote all vectors and matrices in $\mathbb{Z}$ with $\vec{\cdot}$ regardless of whether they were converted from a polynomial or not). For $B < 2^d$ We set $\mathrm{pow}(B) := 1 + 2X + \ldots + 2^{\lfloor \log B \rfloor} X^{\lfloor \log B \rfloor} \in \mathcal{R}_q$, and we shorten $\mathrm{pow}(q)$ as simply pow. Note that $\vec{\mathrm{pow}}$ is the so called gadget vector, and that for $x \in \mathbb{Z}_q$ as well as $x \in \mathcal{R}_q$, we have $x = \langle x^*, \vec{\mathrm{pow}} \rangle$, and for $\mathbf{x} \in \mathcal{R}_q^k$, $\mathbf{x} = \langle \mathbf{x}^*, \mathbf{I}_k \otimes \vec{\mathrm{pow}} \rangle$.

We recall some definitions and lemmas that will be necessary to prove the security of our NIZK.

Lemma B.7. *Let σ be the automorphism of $\mathcal{R}$ that maps X to $X^{-1} := -X^{d-1}$. For any $x, y \in \mathcal{R}$ we have:*

$$\langle \vec{x}, \vec{y} \rangle = x \cdot \sigma(y)[0]$$

Where we recall $\vec{x}$ is the $\mathbb{Z}$ vector of the coefficients of x, and $[0]$ denotes the evaluation in 0.

Lemma B.8. *For any $k \in \mathbb{N}$, and $\vec{x} \in \mathbb{Z}^k$.*

$$x \in \{0,1\}^k \Leftrightarrow \langle x, x - \mathbb{1} \rangle = 0$$

Where we recall $\mathbb{1}$ is the integer vector of appropriate dimension containing only ones.

Definition B.12 (Well spread challenge). *Let $\mathcal{C} \subset \mathcal{R}$, and $b \in [0,1]$. We say that $\mathcal{C}$ is B-well-spread if for all $i \in [l]$ and $y \in \mathbb{Z}_q[x]/(X^{d/l} - \xi_i)$:*

$$\Pr_{c \leftarrow \mathcal{C}}[c \mod (X^{d/l} - \xi_i) = y] \leqslant B$$

Lemma B.9 (Heuristic Johnson-Lindenstrauss [LNP22] Lemma 2.9). *For $m, P \in \mathbb{N}$, a bound $b < P/41m$, and $C_2 = 13/2$ and $C_1 = 337$. Let $\vec{w} \in [\pm P/2]^m$, and let $\vec{y} \in [\pm P/2]^m$. Then:*

$$\Pr_{\vec{\mathbf{R}} \in \{0,1\}^{256\times m}}[\|\vec{\mathbf{R}}\vec{w} + \vec{y}\|_2 < \|\vec{w}\|_2 \sqrt{C_2}] \leqslant 2^{-128}$$

$$\Pr_{\vec{\mathbf{R}} \in \{0,1\}^{256\times m}}[\|\vec{\mathbf{R}}\vec{w}\|_2 > \|\vec{w}\|_2 \sqrt{C_1}] \leqslant 2^{-128}$$

B.5.2 Commitments.

The commitment used in [LNP22] was coined ABDLOP as it is a combination of the Ajtai commitment and BDLOP commitment. We cannot use this commitment because all the masks $\mathbf{m}_{i,j}$ and $\mathbf{m}_{j,i}$ have to be committed individually so that their randomness can be shared between different parties, which is why we will use Ajtai commitments and BDLOP commitments separately.

For $\mathsf{C}_{\mathsf{share}}$ we will use the BDLOP commitment, for a message space $\mathcal{M} = \mathcal{R}_q^{\ell}$ the public parameters are two uniform matrices $\mathbf{A}_2 \in \mathcal{R}_q^{\kappa_{\mathrm{MSIS}} \times \kappa_{\mathbf{A}}}$ and $\mathbf{B}_{\mathbf{m}} \in \mathcal{R}_q^{\ell \times \kappa_{\mathbf{A}}}$, as well as a distribution χ_B over $\mathcal{R}$. A user commits to an arbitrary message $\mathbf{m} \in \mathcal{R}_q^{\ell}$ by sampling $\mathbf{u}_{\mathbf{m}} \leftarrow \chi_B^{\kappa_{\mathbf{A}}}$ and outputting:

$$\begin{bmatrix} \mathbf{t}_{\mathbf{A}} \\ \mathbf{t}_{\mathbf{m}} \end{bmatrix} := \begin{bmatrix} \mathbf{A}_2 \\ \mathbf{B}_{\mathbf{m}} \end{bmatrix} \mathbf{u}_{\mathbf{m}} + \begin{bmatrix} 0 \\ \mathbf{m} \end{bmatrix}$$

A valid opening of $(\mathbf{t}_A, \mathbf{t}_{\mathbf{m}})$ is a pair $(\bar{c}, \bar{\mathbf{u}}) \in \mathcal{R} \times \mathcal{R}^{\kappa_{\mathbf{A}}}$ such that $\|\bar{c}\|_2 \leqslant B_{\mathcal{C}}$, $\|\bar{\mathbf{u}}\|_2 \leqslant B_{\mathrm{BD}}$, and $\bar{c}\mathbf{t}_A = \mathbf{A}_2\bar{\mathbf{u}}$. While this commitment's size scales linearly with the message it has the advantage of committing to arbitrary messages and allowing one to append a commitment to a new message $\mathbf{x}$ by sampling a new matrix $\mathbf{B}_{\mathbf{x}}$ and outputting $\mathbf{t}_{\mathbf{x}} := \mathbf{B}_{\mathbf{x}} \cdot \mathbf{u} + \mathbf{x}$ with the same randomness $\mathbf{u}$ as the existing commitment. In our proof we will commit to $\mathbf{s}_i$ as

$$\begin{bmatrix} \mathbf{t}_{\mathbf{A}} \\ \mathbf{t}_{\mathbf{s}_i} \end{bmatrix} := \begin{bmatrix} \mathbf{A}_2 \\ \mathbf{B}_{\mathbf{s}_i} \end{bmatrix} \cdot \mathbf{u} + \begin{bmatrix} 0 \\ \mathbf{s}_i \end{bmatrix}$$

and we will append new values to this commitment as needed in the proof. To simplify notations we will always use the notation $\mathbf{t}_{\mathbf{x}} := \mathbf{B}_{\mathbf{x}} \cdot \mathbf{u} + \mathbf{x}$ (where $\mathbf{x}$ can be any variable) to denote the appended commitment to $\mathbf{x}$ where $\mathbf{B}_{\mathbf{x}}$ has been previously uniformly sampled from the matrices of the appropriate size.

While we could use fresh BDLOP commitments for $\mathsf{C}_{\mathsf{msk}}$, it will be more efficient to use Ajtai commitments that do not scale in the size of the message. For a message space $\mathcal{M} \subset \mathcal{R}^{\ell}$ the public parameters are two uniform matrices $\mathbf{A}_1 \in \mathcal{R}_q^{\kappa_{\mathrm{MSIS}} \times \ell}$, and $\mathbf{A}_2 \in \mathcal{R}_q^{\kappa_{\mathrm{MSIS}} \times \kappa_c}$, as well as a distribution χ_A over $\mathcal{R}$. We will use the same matrix $\mathbf{A}_2$ as the BDLOP commitment, this is not necessary in practice but helps simplify notations. A user commits to a small message $\mathbf{m} \in \mathcal{R}^{\ell}$ by sampling $\mathbf{u}_{\mathbf{m}} \leftarrow \chi_A^{\kappa_c}$ and outputting:

$$\mathbf{v}_{\mathbf{m}} := \mathbf{A}_1\mathbf{m} + \mathbf{A}_2\mathbf{u}_{\mathbf{m}}$$

A valid opening of $\mathbf{v}_{\mathbf{m}}$ is a tuple $(\bar{c}, \bar{\mathbf{m}}, \bar{\mathbf{u}}) \in \mathcal{R} \times \mathcal{R}^{\ell} \times \mathcal{R}^{\kappa_{\mathbf{A}}}$ such that $\|\bar{c}\|_2 \leqslant B_{\mathcal{C}}$, $\|(\bar{\mathbf{m}}, \bar{\mathbf{u}})\|_2 \leqslant B_{\mathrm{Aj}}$, and $\bar{c}\mathbf{v}_{\mathbf{m}} = \mathbf{A}_1\bar{\mathbf{m}} + \mathbf{A}_2\bar{\mathbf{u}}$. The advantage of this commitment scheme is that its size does not scale with the length of the message. Since Ajtai commitments can only commit to small messages we will commit to the binary decomposition of the masks.

The masks $\mathbf{m}_{i,j}$ (and $\mathbf{m}_{j,i}$) will be committed to as:

$$\mathbf{v}_{i,j} := \mathbf{A}_1 \cdot \mathbf{m}_{i,j}^{*} + \mathbf{A}_2 \cdot \mathbf{u}_{i,j}$$

With $\mathbf{A}_1$ of size $\kappa_{\mathrm{MSIS}} \times (\ell \log q)$. To simplify notations we will consider the vector composed of all commitments:

$$\mathbf{v}_{\mathbf{m}*} := (\mathbf{v}_{\mathbf{m}_{i,j}*}, \mathbf{v}_{\mathbf{m}_{j,i}*})_{j \neq i} = (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1)\mathbf{m}^{*} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2)\mathbf{u}_{\mathbf{m}*}$$

with $\mathbf{m}^{*} := (\mathbf{m}_{i,j}*, \mathbf{m}_{j,i}*)_{j \neq i} \in \mathcal{R}^{\mu}$.

B.5.3 Rewriting R_{z} as quadratic equations in $\mathcal{R}_q$.

In essence [LNP22] uses clever rewriting of norm constraints to construct exact proofs only using quadratic relations over $\mathcal{R}_q$, automorphisms, and very relaxed proofs of shortness (only used to prove that $x \mod q = x$ for some x). We show how to modify the relation R_{z} to only use these building blocks. We will first modify R_{z} into a different relation R'_{z} in which we commit to the binary decomposition of the $\mathbf{m}_{i,j}$ and $\mathbf{m}_{j,i}$ instead, this is purely for efficiency reasons as committing to binary vectors if more efficient than arbitrary vectors (cf. previous paragraph).

Π

Projection:

$\mathbf{s_R} := (\mathbf{r}_i, \mathbf{e}'_i, \mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})$

$\forall i \in [d]: \; y_{\mathbf{R},i} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z},\mathfrak{s}}$

$\mathbf{t}_{y_\mathbf{R}} := \mathbf{B}_{y_\mathbf{R}} \cdot \mathbf{u} + (y_{\mathbf{R},1}, \ldots, y_{\mathbf{R},d})$

Send $(\mathbf{t_A}, \mathbf{t}_{\mathbf{s}_i}, t_{\beta_\mathbf{r}}, t_{\beta_\mathbf{e}}, \mathbf{t}_{y_\mathbf{R}})$

Receive $(\vec{\mathbf{R}} \leftarrow \{0,1\}^{d \times (2\ell+\mu+2)\cdot d})$

$\vec{z}_\mathbf{R} := \vec{\mathbf{R}} \cdot \vec{\mathbf{s}}_\mathbf{R} + \overrightarrow{(y_{\mathbf{R},1}, \cdots, y_{\mathbf{R},d})}$

if $(\mathrm{Rej}(\vec{z}_\mathbf{R}, \gamma) == \perp)$ **do**

  abort

Send $(\vec{z}_\mathbf{R})$

Constant coefficients embedding:

$g_1, \ldots, g_\tau \leftarrow \{x \in \mathcal{R}_q : x[0] = 0\}$

$t_{g_i} := \mathbf{b}_{g_i}^\top \cdot \mathbf{u} + g_i$

Send $(t_{g_1}, \ldots, t_{g_\tau})$

Receive $((v_{i,j}) \leftarrow {\mathbb{Z}_q}^{\tau \times (3+d)})$

for $i \in [\tau]$ **do**

  $h_i = g_i + \sum_{j \in [3+d]} v_{i,j} F_j(\mathbf{s})$

Equation Aggregation:

Receive $(\mu_i) \in \mathcal{R}_q^\tau$

$\mathbf{E} := \sum \mu_i \mathbf{E}'_i$

$\mathbf{e} := \sum \mu_i \mathbf{e}'_i$

$e := \sum \mu_i e'_i$

Quadratic proof:

$\mathbf{s} := (\mathbf{s}_i, \mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e}, y_{\mathbf{R},1}, \ldots, y_{\mathbf{R},d}, g_1, \ldots, g_\tau)$

$\mathbf{y}_{\mathbf{m},1} \leftarrow D^\mu_{\mathfrak{s}_1}$

$\mathbf{y}_{\mathbf{m},2} \leftarrow D^{\kappa_\mathbf{A}}_{\mathfrak{s}_2}$

$\mathbf{y_u} \leftarrow D^{\kappa_\mathbf{A}}_{\mathfrak{s}_2}$

$\mathbf{w_m} := (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1)\mathbf{y}_{\mathbf{m},1} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2)\mathbf{y}_{\mathbf{m},2}$

$\mathbf{w_u} := \mathbf{A}_2 \mathbf{y_u}$

$\mathbf{y} := \left[(\mathbf{B}_{\mathbf{s}_i}\mathbf{y_u})^\top \quad -\mathbf{y}_{\mathbf{m},1}^\top \quad (\mathbf{B}_{\beta_\mathbf{r}}\mathbf{y_u})^\top \quad (\mathbf{B}_{\beta_\mathbf{e}}\mathbf{y_u})^\top \quad (\mathbf{B}_{\mathbf{y_R}}\mathbf{y_u})^\top \quad (\mathbf{b}_{g_1}^\top\mathbf{y_u})^\top \quad \ldots \quad (\mathbf{b}_{g_\tau}^\top\mathbf{y_u})^\top\right]^\top$

$g := (\mathbf{s}, \sigma(\mathbf{s}))^\top \cdot (\mathbf{E} + \mathbf{E}^\top) \cdot (\mathbf{y}, \sigma(\mathbf{y})) + \mathbf{e}^\top \cdot (\mathbf{y}, \sigma(\mathbf{y}))$

$t_g := \mathbf{b}_g^\top \mathbf{u} + g$

$v := (\mathbf{y}, \sigma(\mathbf{y}))^\top \cdot \mathbf{E} \cdot (\mathbf{y}, \sigma(\mathbf{y})) + \mathbf{b}_g \mathbf{y_u}$

Send $(\mathbf{w_m}, \mathbf{w_u}, t_g, v)$

Receive $c \leftarrow \mathcal{C}$

$\mathbf{z}_{\mathbf{m},1} := c\mathbf{m}^* + \mathbf{y}_{\mathbf{m},1}$

$\mathbf{z}_{\mathbf{m},2} := c\mathbf{u_m} + \mathbf{y}_{\mathbf{m},2}$

$\mathbf{z_u} := c\mathbf{u} + \mathbf{y_u}$

if $(\mathrm{Rej}(\mathbf{z}_{\mathbf{m},1}, \gamma_1) == \perp \wedge \mathrm{Rej}((\mathbf{z}_{\mathbf{m},2}, \mathbf{z_u}), \gamma_2) == \perp)$ **do**

  abort

Send $(\mathbf{z}_{\mathbf{m},1}, \mathbf{z}_{\mathbf{m},2}, \mathbf{z_u})$

Figure 14: The exact NIZK $\mathrm{LNP}_\mathbf{z}$.

Π

1 : **Projection:**

2 : Check $\|\vec{z}_{\mathbf{R}}\|_2 \leqslant \sqrt{C_2} B_R$

3 : **Constant coefficients embedding:**

4 : Check $\forall i \in [\tau], h_i[0] = 0$

5 : **Quadratic proof:**

6 : $$\mathbf{z} := c \cdot \begin{bmatrix} \mathbf{t}_{\mathbf{s}_i} \\ 0 \\ t_{\beta_{\mathbf{r}}} \\ t_{\beta_{\mathbf{e}}} \\ \mathbf{t}_{\mathbf{y_R}} \\ t_{g_1} \\ \vdots \\ t_{g_\tau} \end{bmatrix} - \begin{bmatrix} \mathbf{B}_{\mathbf{s}_i}\mathbf{z_u} \\ -\mathbf{z}_{\mathbf{m},1} \\ \mathbf{B}_{\beta_{\mathbf{r}}}\mathbf{z_u} \\ \mathbf{B}_{\beta_{\mathbf{e}}}\mathbf{z_u} \\ \mathbf{B}_{\mathbf{y_R}}\mathbf{z_u} \\ \mathbf{b}_{g_1}^\top \mathbf{z_u} \\ \vdots \\ \mathbf{b}_{g_\tau}^\top \mathbf{z_u} \end{bmatrix} + \mathbf{y}$$

7 : $f := ct_g - \mathbf{b}_g^\top \mathbf{z_u}$

8 : Check $c\mathbf{v_m} + \mathbf{w_m} = (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1) \cdot \mathbf{z}_{\mathbf{m},1} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2) \cdot \mathbf{z}_{\mathbf{m},2}$

9 : Check $c\mathbf{t_A} + \mathbf{w_u} = \mathbf{A}_2\mathbf{z_u}$

10 : Check $(\mathbf{z}, \sigma(\mathbf{z}))^\top \cdot \mathbf{E} \cdot (\mathbf{z}, \sigma(\mathbf{z})) + c\mathbf{e}^\top \cdot (\mathbf{z}, \sigma(\mathbf{z})) + c^2 e - f = v$

11 : Check $\|\mathbf{z}_{\mathbf{m},1}, \mathbf{z}_{\mathbf{m},2}, \mathbf{z_u}\|_2 \leqslant B_{\mathbf{z}}$

Figure 15: Verification of the exact NIZK $\mathrm{LNP}_{\mathbf{z}}$.

Mapping norm inequalities to equalities over $\mathbb{Z}$. We will also commit to two binary polynomials $\beta_{\mathbf{r}}$ and $\beta_{\mathbf{e}}$ such that $\left\langle \vec{\beta}_{\mathbf{r}}, \vec{\mathrm{pow}} \right\rangle = B_{\mathbf{r}}^2 - \|\mathbf{r}_i\|_2^2$ and $\left\langle \vec{\beta}_{\mathbf{e}}, \vec{\mathrm{pow}} \right\rangle = B_{\mathbf{e}}^2 - \|\mathbf{e}'_i\|_2^2$ so that we can transform all norm inequalities into equality tests with 0.

To help clarify the statements in $\mathsf{R}'_{\mathbf{z}}$ we will explicitly state over which ring each equation is considered.

$$\mathsf{R}'_{\mathbf{z}} := \Big\{ (\mathbb{x}, \mathbb{w}) \;\Big|\; \textbf{Commitments:}$$

$$\begin{bmatrix} \mathbf{t_A} \\ \mathbf{t}_{\mathbf{s}_i} \\ t_{\beta_{\mathbf{r}}} \\ t_{\beta_{\mathbf{e}}} \end{bmatrix} := \begin{bmatrix} \mathbf{A}_2 \\ \mathbf{B}_{\mathbf{s}_i} \\ \mathbf{b}_{\beta_{\mathbf{r}}}^\top \\ \mathbf{b}_{\beta_{\mathbf{e}}}^\top \end{bmatrix} \cdot \mathbf{u} + \begin{bmatrix} 0 \\ \mathbf{s}_i \\ \beta_{\mathbf{r}} \\ \beta_{\mathbf{e}} \end{bmatrix} \;\wedge$$

$$\forall j \in \mathsf{SS} \backslash \{i\}, (\mathbf{v}_{i,j} := \mathbf{A}_1 \mathbf{m}^*_{i,j} + \mathbf{A}_2 \mathbf{u}_{i,j}) \;\wedge\; (\mathbf{v}_{j,i} := \mathbf{A}_1 \mathbf{m}^*_{j,i} + \mathbf{A}_2 \mathbf{u}_{j,i}) \;\wedge$$

Relations in $\mathcal{R}_q$:

$$\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot \mathbf{s}_i + \mathbf{r}_i + \boldsymbol{\Delta}_i \in \mathcal{R}_q \;\wedge\; \mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i \in \mathcal{R}_q$$

$$\wedge\; \boldsymbol{\Delta}_i = \sum_{j \in \mathsf{SS} \backslash \{i\}} \left\langle \mathbf{m}^*_{i,j} - \mathbf{m}^*_{j,i}, \vec{\mathrm{pow}} \right\rangle \in \mathcal{R}_q \;\wedge$$

Norm Equations:

$$\|\mathbf{r}_i\|_2^2 + \left\langle \vec{\beta}_{\mathbf{r}}, \vec{\mathrm{pow}}(B_{\mathbf{r}}^2) \right\rangle = B_{\mathbf{r}}^2 \in \mathbb{Z} \;\wedge$$

$$\|\mathbf{e}'_i\|_2^2 + \left\langle \vec{\beta}_{\mathbf{e}}, \vec{\mathrm{pow}}(B_{\mathbf{e}}^2) \right\rangle = B_{\mathbf{e}}^2 \in \mathbb{Z} \;\wedge$$

$$\left\langle \vec{\beta}_{\mathbf{r}}, \vec{\beta}_{\mathbf{r}} - \vec{\mathbb{1}} \right\rangle = 0 \in \mathbb{Z} \;\wedge$$

$$\left\langle \vec{\beta}_{\mathbf{e}}, \vec{\beta}_{\mathbf{e}} - \vec{\mathbb{1}} \right\rangle = 0 \in \mathbb{Z} \;\wedge$$

$$\forall j \in \mathsf{SS} \backslash \{i\}, \left\langle \vec{\mathbf{m}}^*_{i,j}, \vec{\mathbf{m}}^*_{i,j} - \vec{\mathbb{1}} \right\rangle = 0 \in \mathbb{Z} \;\wedge$$

$$\forall j \in \mathsf{SS}\backslash\{i\},\ \left\langle \vec{\mathbf{m}}^*_{j,i}, \vec{\mathbf{m}}^*_{j,i} - \vec{\mathbb{1}} \right\rangle = 0 \in \mathbb{Z}\ \wedge$$

$\Big\}$,

$\mathbb{x} = (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, (\mathbf{v}_{i,j}, \mathbf{v}_{j,i})_{j\in\mathsf{SS}\backslash\{i\}}, \mathbf{t}_A, \mathbf{t}_\mathbf{s}, \mathbf{t}_{\beta_\mathbf{r}}, \mathbf{t}_{\beta_\mathbf{e}})$,

$\mathbb{w} = (\mathbf{s}_i, \mathbf{u}_i, \mathbf{r}_i, \mathbf{e}'_i, (\mathbf{m}^*_{i,j}, \mathbf{m}^*_{j,i}, \mathbf{u}_{i,j}, \mathbf{u}_{j,i})_{j\in\mathsf{SS}\backslash\{i\}}, \beta_\mathbf{r}, \beta_\mathbf{e})$.

Mapping equations in $\mathbb{Z}$ to equations in $\mathbb{Z}_q$. We start by mapping equations in $\mathbb{Z}$ to $\mathbb{Z}_q$ simply by observing that $x = 0 \in \mathbb{Z} \Leftrightarrow x = 0 \in \mathbb{Z}_q \ \wedge\ 0 \leqslant x < q \in \mathbb{Z}$. For simplicity we will also write $\mathbf{m}^* := ((\mathbf{m}^*_{i,j})_{i\in\mathsf{SS}}, (\mathbf{m}^*_{j,i})_{i\in\mathsf{SS}})$.

$$\mathsf{R}'_\mathsf{z} := \Big\{(\mathbb{x}, \mathbb{w}) \ \mid\ \textbf{Commitments:}$$

$$\begin{bmatrix} \mathbf{t}_\mathbf{A} \\ \mathbf{t}_{\mathbf{s}_i} \\ t_{\beta_\mathbf{r}} \\ t_{\beta_\mathbf{e}} \end{bmatrix} := \begin{bmatrix} \mathbf{A}_2 \\ \mathbf{B}_{\mathbf{s}_i} \\ \mathbf{b}^\top_{\beta_\mathbf{r}} \\ \mathbf{b}^\top_{\beta_\mathbf{e}} \end{bmatrix} \cdot \mathbf{u} + \begin{bmatrix} 0 \\ \mathbf{s}_i \\ \beta_\mathbf{r} \\ \beta_\mathbf{e} \end{bmatrix} \ \wedge$$

$$\mathbf{v}_{\mathbf{m}^*} := (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1)\mathbf{m}^* + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2)\mathbf{u}_{\mathbf{m}^*}$$

Relations in $\mathcal{R}_q$:

$$\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot \mathbf{s}_i + \mathbf{r}_i + \mathbf{\Delta}_i \in \mathcal{R}_q \ \wedge\ \mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i \in \mathcal{R}_q$$

$$\wedge\ \mathbf{\Delta}_i = \langle \mathbf{m}^*, (\mathbb{1}_{T-1} \otimes \vec{\mathrm{pow}}, -\mathbb{1}_{T-1} \otimes \vec{\mathrm{pow}}) \rangle \in \mathcal{R}_q \ \wedge$$

Norm Equations:

$$\|\mathbf{r}_i\|_2^2 + \left\langle \vec{\beta_\mathbf{r}}, \vec{\mathrm{pow}}(B^2_\mathbf{r}) \right\rangle = B^2_\mathbf{r} \in \mathbb{Z}_q \ \wedge$$

$$\|\mathbf{r}_i\|_2^2 + \left\langle \vec{\beta_\mathbf{r}}, \vec{\mathrm{pow}}(B^2_\mathbf{r}) \right\rangle < q \in \mathbb{Z} \ \wedge$$

$$\|\mathbf{e}'_i\|_2^2 + \left\langle \vec{\beta_\mathbf{e}}, \vec{\mathrm{pow}}(B^2_\mathbf{e}) \right\rangle = B^2_\mathbf{e} \in \mathbb{Z}_q \ \wedge$$

$$\|\mathbf{e}'_i\|_2^2 + \left\langle \vec{\beta_\mathbf{e}}, \vec{\mathrm{pow}}(B^2_\mathbf{e}) \right\rangle < q \in \mathbb{Z} \ \wedge$$

$$\left\langle \overrightarrow{(\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})}, \overrightarrow{(\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})} - \vec{\mathbb{1}} \right\rangle = 0 \in \mathbb{Z}_q \ \wedge$$

$$\left\langle \overrightarrow{(\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})}, \overrightarrow{(\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})} - \vec{\mathbb{1}} \right\rangle < q \in \mathbb{Z} \ \wedge$$

$\Big\}$,

$\mathbb{x} = (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, \mathbf{v}_{\mathbf{m}^*}, \mathbf{t}_A, \mathbf{t}_\mathbf{s}, \mathbf{t}_{\beta_\mathbf{r}}, \mathbf{t}_{\beta_\mathbf{e}})$,

$\mathbb{w} = (\mathbf{s}_i, \mathbf{u}_i, \mathbf{r}_i, \mathbf{e}'_i, \mathbf{m}^*, \mathbf{u}_{\mathbf{m}^*}, \beta_\mathbf{r}, \beta_\mathbf{e})$.

Mapping equations in $\mathbb{Z}_q$ to $\mathcal{R}_q$. To prove equations over $\mathbb{Z}_q$, [LNP22] observes that by using the automorphism σ that maps X to X^{-1}, we have for any polynomials $x, y \in \mathcal{R}_q$:

$$\langle \vec{x}, \vec{y} \rangle = x \cdot \sigma(y)\ [0]$$

where $[0]$ denotes evaluation in 0 (i.e. the constant term). Equations in the constant term of polynomials will then reduce to equations over $\mathcal{R}_q$ by masking the rest of the polynomial. Using this we can rewrite R_z using only equations over $\mathcal{R}_q$ (in the constant term) and relaxed norm bounds:

$$\mathsf{R}'_\mathsf{z} := \Big\{(\mathbb{x}, \mathbb{w}) \ \mid\ \textbf{Commitments:}$$

$$\begin{bmatrix} \mathbf{t}_\mathbf{A} \\ \mathbf{t}_{\mathbf{s}_i} \\ t_{\beta_\mathbf{r}} \\ t_{\beta_\mathbf{e}} \end{bmatrix} := \begin{bmatrix} \mathbf{A}_2 \\ \mathbf{B}_{\mathbf{s}_i} \\ \mathbf{b}^\top_{\beta_\mathbf{r}} \\ \mathbf{b}^\top_{\beta_\mathbf{e}} \end{bmatrix} \cdot \mathbf{u} + \begin{bmatrix} 0 \\ \mathbf{s}_i \\ \beta_\mathbf{r} \\ \beta_\mathbf{e} \end{bmatrix} \ \wedge$$

$$\mathbf{v}_{\mathbf{m}^*} := (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1)\mathbf{m}^* + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2)\mathbf{u}_{\mathbf{m}^*}$$

Relations in $\mathcal{R}_q$:

$$\mathbf{z}_i = c \cdot L_{\mathsf{SS},i} \cdot \mathbf{s}_i + \mathbf{r}_i + \mathbf{\Delta}_i \in \mathcal{R}_q \ \wedge \ \mathbf{w}_i = \mathbf{A}\mathbf{r}_i + \mathbf{e}'_i \in \mathcal{R}_q$$
$$\wedge \ \mathbf{\Delta}_i = \langle \mathbf{m}^*, (\mathbb{1}_{T-1} \otimes \vec{\text{pow}}, -\mathbb{1}_{T-1} \otimes \vec{\text{pow}}) \rangle \in \mathcal{R}_q \ \wedge$$

Norm Equations:

$$\mathbf{r}_i^\top \cdot \sigma(\mathbf{r}_i) + \beta_{\mathbf{r}} \cdot \sigma(\text{pow}(B_{\mathbf{r}}^2))\ [0] = B_{\mathbf{r}}^2 \in \mathcal{R}_q \ \wedge \tag{8}$$

$$\|\mathbf{r}_i\|_2^2 + \left\langle \vec{\beta_{\mathbf{r}}}, \vec{\text{pow}}(B_{\mathbf{r}}^2) \right\rangle < q \in \mathbb{Z} \ \wedge \tag{9}$$

$$\mathbf{e}_i'^\top \cdot \sigma(\mathbf{e}'_i) + \beta_{\mathbf{e}} \cdot \sigma(\text{pow}(B_{\mathbf{e}}^2))\ [0] = B_{\mathbf{e}}^2 \in \mathcal{R}_q \ \wedge \tag{10}$$

$$\|\mathbf{e}'_i\|_2^2 + \left\langle \vec{\beta_{\mathbf{e}}}, \vec{\text{pow}}(B_{\mathbf{e}}^2) \right\rangle < q \in \mathbb{Z} \ \wedge \tag{11}$$

$$(\mathbf{m}^*, \beta_{\mathbf{r}}, \beta_{\mathbf{e}})^\top \cdot \sigma((\mathbf{m}^*, \beta_{\mathbf{r}}, \beta_{\mathbf{e}}) - \mathbb{1})\ [0] = 0 \in \mathcal{R}_q \ \wedge \tag{12}$$

$$\left\langle \overrightarrow{(\mathbf{m}^*, \beta_{\mathbf{r}}, \beta_{\mathbf{e}})}, \overrightarrow{(\mathbf{m}^*, \beta_{\mathbf{r}}, \beta_{\mathbf{e}})} - \vec{\mathbb{1}} \right\rangle < q \in \mathbb{Z} \ \wedge \tag{13}$$

$\Big\}$,

$$\mathbb{x} = (L_{\mathsf{SS},i}, \mathbf{z}_i, \mathbf{w}_i, c, \mathbf{v}_{\mathbf{m}^*}, \mathbf{t}_A, \mathbf{t}_{\mathbf{s}}, \mathbf{t}_{\beta_{\mathbf{r}}}, \mathbf{t}_{\beta_{\mathbf{e}}}),$$
$$\mathbb{w} = (\mathbf{s}_i, \mathbf{u}_i, \mathbf{r}_i, \mathbf{e}'_i, \mathbf{m}^*, \mathbf{u}_{\mathbf{m}^*}, \beta_{\mathbf{r}}, \beta_{\mathbf{e}}).$$

This is the final relation which we will prove in the next section.

B.5.4 Constructing the ZKP.

Projection proof. The first part of the zero-knowledge proof aims at proving that all equations pertaining to norms can be considered over $\mathcal{R}_q$, i.e. that all elements that should be small are at least smaller than q. To do so we prove a stronger relation than Eqs. (9), (11) and (13) in which we show that $\|(\mathbf{r}_i, \mathbf{e}'_i, \beta_{\mathbf{r}}, \beta_{\mathbf{e}}, \mathbf{m}^*)\|_2 \leqslant B_R$ for some B_R such that:

$$B_R{}^2 + \sqrt{(2\ell + \mu + 2)d} B_R < q$$
$$B_R{}^2 + 2\max(B_{\mathbf{e}}^2, B_{\mathbf{r}}^2) - 1 < q$$

If those equations hold then $\|\beta_{\mathbf{r}}\|_2 \leqslant B_R$ implies $|\left\langle \vec{\beta_{\mathbf{r}}}, \vec{\beta_{\mathbf{r}}} - \vec{\mathbb{1}} \right\rangle| \leqslant \|\beta_{\mathbf{r}}\|_2^2 + \|\beta_{\mathbf{r}}\|\|\mathbb{1}\| \leqslant B_R{}^2 + \sqrt{d} B_R < q$ (the same argument applies to $\beta_{\mathbf{e}}$, $\mathbf{m}^*_{i,j}$, and $\mathbf{m}^*_{j,i}$). Assuming that extraction from subsequent proofs combined with the previous equation proves that $\beta_{\mathbf{r}}$ and $\beta_{\mathbf{e}}$ are binary (which is proven in the following rounds) then $\|\mathbf{r}_i\|_2 \leqslant B_R$ implies $|\|\mathbf{r}_i\|_2^2 + \left\langle \vec{\beta_{\mathbf{r}}}, \vec{\text{pow}}(B_{\mathbf{r}}^2) \right\rangle| \leqslant B_R{}^2 + 2B_{\mathbf{r}}^2 - 1 < q$ (the same argument applies to $\mathbf{e}'_i$).

Lines 1 to 10 of Fig. 14 correspond to the projection proof. From them and Lemma B.9 we could extract some vector $\tilde{\mathbf{s}}_{\mathbf{R}}$ such that $\|\tilde{\mathbf{s}}_{\mathbf{R}}\|_2 \leqslant B_R$, however this extraction does not verify any equations yet. We first want this witness to verify all the equations in R'_{z}, and additionally we want to prove that $\vec{z}_{\mathbf{R}}$ was properly computed in Fig. 14. To that end we commit to every integer $y_{\mathbf{R},i}$ and add them to the witness and the equation $\vec{z} = \vec{\mathbf{R}} \cdot \vec{\mathbf{s}}_{\mathbf{R}} + \vec{y}_{\mathbf{R}} \in \mathbb{Z}_q^{256}$ to the list of equations to be verified. We commit to each $y_{\mathbf{R},i}$ individually because mapping the operation $\vec{\mathbf{R}} \cdot \vec{\mathbf{s}}_{\mathbf{R}}$ to $\mathcal{R}_q$ is complicated while mapping $\vec{\rho}_i^\top \cdot \vec{\mathbf{s}}_{\mathbf{R}}$ (with $\vec{\rho}_i^\top$ the i^{th} row of $\mathbf{R}$) to $\mathcal{R}_q$ is easy (cf. next paragraph). This entails that we will add $d = 256$ additional linear equations to the constraints. Since all equations can be collapsed into one this is not an issue.

Constant term proof. Now that we are only left with modular equations, we will rewrite all constraints as a set of quadratic equations in $(\mathbf{s}, \sigma(\mathbf{s}))$, with

$$\mathbf{s} := (\mathbf{s}_i, \mathbf{m}^*, \beta_{\mathbf{r}}, \beta_{\mathbf{e}}, y_{\mathbf{R},1}, \ldots, y_{\mathbf{R},d})$$

For clarity we will consider $\mathbf{s}$ as a vector with five coordinates (which are in fact vectors over $\mathcal{R}_q$ of size ℓ,μ,1,1 and d). The equation $(\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})^\top \sigma((\mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e}) - \mathbb{1})[0] = 0$ can be rewritten as

$$F_3(\mathbf{s})[0] := \left((\mathbf{s}, \sigma(\mathbf{s}))^\top \cdot \mathbf{E}_3 \cdot (\mathbf{s}, \sigma(\mathbf{s})) + \mathbf{e}_3^\top (\mathbf{s}, \sigma(\mathbf{s}))\right)[0] = 0 \text{ with } \begin{cases} \mathbf{E}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ \mathbf{e}_3 = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix}^\top \end{cases}$$

Since $\mathbf{r}_i = \mathbf{z}_i - cL_{\mathsf{SS},i}\mathbf{s}_i - \sum_{j \in \mathsf{SS}\setminus\{i\}} \left\langle \mathbf{m}^*_{i,j} - \mathbf{m}^*_{j,i}, \vec{\text{pow}} \right\rangle$, and $\mathbf{e}'_i = \mathbf{w}_i - \mathbf{A}\cdot\mathbf{r}_i$, it is clear that the equations $\mathbf{r}_i^\top \cdot \sigma(\mathbf{r}_i) + \beta_\mathbf{r} \cdot \sigma(\text{pow}(B^2_\mathbf{r}))\ [0] = B^2_\mathbf{r}$ and $\mathbf{e}_i'^\top \cdot \sigma(\mathbf{e}'_i) + \beta_\mathbf{e} \cdot \sigma(\text{pow}(B^2_\mathbf{e}))\ [0] = B^2_\mathbf{e}$ can be rewritten as (the constant term in) quadratic equations in $\mathbf{s}$. For completion we make these equations explicit:

$$\mathbf{r}_i^\top \cdot \sigma(\mathbf{r}_i) + \beta_\mathbf{r} \cdot \sigma(\text{pow}(B^2_\mathbf{r}))\ [0] = B^2_\mathbf{r} \Leftrightarrow F_1(\mathbf{s}) := ((\mathbf{s}, \sigma(\mathbf{s}))^\top \cdot \mathbf{E}_1 \cdot (\mathbf{s}, \sigma(\mathbf{s})) + \mathbf{e}_1^\top (\mathbf{s}, \sigma(\mathbf{s})) + e_1)[0] = 0$$
$$\mathbf{e}_i'^\top \cdot \sigma(\mathbf{e}'_i) + \beta_\mathbf{e} \cdot \sigma(\text{pow}(B^2_\mathbf{e}))\ [0] = B^2_\mathbf{e} \Leftrightarrow F_2(\mathbf{s}) := ((\mathbf{s}, \sigma(\mathbf{s}))^\top \cdot \mathbf{E}_2 \cdot (\mathbf{s}, \sigma(\mathbf{s})) + \mathbf{e}_2^\top (\mathbf{s}, \sigma(\mathbf{s})) + e_2)[0] = 0$$

Let $\mathbf{V} := (-cL_{\mathsf{SS},i}\cdot\mathbf{I}_k, -\vec{\text{pow}}^\top \otimes \mathbf{I}_k, \ldots, -\vec{\text{pow}}^\top \otimes \mathbf{I}_k, \vec{\text{pow}}^\top \otimes \mathbf{I}_k, \ldots, \vec{\text{pow}}^\top \otimes \mathbf{I}_k, 0, 0, 0)$, we have $\mathbf{r}_i = \mathbf{V}\cdot\mathbf{s} + \mathbf{z}_i$. From this we get:

$$\begin{cases} \mathbf{E}_1 = \begin{bmatrix} 0 & \mathbf{V}^\top \cdot \sigma(\mathbf{V}) \\ 0 & 0 \end{bmatrix} \\ \mathbf{e}_1^\top = \begin{bmatrix} \sigma(\mathbf{z}_i)^\top \mathbf{V} & \mathbf{z}_i^\top \sigma(\mathbf{V}) \end{bmatrix} + \begin{bmatrix} 0 & 0 & \sigma(\text{pow}(B^2_\mathbf{r})) & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ e_1 = \mathbf{z}_i^\top \sigma(\mathbf{z}_i) \end{cases}$$

Similarly we have $\mathbf{e}'_i = \mathbf{W}\mathbf{s} + \mathbf{w}_i - \mathbf{A}\mathbf{z}_i$ with $\mathbf{W} := -\mathbf{A}\cdot\mathbf{V}$, which gives:

$$\begin{cases} \mathbf{E}_2 = \begin{bmatrix} 0 & \mathbf{W}^\top \cdot \sigma(\mathbf{W}) \\ 0 & 0 \end{bmatrix} \\ \mathbf{e}_2^\top = \begin{bmatrix} \sigma(\mathbf{z}_i)^\top \mathbf{V} & \mathbf{z}_i^\top \sigma(\mathbf{V}) \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & \sigma(\text{pow}(B^2_\mathbf{e})) & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \\ e_2 = (\mathbf{w}_i - \mathbf{A}\mathbf{z}_i)^\top \sigma(\mathbf{w}_i - \mathbf{A}\mathbf{z}_i) \end{cases}$$

Finally we need to add $\vec{z}_\mathbf{R} = \vec{\mathbf{R}} \cdot \vec{\mathbf{s}}_\mathbf{R} + \overrightarrow{(y_{\mathbf{R},1}, \ldots, y_{\mathbf{R},d})}$ to the constraints. As explained before we set $\vec{\rho}_i^\top \in \mathbb{Z}_q^{(2\ell+\mu+2)d}$ as the i^{th} row of $\mathbf{R}$ and define $\rho_i \in \mathcal{R}_q^{(2\ell+\mu+2)}$ the corresponding polynomial vector. The equation F_{3+i} for any $i \in [d]$ can be defined as:

$$\sigma(\rho_i)^\top \cdot \mathbf{s}_\mathbf{R} + y_{\mathbf{R},i}[0] = z_{\mathbf{R},i} \Leftrightarrow F_{3+i}(\mathbf{s}) := (\mathbf{e}_{3+i}^\top(\mathbf{s}, \sigma(\mathbf{s})) + e_{3+i})[0] = 0$$

With:

$$\mathbf{e}_{3+i} := \begin{bmatrix} \sigma(\rho_i)^\top \cdot \begin{bmatrix} & & \mathbf{V} & & \\ & & \mathbf{W} & & \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \\ \begin{matrix} 0 \\ \vdots \\ 0 \\ 1 \\ 0 \\ \vdots \\ 0 \end{matrix} \end{bmatrix}; e_{3+i} = \sigma(\rho_i)^\top \cdot (\mathbf{z}_i, \mathbf{w}_i - \mathbf{A}\mathbf{z}_i, 0 \dots, 0) + z_{\mathbf{R}_i}$$

All the modular constraints are now summarized as $F_i(\mathbf{s})[0] = 0 \mod \mathcal{R}_q$ for $i \in [d+3]$. To prove that the constant coefficient of each $F_i(\mathbf{s})$ is zero, we can sample a random polynomial g_i such that $g_i[0] = 0$, commit to it, then receive a uniform $v_i \in \mathbb{Z}_q$ and output $h_i = g_i + v_i \cdot F_i$ so the verifier can check that its constant coefficient is zero. Such a protocol has soundness $1/q$ so it will have to be repeated $\tau = \log q/\lambda$ times (in fact an optimisation uses the trace function and checks that both constant coefficient and coefficient of degree $d/2$ are zero but we do not include this improvement for simplicity). This protocol can be further optimized by noticing that all equations can be collapsed into one while keeping the same soundness. This protocol is described in lines 11 to 17 of Fig. 14.

Equation aggregation. We now need to prove that all the h_i polynomials are well formed. For each $i \in [\tau]$ the polynomial h_i corresponds to a quadratic equation $F'(\mathbf{s}')$ in $\mathbf{s}' := (\mathbf{s}, g_1, \dots, g_\tau)$. More precisely the coefficients of F'_i are:

$$\begin{cases} \mathbf{E}'_i = \begin{bmatrix} \sum_{j \in [3+d]} v_{i,j} \mathbf{E}_j & 0 \\ 0 & 0 \end{bmatrix} \\ \mathbf{e}_i'^\top = \begin{bmatrix} \sum_{j \in [3+d]} v_{i,j} \mathbf{e}_j \\ 0 \end{bmatrix} \\ e'_i = \sum_{j \in [3+d]} v_{i,j} e_j - h_i \end{cases}$$

We can once again collapse these τ quadratic equations into a single one by receiving τ random polynomials μ_i from the verifier and summing the equations. The soundness error of this protocol is $q^{-d/l}$ which is negligible. This protocol is described in lines 18 to 22 of Fig. 14.Once this is done we compute a single proof for quadratic relations.

Proof for quadratic equation. Finally we are left with proving only one quadratic relation over $\mathcal{R}_q$. To do so, [LNP22] uses a standard Fiat-Shamir with abort proof with an additional commitment to some garbage terms used for the proof. At a high level we want to prove that $\mathbf{s}^\top \mathbf{E}\mathbf{s} + \mathbf{e}^\top \mathbf{s} + e = 0$, in a classic FSwA type proof the prover's response is of the form $\mathbf{z} := c\mathbf{s} + \mathbf{y}$, from which we have:

$$\mathbf{z}^\top \mathbf{E}\mathbf{z} + c \cdot \mathbf{e}^\top \mathbf{z} + c^2 \cdot e = c^2 \cdot (\mathbf{s}^\top \mathbf{E}\mathbf{s} + \mathbf{e}^\top \mathbf{s} + e) + c(\mathbf{s}^\top (\mathbf{E} + \mathbf{E}^\top)\mathbf{y} + \mathbf{e}^\top \mathbf{y}) + \mathbf{y}^\top \mathbf{E}\mathbf{y}$$

Since the term in c^2 evaluates to 0 we can verify this relation by committing to $\mathbf{s}^\top (\mathbf{E} + \mathbf{E}^\top)\mathbf{y} + \mathbf{e}^\top \mathbf{y}$ and $\mathbf{y}^\top \mathbf{E}\mathbf{y}$.

Since we use multiple commitments rather than a single ABDLOP commitment as in [LNP22] our proof is slightly more complicated. The only difference being that we will need to mask the randomness and secret of $\mathbf{v_m}$, as well as the randomness of $\mathbf{t_s}$.This protocol is described in lines 23 to 41 of Fig. 14.

The completeness and simulatability of this protocol are proven in [LNP22]. The proof for soundness is also given in [LNP22] but we cast it in the framework of [AAB+24] in the next section. Notably we show in Lemma B.12 that a break in binding results in solving some MSIS instance.

B.5.5 Soundness of the NIZK.

We use the same approach as [AAB+24] to analyze the soundness of the Zero-knowledge proof presented in the previous section. It is worth noting that we do not require coordinate-wise special soundness but only special soundness, i.e. we use $(r_1, \ldots, r_\mu) = (1, \ldots, 1)$ in the definition of CWSS . We prove that the protocol $\mathrm{LNP}_{\mathbf{z}}$ described in Figs. 14 and 15 is $(\mathbf{K}, \mathbf{\Phi})$ special-sound where $\mathbf{K} = (2, 2, 2, 3)$ and we will describe the challenge predicate Φ^{chal} and the commitment predicate $(\Phi^{\mathsf{prop}}, \Phi^{\mathsf{bind}})$ guaranteed at each of the four rounds of the protocol, backward starting from the last round .

Lemma B.10. *Let $\gamma, \gamma_1, \gamma_2 > 0$ be rejection probabilities. Let $\nu > 0$, $t > 1.64$, $\kappa_{\mathbf{A}} > 0$, $\kappa_{\mathrm{MSIS}} > 0$, let $\chi_A = \chi_B = D_\nu$, let $\mathfrak{s} = \gamma\sqrt{C_1 \cdot (B_{\mathbf{r}}^2 + B_{\mathbf{e}}^2 + (\mu+2)d)}$, $\mathfrak{s}_1 = \gamma_2 T_{op}\sqrt{(\mu+2)d}$, $\mathfrak{s}_2 = \gamma_2 T_{op}\nu\sqrt{2\kappa_{\mathbf{A}} d}$. For $B_R \geqslant t\mathfrak{s}\sqrt{d/C_2}$, $B_{\mathbf{z}} \geqslant \sqrt{{\mathfrak{s}_1}^2\mu d + 2{\mathfrak{s}_2}^2\kappa_{\mathbf{A}} d}$, $q > \max\{{B_R}^2 + \sqrt{(2\ell+\mu+2)d}B_R, 41d(2\ell+\mu+2)\sqrt{C_2}B_R, {B_R}^2 + 2B_{\mathbf{r}}^2, {B_R}^2 + 2B_{\mathbf{e}}^2\}$.*

$LNP_{\mathbf{z}}$ with parameters $\mathsf{pp} = (\mathbf{A}_1, \mathbf{A}_2)$ is complete and zero-knowledge under $\mathsf{MLWE}_{\kappa_{\mathbf{A}}-(\kappa_{\mathrm{MSIS}}+2+d+\tau),\nu}$. It is also $(\mathbf{R}, \mathbf{K}, \mathbf{\Phi})$-coordinate-wise predicate-special-sound, with $\mathbf{R} = (1, 1, 1, 1)$, $\mathbf{K} = (2, 2, 2, 3)$ and $\mathbf{\Phi}$ consisting of predicates $\Phi_{4,1}^{\mathsf{chal}}, \Phi_{4,2}^{\mathsf{chal}}, \Phi_4^{\mathsf{bind}}, \Phi_1^{\mathsf{prop}}, \ldots, \Phi_4^{\mathsf{prop}}$ with failure densities $p_{4,1}^{\mathsf{chal}} = p_{4,2}^{\mathsf{chal}} = lB$, $p_1^{\mathsf{com}} = 2^{-d/2}$, $p_2^{\mathsf{com}} = q^{-\tau}$, $p_3^{\mathsf{com}} = q^{-d/l}$, and $p_4^{\mathsf{com}} = 2lB$, respectively. Further, the predicate system $\mathbf{\Phi}$ admits binding relation

$$\mathsf{R}_{\mathsf{msis}}^{LNP_{\mathbf{z}}} = \mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}}, \ell \log q + \kappa_{\mathbf{A}}, [\mathbf{A}_1, \mathbf{A}_2], 8T_{op}B_{\mathbf{z}}} \cup \mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}}, \kappa_{\mathbf{A}}, \mathbf{A}_2, 8T_{op}B_{\mathbf{z}}}$$

Proof. We do not prove completeness and zero-knowledge as they are identical to the proof of [LNP22]. We prove soundness in the framework of [AAB+24] in the rest of this section. □

Protocol for quadratic equations We define Φ_4 and prove that any transcript tree of type $(1, 1, 1, 3)$ satisfies Φ_4 with negligible failure density.

For $i \in [3]$, let $c^{(i)}$ and $(\mathbf{z}_{\mathbf{m},1}^{(i)}, \mathbf{z}_{\mathbf{m}_2}^{(i)}, \mathbf{z}_{\mathbf{u}}^{(i)})$ be the challenges and responses corresponding to the three transcripts.

$$\Phi_{4,1}^{\mathsf{chal}}(c^{(1)}) = 1 \Leftrightarrow c^{(1)} \in \mathcal{R}_q^\times$$
$$\Phi_{4,2}^{\mathsf{chal}}(c^{(1)}, c^{(2)}) = 1 \Leftrightarrow c^{(2)} - c^{(1)} \in \mathcal{R}_q^\times$$

Let $\bar{c} := c^{(2)} - c^{(1)}$, and $\bar{\mathbf{z}}_{\mathbf{x}} := \mathbf{z}_{\mathbf{x}}^{(2)} - \mathbf{z}_{\mathbf{x}}^{(1)}$ for $\mathbf{x} \in \{\{\mathbf{m}, 1\}, \{\mathbf{m}, 2\}, \{\mathbf{u}\}\}$. For $i \in [3]$ and $\mathbf{x} \in \{\{\mathbf{m}, 1\}, \{\mathbf{m}, 2\}, \{\mathbf{u}\}\}$, set $\mathbf{y}_{\mathbf{x}}^{(i)} := \mathbf{z}_{\mathbf{x}}^{(i)} - \bar{\mathbf{z}}_{\mathbf{x}} \cdot c^{(i)}/\bar{c}$. We define the extraction algorithm for this output: $E_4(i) = (\mathbf{y}_{\mathbf{m},1}^{(i)}, \mathbf{y}_{\mathbf{m},2}^{(i)}, \mathbf{y}_{\mathbf{u}}^{(i)})$, the binding property of the protocol states that this extraction's output should be independent of the challenge:

$$\Phi_4^{\mathsf{bind}}(t) = 1 \Leftrightarrow E_4(1) = E_4(2) = E_4(3)$$

Let $\mathbf{m}^{*(1)} := (\mathbf{z}_{\mathbf{m},1}^{(1)} - \mathbf{y}_{\mathbf{m},1}^{(1)})/c^{(1)}$, $\mathbf{u}_{\mathbf{m}}{}^{(1)} := (\mathbf{z}_{\mathbf{m},2}^{(1)} - \mathbf{y}_{\mathbf{m},2}^{(1)})/c^{(1)}$, $\mathbf{u}^{(1)} := (\mathbf{z}_{\mathbf{u}}^{(1)} - \mathbf{y}_{\mathbf{u}}^{(1)})/c^{(1)}$. We can now define the extracted values of the main commitment:

$$\mathbf{s}^{(1)} := \begin{bmatrix} \mathbf{s}_i^{(1)} \\ \mathbf{m}^{*(1)} \\ \beta_{\mathbf{r}}^{(1)} \\ \beta_{\mathbf{e}}^{(1)} \\ \mathbf{y}_{\mathbf{R}}^{(1)} \\ g_1^{(1)} \\ \vdots \\ g_\tau^{(1)} \end{bmatrix} := \begin{bmatrix} \mathbf{t}_{\mathbf{s}_i} \\ \mathbf{m}^{*(1)} \\ t_{\beta_{\mathbf{r}}} \\ t_{\beta_{\mathbf{e}}} \\ \mathbf{t}_{\mathbf{y}_{\mathbf{R}}} \\ t_{g_1} \\ \vdots \\ t_{g_\tau} \end{bmatrix} - \begin{bmatrix} \mathbf{B}_{\mathbf{s}_i} \\ 0 \\ \mathbf{B}_{\beta_{\mathbf{r}}} \\ \mathbf{B}_{\beta_{\mathbf{e}}} \\ \mathbf{B}_{\mathbf{y}_{\mathbf{R}}} \\ \mathbf{b}_{g_1}^\top \\ \vdots \\ \mathbf{b}_{g_\tau}^\top \end{bmatrix} \cdot \mathbf{u}^{(1)}$$

Observe that if $\Phi_4^{\mathsf{bind}} = 1$ then we could have defined $\mathbf{s}^{(2)}$ or $\mathbf{s}^{(3)}$ in the same manner and obtained the same result, i.e. $\mathbf{s}^{(1)} = \mathbf{s}^{(2)} = \mathbf{s}^{(3)}$. Since all relevant commitments have been extracted in this step, the following properties will simply consist in checking that the extracted values verify the relevant functions.

The relation property to be verified for this step is:

$$\Phi_4^{\mathsf{prop}}(t) = 1 \Leftrightarrow (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)}))^\top \mathbf{E}(\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + \mathbf{e}^\top(\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + e = 0$$

Lemma B.11. *The predicates $\Phi_{4,1}^{\mathsf{chal}}$ and $\Phi_{4,2}^{\mathsf{chal}}$ have failure density $p_{4,1}^{\mathsf{chal}} = p_{4,2}^{\mathsf{chal}} = lB$.*

Proof. As the proof is identical to Lemma I.1 of [AAB+24] we omit it. □

Lemma B.12. *If a tree $t \in (1,1,1,3)$ does not verify $\Phi_4^{\mathsf{bind}}(t)$ then there exists an extractor which outputs a solution to either $\mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}}, \ell \log q + \kappa_{\mathbf{A}}, [\mathbf{A}_1, \mathbf{A}_2], 8T_{op}B_{\mathbf{z}}}$ or $\mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}}, \kappa_{\mathbf{A}}, \mathbf{A}_2, 8T_{op}B_{\mathbf{z}}}$ from t.*

Proof. By construction we have $E_4(1) = E_4(2)$. We assume that $E_4(1) \neq E_4(3)$, which implies that $\mathbf{y}_{\mathbf{m},1}^{(1)} \neq \mathbf{y}_{\mathbf{m},1}^{(3)}$, or $\mathbf{y}_{\mathbf{m},2}^{(1)} \neq \mathbf{y}_{\mathbf{m},2}^{(3)}$, or $\mathbf{y}_{\mathbf{u}}^{(1)} \neq \mathbf{y}_{\mathbf{u}}^{(3)}$.

Case 1: $\mathbf{y}_{\mathbf{m},1}^{(1)} \neq \mathbf{y}_{\mathbf{m},1}^{(3)}$, or $\mathbf{y}_{\mathbf{m},2}^{(1)} \neq \mathbf{y}_{\mathbf{m},2}^{(3)}$. From the verification equations of transcripts 1 and 2 we obtain the following equality:

$$(\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1) \cdot \bar{\mathbf{z}}_{\mathbf{m},1} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2) \cdot \bar{\mathbf{z}}_{\mathbf{m},2} = \bar{c}\mathbf{v}_m$$

Since $\mathbf{y}_{\mathbf{m},1}^{(i)} := \mathbf{z}_{\mathbf{m},1}^{(i)} + \bar{\mathbf{z}}_{\mathbf{m},1} \cdot c^{(i)}/\bar{c}$ for $i \in [3]$ we have:

$$\begin{aligned}
&(\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1) \cdot (\mathbf{y}_{\mathbf{m},1}^{(3)} - \mathbf{y}_{\mathbf{m},1}^{(1)}) + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2) \cdot (\mathbf{y}_{\mathbf{m},2}^{(3)} - \mathbf{y}_{\mathbf{m},2}^{(1)}) \\
&\quad = (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1) \cdot (\mathbf{z}_{\mathbf{m},1}^{(3)} - \mathbf{z}_{\mathbf{m},1}^{(1)}) + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2) \cdot (\mathbf{z}_{\mathbf{m},2}^{(3)} - \mathbf{z}_{\mathbf{m},2}^{(1)}) \\
&\quad\quad - \frac{c^{(3)} - c^{(1)}}{\bar{c}} ((\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1) \cdot \bar{\mathbf{z}}_{\mathbf{m},1} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2) \cdot \bar{\mathbf{z}}_{\mathbf{m},2}) \\
&\quad = c^{(3)}\mathbf{v}_{\mathbf{m}} - c^{(1)}\mathbf{v}_{\mathbf{m}} - \frac{c^{(3)} - c^{(1)}}{\bar{c}} \cdot \bar{c}\mathbf{v}_m \\
&\quad = 0
\end{aligned}$$

If $\mathbf{y}_{\mathbf{m},1}^{(1)} \neq \mathbf{y}_{\mathbf{m},1}^{(3)}$ (the same argument applies if $\mathbf{y}_{\mathbf{m},2}^{(1)} \neq \mathbf{y}_{\mathbf{m},2}^{(3)}$) then there exists an index $j \in \mathsf{SS} \backslash i$ such that $\mathbf{y}_{\mathbf{m}_{i,j}}^{(3)} \neq \mathbf{y}_{\mathbf{m}_{i,j}}^{(1)}$, and $\bar{c}(\mathbf{y}_{\mathbf{m}_{i,j}}^{(3)} - \mathbf{y}_{\mathbf{m}_{i,j}}^{(1)}, \mathbf{y}_{\mathbf{m},2}^{(3)} - \mathbf{y}_{\mathbf{m},2}^{(1)})$ is a solution for the MSIS instance with matrix $[\mathbf{A}_1, \mathbf{A}_2]$. By substituting the value of $\mathbf{y}_{\mathbf{m},1}$ and $\mathbf{y}_{\mathbf{m},2}$ we get the following bound:

$$\begin{aligned}
\|\bar{c}(\mathbf{y}_{\mathbf{m}_{i,j}}^{(3)} - \mathbf{y}_{\mathbf{m}_{i,j}}^{(1)}, \mathbf{y}_{\mathbf{m},2}^{(3)} - \mathbf{y}_{\mathbf{m},2}^{(1)})\|_2 &\leqslant 2\|\bar{c}(\mathbf{y}_{\mathbf{m}_{i,j}}^{(3)}, \mathbf{y}_{\mathbf{m},2}^{(3)})\|_2 \\
&\leqslant 2\|\bar{c}(\mathbf{y}_{\mathbf{m},1}^{(3)}, \mathbf{y}_{\mathbf{m},2}^{(3)})\|_2 \\
&\leqslant 2\|\bar{c}(\mathbf{z}_{\mathbf{m},1}^{(3)}, \mathbf{z}_{\mathbf{m},2}^{(3)}) + c^{(1)}(\bar{\mathbf{z}}_{\mathbf{m},1}, \bar{\mathbf{z}}_{\mathbf{m},2})\|_2 \\
&\leqslant 4T_{op}\|(\mathbf{z}_{\mathbf{m},1}^{(3)}, \mathbf{z}_{\mathbf{m},2}^{(3)})\|_2 + 2T_{op}\|(\bar{\mathbf{z}}_{\mathbf{m},1}, \bar{\mathbf{z}}_{\mathbf{m},2})\|_2 \\
&\leqslant 8T_{op}B_{\mathbf{z}}
\end{aligned}$$

Case 2: $\mathbf{y}_{\mathbf{u}}^{(1)} \neq \mathbf{y}_{\mathbf{u}}^{(3)}$. The proof is identical but only uses the matrix $\mathbf{A}_2$ that is used for BDLOP commitments. We have:

$$\mathbf{A}_2\bar{\mathbf{z}}_{\mathbf{u}} = \bar{c}\mathbf{t}_{\mathbf{A}}$$

Since $\mathbf{y}_{\mathbf{u}}^{(i)} := \mathbf{z}_{\mathbf{u}}^{(i)} + \bar{\mathbf{z}}_{\mathbf{u}} \cdot c^{(i)}/\bar{c}$ for $i \in [3]$ we have:

$$\begin{aligned}
\mathbf{A}_2 \cdot (\mathbf{y}_{\mathbf{u}}^{(3)} - \mathbf{y}_{\mathbf{u}}^{(1)}) &= \mathbf{A}_2 \cdot (\mathbf{z}_{\mathbf{u}}^{(3)} - \mathbf{z}_{\mathbf{u}}^{(1)}) - \frac{c^{(3)} - c^{(1)}}{\bar{c}} \mathbf{A}_2\bar{\mathbf{z}}_{\mathbf{u}} \\
&= 0
\end{aligned}$$

And $\bar{c}(\mathbf{y}_{\mathbf{u}}^{(3)} - \mathbf{y}_{\mathbf{u}}^{(1)})$ is a solution for the MSIS instance with matrix $\mathbf{A}_2$ and norm $8T_{op}B_{\mathbf{z}}$.

□

Lemma B.13. *The predicate Φ_4^{prop} has failure density $p_4^{\mathsf{com}} \leqslant 2lB$.*

Proof. We prove that the predicate Φ_4^{prop} is directly implied by $\Phi_{4,1}^{\mathsf{chal}} \wedge \Phi_{4,2}^{\mathsf{chal}}$ which gives the desired bound. Assume that both $\Phi_{4,1}^{\mathsf{chal}}$ and $\Phi_{4,2}^{\mathsf{chal}}$ are true as well as Φ_4^{bind}, then we can verify that $\mathbf{z}^{(i)} = c^{(i)}\mathbf{s}^{(1)} + \mathbf{y}^{(1)}$. Substituting in the verification equation $(\mathbf{z}, \sigma(\mathbf{z}))^\top \cdot \mathbf{E} \cdot (\mathbf{z}, \sigma(\mathbf{z})) + c\mathbf{e}^\top \cdot (\mathbf{z}, \sigma(\mathbf{z})) + c^2 e - f = v$ gives:

$$ {c^{(i)}}^2 \left((\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)}))^\top \cdot \mathbf{E} \cdot (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + \mathbf{e}^\top \cdot (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + e \right) + c^{(i)} g_1' + g_0' = 0 $$

for all $i \in [3]$, where $g_1' := (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)}))^\top \cdot (\mathbf{E} + \mathbf{E}^\top) \cdot (\mathbf{y}^{(1)}, \sigma(\mathbf{y}^{(1)})) + \mathbf{e}^\top \cdot (\mathbf{y}^{(1)}, \sigma(\mathbf{y}^{(1)}))$ and $g_0' := (\mathbf{y}^{(1)}, \sigma(\mathbf{y}^{(1)}))^\top \cdot \mathbf{E} \cdot (\mathbf{y}^{(1)}, \sigma(\mathbf{y}^{(1)})) - v$. From this we have:

$$ \begin{bmatrix} 1 & c^{(1)} & {c^{(1)}}^2 \\ 1 & c^{(2)} & {c^{(2)}}^2 \\ 1 & c^{(3)} & {c^{(3)}}^2 \end{bmatrix} \cdot \begin{bmatrix} g_0' \\ g_1' \\ (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)}))^\top \cdot \mathbf{E} \cdot (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + \mathbf{e}^\top \cdot (\mathbf{s}^{(1)}, \sigma(\mathbf{s}^{(1)})) + e \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} $$

Since $\Phi_{4,1}^{\mathsf{chal}}$ and $\Phi_{4,2}^{\mathsf{chal}}$ are true the determinant of this Vandermonde matrix is invertible and the property Φ_4^{prop} is verified. □

Protocol for aggregation. From the extraction of $E_4(1)$ we can set the vector $\mathbf{s} := \mathbf{s}^{(1)}$ as defined before, and under the assumption $\Phi_4^{\mathsf{bind}}(t_1) = 1$ we have that $F(\mathbf{s}) = 0$. We can set the properties verified by the transcript as:

$$ \Phi_3^{\mathsf{prop}}(t) = 1 \Leftrightarrow \forall i \in [\tau]\ F_i'(\mathbf{s}) = 0 $$

We do not require any explicit binding or challenge predicates (i.e. $\Phi_3^{\mathsf{bind}}(t) := 1$ and $\Phi_3^{\mathsf{chal}}(t) := 1$).

Lemma B.14. *The predicate Φ_3^{prop} has failure density $p_3^{\mathsf{com}} \leqslant q^{-d/l}$.*

Proof. Let $t = (t_1, t_2)$ be a $(1, 1, 2, 3)$ tree of transcripts. The argument is quasi-identical to Lemma I.2 of [AAB+24] but we present it with our notations. Assume $\Phi_3^{\mathsf{prop}}(t) = 0$, then there exists some $j \in [\tau]$ such that $F_j'(\mathbf{s}) \neq 0$, from the extraction of the last round and the verification algorithm we have $\sum_{i\in[\tau]} \mu_i^{(1)} F_i'(\mathbf{s}) = \sum_{i\in[\tau]} \mu_i^{(2)} F_i'(\mathbf{s}) = 0$. For any pair $((\mu_i)_{i\in[\tau]}, (\mu_i')_{i\in[\tau]})$ we have:

$$ (\mu_j - \mu_j') F_j'(\mathbf{s}) = f((\mu_i)_{i\neq j}, (\mu_i')_{i\neq j}, \mathbf{s}) $$

where f is the sum of the remaining terms. Let $\mathcal{R}_q/P(X)$ be a CRT slot in which $F_j'(\mathbf{s}) \mod P \neq 0$, since this CRT slot is a field of size $q^{d/l}$, $(F_j'(\mathbf{s}) \mod P)^{-1}$ exists and we have that $\mu_j - \mu_j' \mod P$ is uniquely defined. Hence there exist exactly $q^{d/l}$ pairs $(\mu_j \mod P, \mu_j' \mod P)$ such that $(\mu_j - \mu_j')F_j'(\mathbf{s}) = f((\mu_i)_{i\neq j}, (\mu_i')_{i\neq j}, \mathbf{s}) \mod P$, over a total number of pairs of $q^{2d/l}$. From this we get that the failure density of Φ_3^{prop} is at most $q^{d/l}/q^{2d/l} = q^{-d/l}$. □

Protocol for constant coefficient. With $\mathbf{s}$ defined as before, the property verified at the end of this protocol should be that all input functions $(F_i)_{i\in[3]}$ should have a vanishing constant coefficient in $\mathbf{s}$,i.e.

$$ \Phi_2^{\mathsf{prop}}(t) = 1 \Leftrightarrow \forall i \in [3 + d]\ F_i(\mathbf{s})[0] = 0 $$

Once again we do not need an extra predicate on the challenges, nor the binding as all the binding was proven in the last step of the protocol (i.e. $\Phi_3^{\mathsf{bind}}(t) := 1$ and $\Phi_3^{\mathsf{chal}}(t) := 1$).

Lemma B.15. *The predicate Φ_2^{prop} has failure density $p_2^{\mathsf{com}} \leqslant q^{-\tau}$.*

Proof. Let $t = (t_1, t_2)$ be a $(1, 2, 2, 3)$ tree of transcripts. Suppose $\Phi_2^{\mathsf{prop}}(t) = 0$ then there is $j^* \in [3 + d]$ such that $F_{j^*}(\mathbf{s})[0] \neq 0$. For any $i \in [\tau]$ we have $h_i^{(1)} - h_i^{(2)} = \sum_{j\in[3+d]} (v_{i,j}^{(1)} - v_{i,j}^{(2)}) F_j(\mathbf{s})$. Since q is prime we get

$$ v_{i,j^*}^{(1)} - v_{i,j^*}^{(2)} = f((v_{i,j}^{(1)}, v_{i,j}^{(2)}, F_j(\mathbf{s}))_{j\neq j^*}) $$

and there exist exactly q values for the pair $(v_{i,j^*}^{(1)}, v_{i,j^*}^{(2)})$, over a total set of size q^2, that satisfy this equation. By repeating the same argument for all $i \in [\tau]$ we obtain a failure density of $q^{-\tau}$. □

Protocol for approximate norm proofs Finally at the top level we enforce that the extracted witness is short. For $\mathbf{s_R} = (\mathbf{s}_i, \mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})$. We have:

$$\Phi_1^{\mathsf{prop}}(t) = 1 \Leftrightarrow \|\mathbf{s_R}\|_2 \leqslant B_R$$

Lemma B.16. *The predicate Φ_1^{prop} has failure density $p_1^{\mathsf{com}} \leqslant 2^{-d/2}$.*

Proof. Since equations $F_i(\mathbf{s})$ for $3 < i \leqslant 3+d$ correspond to the equation $\vec{z}_\mathbf{R} = \vec{\mathbf{R}} \cdot \vec{\mathbf{s}}_\mathbf{R} + \vec{\mathbf{y}}_\mathbf{R}$, we know that this equation is true over $\mathbb{Z}_q$ for the extracted $\mathbf{s}$. Since $\Phi_4^{\mathsf{bind}} = 1$, the commitments $(\mathbf{t_A}, \mathbf{t}_{\mathbf{s}_i}, t_{\beta_\mathbf{r}}, t_{\beta_\mathbf{e}}, \mathbf{t}_{y_\mathbf{R}})$ are binding, the size condition on q given in Lemma B.10 correspond to the constraints of Lemma B.9. Suppose that $\Phi_1^{\mathsf{prop}} = 0$ then :

$$\Pr[\|\vec{\mathbf{R}}\vec{\mathbf{s}}_\mathbf{R} + \vec{\mathbf{y}}_\mathbf{R}\|_2 < \sqrt{C_2} B_R] \leqslant 2^{-d/2}$$

which gives the desired failure density. □

Extracting the final witness. In the previous section we have shown that for appropriate parameters we can extract witnesses such that $\Phi_i^{\mathsf{prop}} = 1$ for $i \in [4]$ we now collect these extractions to reconstruct a witness $(\mathbf{s}_i, \mathbf{r}_i, \mathbf{e}'_i, \mathbf{m}_{i,j}, \mathbf{m}_{j,i})$ for $\mathsf{R_z}$.

From Φ_1^{prop} and Φ_2^{prop} we extract $\mathbf{s_R} = (\mathbf{r}_i, \mathbf{e}'_i, \mathbf{m}^*, \beta_\mathbf{r}, \beta_\mathbf{e})$ with norm smaller than B_R that verifies all the necessary relations and equations modulo q, we need to prove that the relevant equations are verified in $\mathbb{Z}$. We know that $\|\mathbf{r}_i\|_2 \leqslant B_R$ and $\|\beta_\mathbf{r}\|_2 \leqslant B_R$. From the second equality we have:

$$\begin{aligned}\left\langle \vec{\beta}_\mathbf{r}, \vec{\beta}_\mathbf{r} - \mathbb{1} \right\rangle &\leqslant \|\beta_\mathbf{r}\|_2 \|\beta_\mathbf{r} - \mathbb{1}\|_2 \\ &\leqslant B_R(B_R + \sqrt{d}) \\ &< q\end{aligned}$$

where the last inequality comes from the condition $q > {B_R}^2 + \sqrt{(2\ell + \mu + 2)d} B_R$ of Lemma B.10. Since this equation now stands over the integers we can deduce from Lemma B.8 that $\beta_\mathbf{r}$ is binary even without the modulo q . From this we get:

$$\begin{aligned}\|\mathbf{r}_i\|^2 + \left\langle \vec{\beta}_\mathbf{r}, \vec{\mathrm{pow}}(B_\mathbf{r}^2) \right\rangle &\leqslant {B_R}^2 + \sum_{i \leqslant \lfloor \log B_\mathbf{r}^2 \rfloor} \beta_{\mathbf{r}_i} 2^i \\ &\leqslant {B_R}^2 + 2B_\mathbf{r}^2 - 1 \\ &< q\end{aligned}$$

where the last inequality comes from the condition $q > {B_R}^2 + 2B_\mathbf{r}^2$ of Lemma B.6. Hence the norm inequality is also true without the modulo q which gives $\|\mathbf{r}_i\| \leqslant B_\mathbf{r}$. By applying the same argument to $\mathbf{e}'_i$, $\beta_\mathbf{e}$, and $\mathbf{m}^*$ we prove that $\|\mathbf{e}'_i\|_2 \leqslant B_\mathbf{e}$ and that all $\mathbf{m}^*_{i,j}$ and $\mathbf{m}^*_{j,i}$ are binary. Combined with the relations F_i this gives a witness that satisfies $\mathsf{R_z}$.

B.6 Putting Everything Together

Finally, let us construct our ZK-SNARK for relation $\mathsf{R_z}$. On a high-level, we prove the relation $\mathsf{R_z}$ with $\mathsf{FS}[\mathrm{LNP}_\mathsf{z}]$ to obtain π. To compress the proof π, we then prove knowledge of some π that satisfies the verification equations in $\mathsf{FS}[\mathrm{LNP}_\mathsf{z}]$ via a recursively-composed variant of FS[LaBRADOR]. Unfortunately, this strategy fails as π contains challenges output by random oracles. Instead, we first compress each flow of LNP_z via Ajtai commitments $\mathbf{D}_i$. That is, the prover sends a commitment $\mathbf{D}_i$ to the prover's message m_i instead of m_i in plain. In the last flow, the openings to $\mathbf{D}_i$ together with the responses $\mathbf{z}$ are sent. Verification is as before, except that also the verifier checks that $\mathbf{D}_i$ is a commitment it m_i. We denote this modified protocol by $\mathrm{LNP}_\mathsf{z}^{\mathsf{com}}$.

Then, we compose $\mathrm{LNP}_\mathsf{z}^{\mathsf{com}}$ with (recursively composed) LaBRADOR to prove knowledge of a $\mathrm{LNP}_\mathsf{z}^{\mathsf{com}}$ response that verifies. Let us call this protocol LNPLab. The final SNARK is given by FS[LNPLab]. As PSS-sound protocols compose nicely, the protocol FS[LNPLab] is multi-extraction sound by Lemmata B.3 and B.4. Further, as $\mathsf{FS}[\mathrm{LNP}_\mathsf{z}^{\mathsf{com}}]$ is zero-knowledge, the zero-knowledge property of FS[LNPLab] follows immediately. We elaborate below.

B.6.1 $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$: A Composition-friendly $\mathsf{LNP}_{\mathsf{z}}$

Before fusing $\mathsf{LNP}_{\mathsf{z}}$ and LaBRADOR, let us provide more details on $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ sketched above. First, let us express the verification for $\mathsf{LNP}_{\mathsf{z}}$ (cf. Fig. 15) as a LaBRADOR relation

$$\begin{aligned}\mathsf{R}_{\mathsf{LNP}_{\mathsf{z}}} := \Big\{(\mathbb{x},\mathbb{w}) \;\mid\; \forall i\in[3],\; f_i(\mathbf{z}_{\mathbf{m},1},\mathbf{z}_{\mathbf{m},2},\mathbf{z}_{\mathbf{u}})=0 \;\wedge\; \|(\mathbf{z}_{\mathbf{m},1},\mathbf{z}_{\mathbf{m},2},\mathbf{z}_{\mathbf{u}})\|_2 \leqslant \beta \Big\},\\ \mathbb{x} = (\mathbf{t}_{\mathbf{s}_i}, t_{\beta_{\mathbf{r}}}, t_{\beta_{\mathbf{e}}}, \mathbf{t}_{\mathbf{y_R}}, t_{g_1},\ldots,t_{g_\tau}, \mathbf{w}_{\mathbf{m},1}, \mathbf{w}_{\mathbf{m},2}, \mathbf{w}_{\mathbf{u}}, \mathbf{E}, \mathbf{e}, e)\\ \mathbb{w} = (\mathbf{z}_{\mathbf{m},1}, \mathbf{z}_{\mathbf{m},2}, \mathbf{z}_{\mathbf{u}}).\end{aligned}$$

Where f_1, f_2, f_3 correspond to:

$$\begin{aligned}&c\mathbf{v}_{\mathbf{m}} + \mathbf{w}_{\mathbf{m},2} = (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_1)\cdot \mathbf{z}_{\mathbf{m},1} + (\mathbf{I}_{2(T-1)} \otimes \mathbf{A}_2)\cdot \mathbf{z}_{\mathbf{m},2})\\ &c\mathbf{t}_{\mathbf{A}} + \mathbf{w}_{\mathbf{u}} = \mathbf{A}_2\mathbf{z}_{\mathbf{u}}\\ &(\mathbf{z},\sigma(\mathbf{z}))^\top \cdot \mathbf{E}\cdot(\mathbf{z},\sigma(\mathbf{z})) + c\mathbf{e}^\top\cdot(\mathbf{z},\sigma(\mathbf{z})) + c^2 e - f = v\end{aligned}$$

With $\mathbf{z}, f, v$ defined as in Fig. 15, and $\beta = B_{\mathbf{z}}$. While LaBRADOR can be used to prove knowledge of this relation, the resulting proof will not be small since the statement (and especially $\mathbf{w}_{\mathbf{m}}$) is not small. To circumvent this issue the prover can add an extra layer of commitment on all the communication that is too large. If we set $\mathsf{C}_{\mathrm{LaB}}$ to be the compressing, non-hiding commitment used in LaBRADOR (which is simply an Ajtai commitment without the matrix $\mathbf{A}_2$ and randomness $\mathbf{u}$, where the input message is binary decomposed before being committed to), the prover will compute $\mathbf{x}_{\mathbf{A}} := \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(\mathbf{t}_{\mathbf{A}}, \mathbf{t}_{\mathbf{s}_i}, t_{\beta_{\mathbf{r}}}, t_{\beta_{\mathbf{e}}}, \mathbf{t}_{\mathbf{y_R}})$, $\mathbf{x}_h := \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(h_1,\ldots,h_\tau)$, and $\mathbf{x}_{\mathbf{w}} := \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(\mathbf{w}_{\mathbf{m}}, \mathbf{w}_{\mathbf{u}}, t_g, v)$ and send these commitments instead of their respective messages (the rest of the messages output by the prover are already small). The relation $\mathsf{LNP}_{\mathsf{z}}$ can then be rewritten to take $\mathbf{x}_{\mathbf{A}}, \mathbf{x}_h$ and $\mathbf{x}_{\mathbf{w}}$ as part of the statement and their messages as part of the witness. It should also check that all commitments are well formed, which reduces to proving that the messages are binary and verify the commitment equation, which in turn can be rewritten as equations over $\mathcal{R}_q$ and norm checks. After these modifications, we obtain the protocol $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ with verification relation

$$\begin{aligned}\mathsf{R}^{\mathsf{com}}_{\mathsf{LNP}_{\mathsf{z}}} := \Big\{(\mathbb{x},\mathbb{w}) \;\mid\; &\forall i\in[3],\; f_i(\mathbf{z}_{\mathbf{m},1},\mathbf{z}_{\mathbf{m},2},\mathbf{z}_{\mathbf{u}})=0 \;\wedge\; \|(\mathbf{z}_{\mathbf{m},1},\mathbf{z}_{\mathbf{m},2},\mathbf{z}_{\mathbf{u}})\|_2 \leqslant \beta\\ &\mathbf{x}_{\mathbf{A}} := \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(\mathbf{t}_{\mathbf{A}}, \mathbf{t}_{\mathbf{s}_i}, t_{\beta_{\mathbf{r}}}, t_{\beta_{\mathbf{e}}}, \mathbf{t}_{\mathbf{y_R}}) \;\wedge\\ &\mathbf{x}_h = \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(h_1,\ldots,h_\tau) \;\wedge\; \mathbf{x}_{\mathbf{w}} = \mathsf{C}_{\mathrm{LaB}}.\mathsf{Com}(\mathbf{w}_{\mathbf{m}}, \mathbf{w}_{\mathbf{u}}, t_g, v) \Big\},\end{aligned} \tag{14}$$

$$\begin{aligned}\mathbb{x} &= (\mathbf{x}_{\mathbf{A}}, \mathbf{x}_h, \mathbf{x}_{\mathbf{w}}, \mathbf{E}, \mathbf{e}, e)\\ \mathbb{w} &= (\mathbf{t}_{\mathbf{A}}, \mathbf{t}_{\mathbf{s}_i}, t_{\beta_{\mathbf{r}}}, t_{\beta_{\mathbf{e}}}, \mathbf{t}_{\mathbf{y_R}}, t_{g_1},\ldots,t_{g_\tau}, \mathbf{w}_{\mathbf{m},1}, \mathbf{w}_{\mathbf{m},2}, \mathbf{w}_{\mathbf{u}}, \mathbf{z}_{\mathbf{m},1}, \mathbf{z}_{\mathbf{m},2}, \mathbf{z}_{\mathbf{u}}).\end{aligned}$$

While now the witness is much larger, the statement itself is compact. As LaBRADOR is succinct, this yields a more compact proof size.

Security Correctness of the interactive proof $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ follows from correctness of $\mathsf{LNP}_{\mathsf{z}}$ and $\mathsf{C}_{\mathrm{LaB}}$. For soundness, observe that if during extraction the $\mathsf{C}_{\mathrm{LaB}}$ commitments are never opened to distinct values, then the proof in Section B.5.5 applies verbatim. To ensure that this is indeed guaranteed, we simply add an additional commitment predicate per level that ensures the desired consistency. As we use the commitments $\mathsf{C}_{\mathrm{LaB}}$ from LaBRADOR, the binding relation of the obtained predicate system also includes the relation $\mathsf{R}_{\mathsf{msis}}$ from LaBRADOR (cf. Lemma B.5). Note that while the condition verified by the witness of the $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ relation is $\|(\mathbf{z}_{\mathbf{m},1},\mathbf{z}_{\mathbf{m},2},\mathbf{z}_{\mathbf{u}})\|_2 \leqslant \beta(= B_{\mathbf{z}})$, the witness extracted by the predicate Φ_1^{prop} of LaBRADOR has a slack $\sqrt{\lambda/C_2}$. This does not impact the extraction of $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ as it is an exact proof, it does however impact the MSIS instance corresponding to its binding relation. The relation $\mathsf{R}_{\mathsf{msis}}^{\mathsf{LNP}_{\mathsf{z}}}$ defined in Lemma B.10 will have an added slack, becoming:

$$\mathsf{R}_{\mathsf{msis}}^{\mathsf{LNP}_{\mathsf{z}}} = \mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}},\ell\log q+\kappa_{\mathbf{A}},[\mathbf{A}_1,\mathbf{A}_2],8T_{op}\sqrt{\lambda/C_2}B_{\mathbf{z}}} \cup \mathsf{R}_{\mathsf{msis}}^{\kappa_{\mathrm{MSIS}},\kappa_{\mathbf{A}},\mathbf{A}_2,8T_{op}\sqrt{\lambda/C_2}B_{\mathbf{z}}}$$

The impact of this slack is only marginal.

B.6.2 Composing $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ with LaBRADOR

Let us finally describe our ZK-SNARK based on the interactive protocols $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ and LaBRADOR. The interactive proof LNPLab is defined as follows. First, the prover and verifier run $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ up until the prover outputs its final $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ message $\mathbb{w}$ (as specified in Eq. (14)). Instead of sending $\mathbb{w}$, the prover and verifier run LaBRADOR to prove knowledge of $\mathbb{w} \in \mathsf{R}_{\mathrm{LNP}_{\mathsf{z}}}^{\mathsf{com}}(\mathbb{x})$, where $\mathbb{x}$ is the public view of the LNPLab protocol so far (as defined in Eq. (14)). As described in [BS23], the LaBRADOR proof is recursed t times to obtain a succinct proof. Our final NIZK is then FS[LNPLab]. Note that as $\mathsf{LNP}_{\mathsf{z}}$ and LaBRADOR rely on public parameters pp (as specified in Tables 4 and 5), we include these in the crs fo FS[LNPLab].

Security Analysis. Let us analyze the security of FS[LNPLab]. Correctness follows by correctness of LaBRADOR and $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$, and since FS preserves correctness. Also, zero-knowledge is straightforward. To simulate a proof, the simulator first simulates $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ via the zero-knowledge simulator[13] to obtain π'. Then, the simulator runs Fiat-Shamir compiled LaBRADOR honestly to prove knowledge of the response in π' and outputs the obtained FS[LNPLab] proof π. It is straightforward to prove that this simulator suffices and we omit details.

Let us turn toward soundness. As coordinate-wise PSS interactive proofs compose nicely as explained in [AAB+24, Section I.5], the predicate system $\boldsymbol{\Phi}$ of LNPLab is obtained by applying $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$'s predicates (as described in Lemma B.10 and extended the commitment predicates for $\mathsf{C}_{\mathrm{LaB}}$ as described above) and then LaBRADOR's predicates t times (as described in Lemma B.6). In particular, $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ is $(\mathbf{K}, \mathbf{R}, \boldsymbol{\Phi})$-coordinate-wise predicate-special-sound for relation R_{z}, where

$$\mathbf{K} = (2,2,2,3,\underbrace{2,2,2,3,\ldots,2,2,2,3}_{t \text{ times}}), \qquad \mathbf{R} = (1,1,1,1,\underbrace{1,1,1,r_1,\ldots,1,1,1,r_t}_{t \text{ times}}).$$

and for $\boldsymbol{\Phi}$ as follows:

$$\text{Level 1 to 4:} \quad \Phi_{4,1}^{\mathsf{chal}}, \Phi_{4,2}^{\mathsf{chal}}, (\Phi_1^{\mathsf{bind}}, \Phi_1^{\mathsf{prop}}, \cdots, \Phi_4^{\mathsf{bind}}, \Phi_4^{\mathsf{prop}}) \qquad (\star)$$

$$\text{Level 5i to 5i+4:} \quad \Phi_{4,1}^{\mathsf{chal}}, \Phi_{4,2}^{\mathsf{chal}}, (\Phi_1^{\mathsf{bind}}, \Phi_1^{\mathsf{prop}}, \cdots, \Phi_4^{\mathsf{bind}}, \Phi_4^{\mathsf{prop}}) \qquad (\diamond)$$

where $i \in [t]$, the predicates marked by $(\star)$ are the predicates for $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ and the predicates marked by $(\diamond)$ are the LaBRADOR predicates. Note that for the latter, the norm bound enforced by Φ_1^{prop} have slack $\sqrt{\lambda/C_2}$ (cf. Section B.4). Similarly, the binding relation is $\mathsf{R}_{\mathsf{snd}}^{\mathsf{bind}} = \mathsf{R}_{\mathsf{msis}}^{\mathrm{LNP}_{\mathsf{z}}^{\mathsf{com}}} \cup \mathsf{R}_{\mathsf{msis}}^{\mathrm{LaB}_i}$, where $\mathsf{R}_{\mathsf{msis}}^{\mathrm{LNP}_{\mathsf{z}}^{\mathsf{com}}}$ is the binding relation of $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$ and $\mathsf{R}_{\mathsf{msis}}^{\mathrm{LaB}_i}$ is the binding relation of the i-th iteration of LaBRADOR as described in Lemma B.6. Further, note that as discussed above, the relation $\mathsf{R}_{\mathsf{msis}}^{\mathrm{LNP}_{\mathsf{z}}^{\mathsf{com}}}$ has an added slack of $\sqrt{\lambda/C_2}$ and the relation $\mathsf{R}_{\mathsf{msis}}^{\mathrm{LaB}_1}$ uses $\beta = B_{\mathbf{z}}$.

As we established above that LNPLab is coordinate-wise PSS, we can apply our extractor $\mathcal{E}_{\mathsf{FS},N}$ as described in Section B.3. Let H denote the collection of random oracles due to applying Fiat-Shamir on LNPLab. That is, given some prover $\mathcal{A}$ with oracle access to H that outputs N proof-statement pairs $(\pi_i, \mathbb{x}_i)$, we run the extractor $\mathcal{E}_{\mathsf{FS},N}^{\mathsf{H},\mathcal{A}}$, and obtain $(\mathsf{aux}, \mathbb{w}_1, \ldots, \mathbb{w}_N)$ as output. If $\mathcal{E}_{\mathsf{FS},N}$ succeeds, *i.e.*, it outputs a non-$\perp$ message, then we have $\mathbb{w}_i \in (\mathsf{R}_{\mathsf{z}} \cup \mathsf{R}_{\mathsf{snd}}^{\mathsf{bind}})(\mathbb{x}_i)$ by Lemma B.3. If $\mathbb{w}_i \in \mathsf{R}_{\mathsf{snd}}^{\mathsf{bind}}$, we can embed an MSIS instance (of appropriate dimension) into the crs and reduce from MSIS. Otherwise, we have $\mathbb{w}_i \in \mathsf{R}_{\mathsf{z}}$ as desired.

It remains to analyze the success probability of $\mathcal{E}_{\mathsf{FS},N}$ which determines the knowledge error κ_{LNPLab} of FS[LNPLab]. By Lemma B.4, we have

$$\kappa_{\mathrm{LNPLab}} = N \cdot \kappa,$$

where κ is the knowledge error for $\boldsymbol{\Phi}$ for single proof extraction, chosen as in Lemma B.4. (Note that κ depends on the probability densities of $\boldsymbol{\Phi}$ given in Lemmata B.6 and B.10.) Also, $\mathcal{E}_{\mathsf{FS},N}$ invokes $\mathcal{A}$ is in expectation at most $N \cdot (K_m + Q(K_m - 1))$ by Lemma B.4, where $K_m = \prod_{i=1}^{\mu} k_i$. This concludes our analysis.

[13] It is straightforward to extend the simulator in [LNP22] to the additional commitment level in $\mathsf{LNP}_{\mathsf{z}}^{\mathsf{com}}$.

B.6.3 Parameters of the composed proof.

We do not compute exact values for the combined parameters of $\mathrm{LNP}_{\mathsf{z}}^{\mathsf{com}}$ and LaBRADOR as our purpose is primarily to prove that the NIZKs can be compressed to avoid growing in the order of the number of participants. We however give approximations based on the sizes proposed in LaBRADOR.

We first note that while setting the column size κ_{MSIS} of $\mathbf{A}_1$ and $\mathbf{A}_2$ to one (with dimension $d = 256$) is possibly too optimistic, taking $\kappa_{\mathrm{MSIS}} = 2$ should be more than enough for the MSIS security. This results in Ajtai commitments of size $512 \cdot \log q = 3136B$. Optimizations can be used to reduce the size of the output of $\mathrm{LNP}_{\mathsf{z}}$ both when running the exact NIZK (such as bimodal gaussians) and when parsing the output $\mathbf{z}$ as a witness to LaBRADOR by first decomposing it in a smaller base, however the results from [BS23] show that proof size grows very slowly with the size of the witness (the parameters given in [BS23] change the witness size simultaneously with the number of constraints). In our case if we were to binary decompose the witness $\mathbf{z}$ output by the $\mathrm{LNP}_{\mathsf{z}}$ prover to obtain constraints of the same form as [BS23] we would obtain a witness of dimension much less than 2^{15} (in fact closer to 2^{12}) and a very small number of constraints. It is however not obvious how our larger modulo q impacts the parameters nor how small the compressing commitments used in the $\mathrm{LNP}_{\mathsf{z}}$ algorithm can be. Given the very slowly scaling nature of LaBRADOR a conservative estimate for the final proof would be their highest parameter set of 2^{25} constraint resulting in a proof size of less than $60kB$. It is worth noting that with this proof size the main contribution to the communication cost will quickly become the commitments to the masks $\mathbf{m}_{i,j}$.